

The Shadow Files

Volume 3
Hard Targets

YEHOR SELIN

Contents

Before You Begin	4
When the Trace Is Buried	6
How Volume Three Works	9
A Bridge from Volumes One and Two	12
The Ghost in the Cache	16
Reconstructing a Deleted Profile	34
Reading a Photograph for a Place	51
Reading a Photograph for a Time	68
Domains and Registration History	82
Certificate Transparency	100
From Infrastructure Back to a Person	117
Passive DNS and Fingerprints	135
The Full Hard Target	154
What You Can Do Now	179
Appendix A — A Working Toolbox (2026)	182
Appendix B — Templates	188

Appendix C — Red Lines	199
Appendix D — Take It Back	203
Glossary	208

Before You Begin

This is the third volume of The Shadow Files — a practical course in open-source intelligence, OSINT. It carries straight on from Volumes 1 and 2. If you have not read the foundations — the loop, the confidence ladder, the discipline of stopping at the line — and the cross-platform work — walking a person across the internet without losing them — read them first. This book assumes both.

Everything here is invented

Every name, username, email address, phone number, photograph, domain, IP address, WHOIS record, certificate, coordinate and case study in this book is entirely fictional. Each one was built to teach a method — never to describe a real person, a real site, or a real place. Any resemblance to a real individual, account, domain, or event is pure coincidence.

Because every detail is invented, the examples are shown in full — complete-looking records instead of blanked-out symbols — so you see exactly how a real trail behaves without any of it touching a real target.

The rules, in one line

Open sources only. Nothing that needs a login, a password, or a way around a paywall. Respect people's privacy and the law. That is the whole frame, and it does not loosen because this book gets harder.

Two things worth keeping in mind. Volume 3 teaches you to read what people thought they had erased and to place them in the world by things they never captioned. Both feel invasive to a beginner and they are — used badly. Know the law where you work: reading a public archive is not the same as tricking a service into giving up private data, and geolocating a picture someone posted publicly is not the same as tracking a person's movements. Second, most of the damage in this craft is done to people who were never the subject — the neighbour caught in a shadow, the family member behind a redacted WHOIS, the child in a photo's background. Take only what answers your question and stop there.

A professional is not the person who can find the most. It is the person who knows where to stop.

When the Trace Is Buried

What changes in Volume Three

In Volume 1 you learned to pull one thread. In Volume 2 you learned to weave threads from many platforms into one confident identity. Both books assumed the same thing: the person, or the account, or the photo, is *there*. You just had to find it and read it well.

Volume 3 is about the cases where it is not there — at least not where a normal search would look.

The account was deleted three years ago and the platform says the profile "does not exist." The photograph has no caption, no metadata, no obvious landmark, and the person who posted it will not answer questions. The domain is behind a privacy proxy and the registrant is a string of Xs. The operation is not on Instagram at all — it is on a server in a rack somewhere, and the only thing pointing at it is a certificate a machine issued at 3 AM last April.

This is where beginners quit and where professionals start. Because none of it is really gone. It is *displaced* — pushed off the obvious surface and onto a different one: an archive, a shadow on the ground, a certificate log, a favicon hash. You have to know the other surface exists, and you have to know how to read

it.

Who this book is for

You, if the first two volumes are second nature and you want to work the cases where the easy pivots run out. You do not need to be a network engineer or a photographer. You need patience, a taste for slow work, and the mindset the first two books built. If a technical word ever trips you up, there is a glossary at the back — flip to it any time.

What you will be able to do by the end

Reconstruct an account after it has been deleted. Place a photograph on a map from the shadows in the frame. Date it from the weather and the sun. Follow a domain from its first WHOIS record to whoever forgot to redact themselves in 2016. Read a certificate transparency log the way you read a comment thread. Turn infrastructure back into people. And — the skill that ties all of it together — know, still, exactly when to stop.

What this volume is not

It is not a manual for breaking into systems. Nothing in here needs a login, a password, or a way past a lock. Every technique reads records the world already made public — for administrative reasons, for security reasons, or by accident — and reads them properly. The professional's world of due diligence, chain-of-custody reports, counter-investigation and automation waits for Volume 4. Here we master one thing well: the hard targets.

If you cannot see the target directly, look at what the world around it recorded.

How Volume Three Works

The series, this book, and how each chapter is built.

The Shadow Files is a series of four volumes. Each one takes you a level higher, and each assumes only what the last one taught.

- **Volume 1 — Foundations.** The mindset, the rules, and your first real skills.
- **Volume 2 — Cross-Platform Attribution.** Following one person across social networks, messengers, photos, and leaks — into a single, confident identity.
- **Volume 3 — Hard Targets.** Deleted accounts, geolocation, chronolocation, and the infrastructure behind operations.
(This book.)
- **Volume 4 — The Professional.** Corporate work, reports, counter-investigation, and turning skill into a craft.

A note on scope: this volume finally goes past the profile. Half of it is still about people — the ones who thought they had erased themselves, the ones you place through a window they never captioned. The other half is about the things people build and forget to hide: domains, hosts, certificates, trackers. But even the infrastructure chapters end where every chapter in this series ends — at a person.

Inside this volume

1. The Ghost in the Cache 2. Reconstructing a Deleted Profile
3. Reading a Photograph for a Place 4. Reading a Photograph
for a Time 5. Domains and Registration History 6. Certificate
Transparency 7. From Infrastructure Back to a Person 8. Passive
DNS and Fingerprints 9. The Full Hard Target

How every chapter is built

The same rhythm as Volumes 1 and 2, so you always know
where you are:

- The idea — in plain words, no jargon.
- The method — how a professional actually thinks and works, step by step.
- A worked case — a short, invented investigation you can follow, with the dead ends left in.
- An exercise — something safe and legal you do yourself.
- A recap — the few things to remember.

One promise about tools

As before: method first, tools second. Some tools named here may be gone by the time you read this, and new ones will have taken their place. That is fine. The *thinking* does not change. Learn the method, and any tool is just a faster way to do something you already understand. Volume 3 is more tool-heavy than Volume 2 by nature — you will read certificate logs and reverse-DNS databases, and those are literal services — but the *questions* you take to those services are the same three questions you have been asking since Volume 1: what is exposed, what can I pivot from, what am I attributing to whom.

A Bridge from Volumes One and Two

The foundations, in one page, before we go deeper.

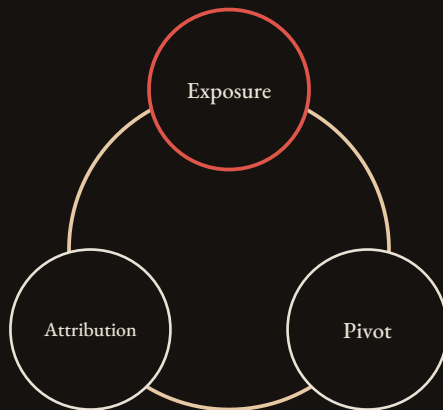
Before we go anywhere new, let me put the foundations back in your hand — quickly. If any of this feels unfamiliar, that is your sign to go back to the earlier books before you go forward here. Everything in this volume leans on these four things.

The loop

Every investigation, however weird the surface, is the same three moves, run in a circle.

Exposure → pivot → attribution.

THE LOOP



one clue, run in a circle until it points at a person

The three moves run as a cycle — attribution feeds the next exposure.

Exposure is a trace someone or something left in the open. Pivot is turning that trace into the next one. Attribution is when enough traces line up to point at one real identity — a person, a group, an operator. In Volume 3, the exposure is often historical (a snapshot, a log entry, an old record) rather than live. The loop does not care. You run it the same way.

The confidence ladder

A match is not a fact. We always say *how sure* we are, in four fixed words:

- weak — one unconfirmed clue. A lead, never a conclusion.
- moderate — two independent clues agree.
- strong — several independent signals that are, together, hard to explain by any rival hypothesis. Usually three or more. Two heavy, diagnostic ones can reach strong on their own. Three weak, common ones may not.
- confirmed — a hard unique identifier or a genuine self-link, independently nailed down so anyone could re-check it.

THE CONFIDENCE LADDER



Four rungs you climb only on independent signals: weak, moderate, strong, confirmed.

The ladder matters more in Volume 3 than anywhere. You are reading traces that were not designed to identify anyone — a shadow, a serial in a WHOIS record, a favicon hash — and it is very easy to mistake a suggestive shape for a proof.

Independence, and why it is hard here

Two facts from the same underlying source count as one vote. In Volume 2 that was mostly about not double-counting a self-linked handle. In Volume 3 the trap is larger: a WHOIS record, the reverse-DNS on the same host, and the certificate that host serves are often *three views of the same file* — one operator's setup — not three independent signals. Learn to ask, every time: *would these two facts still both be true if my theory were wrong?* If the answer is no, you have one signal, not two.

The line, and where to stop

Open sources only. No logins, no bypasses, no scraping past a paywall or a robots file, no talking to the target under a false name. Volume 3's methods make it very easy to keep going — a domain leads to a host leads to a customer leads to a house — so name the legitimate question you are answering, in one plain sentence, before the first pivot. If you cannot, you have curiosity, not a case.

Foundations in hand? Good. Let us go into the buried places.



CHAPTER ONE

The Ghost in the Cache

When a target hits delete, they only delete the copy they own.

A city councilman scrubs a campaign page on a Friday night. By Sunday afternoon the page is gone from every place he thought to look, and he sleeps a little easier. He types his own name into a search bar and sees only clean results. He tells his chief of staff the problem is handled.

He is wrong, and not because he missed a setting. He is wrong because he only ever controlled one copy.

A beginner opens the same URL on Monday, sees a 404, writes "deleted" in the notes, and closes the tab. That is where most

cases go to die — not because the work is hard, but because the word *deleted* sounds final and the internet has trained us to trust it.

A professional opens the same URL, sees the same 404, and does not feel anything much. The page under his control is not the interesting page. The interesting pages are the ones he never owned — the archive snapshot from 2020, the supporter's blog that quoted three paragraphs verbatim, the news aggregator that cached the preview card, the neighbour who screenshotted a post to argue with it, the RSS feed that pulled the headline into a database nobody has cleaned in six years. Each copy has its own owner, its own schedule, its own decay curve. None of them heard about the delete on Friday night.

That gap between what the target believes and what the ecosystem remembers is the whole subject of this chapter.

The idea: your delete only touches your copy

Publishing on the modern web is closer to sneezing in a crowded room than to writing in a private diary. The moment you post, other systems reach out and take copies for reasons that have nothing to do with you: to index, to preview, to archive, to embed, to quote, to argue with. Each copy lives on somebody else's schedule. When you press delete, you talk only to the one server that trusts your login. The others do not get the memo, and most of them would not act on it if they did.

That is the principle. Everything else in this chapter follows from it.

Deletion under the user's control is not deletion in the ecosystem.

The useful mental model is **cache decay**. Every copy of a page has a half-life — some short, some absurdly long. A search engine's preview text may refresh in days. A large web archive may hold a snapshot for a decade or more. A friend's screenshot has no expiry at all; it lasts until the friend cleans out their phone, which is to say, effectively forever. If you understand the decay curves, you understand where to look and how long you have.

THE CACHE TREE



delete removes the trunk — the branches keep their own lives

A single deleted page fans out into layers with wildly different half-lives — some measured in hours, others in decades — and

your delete button only ever reaches the trunk.

Two things follow. No single layer is complete — if you find nothing in Wayback you have not looked, you have looked in one place. And the layers age at different rates, so a case cold in 2026 may still be warm inside an aggregator nobody has pruned since 2019.

Where copies actually live in 2026

There are seven layers worth checking before you call something gone. Learn them as a checklist. Do not skip a layer because the last case did not need it — different targets bleed into different layers.

- **General web archives.** The Wayback Machine and archive.today are the workhorses. Wayback runs on scale and automation; archive.today runs on people saving pages by hand, and it captures things Wayback often misses (paywalled news, single tweets, pages a site owner asked Wayback to hide). Different tools, different coverage — check both.
- **Targeted archival by researchers and activists.** Small snapshot rings maintained by academics, journalists, and hobbyists — political-ad monitors, extremism trackers, image-board mirrors, court-document mirrors. They are narrow but deep, and they often preserve things the big archives never touched.

- **Search-engine caches.** Google's cache: operator was quietly retired in 2024 and the cached-page link no longer appears in results. Bing kept a version of the feature longer but has thinned it. Treat search-engine cache as a bonus layer, not a reliable one — most days it is empty.
- **Front-end mirrors and read-only frontends.** For a few years there were lightweight read-only frontends for the big platforms — the Nitter pattern for Twitter, Bibliogram for Instagram, Invidious for YouTube. Most of the public instances are dead by 2026 as the platforms tightened their APIs. The *pattern* still recurs: whenever a platform is hostile to logged-out reading, someone builds a scraper mirror. Some of those mirrors quietly preserve content the source has since deleted.
- **Syndication and RSS aggregators.** News scrapers, feed readers, and email-digest services pulled the page as it went live and stored the headline, the preview text, and often the opening paragraphs in their own database. Many of these were built in 2011 and never garbage-collected. Their search boxes are old and slow and they are goldmines.
- **Third parties who quoted or screenshotted.** A supporter's blog copies three paragraphs. A critic pastes the whole thing into a forum thread to argue with it. A journalist screenshots a claim for a story. None of these delete when the source does — they have their own owners and their own reasons to keep the copy.

- The mentions the delete never reached. Comments the target left on other people's posts. Photos in which the target was tagged. Reply threads. Quote-posts on other platforms. Embed caches on news sites that pulled the widget. Share URLs sent in a group chat two years ago. The target deleted a page; they did not delete their own footprints across everyone else's pages, and nothing about pressing delete on the trunk touches the leaves.

What still works in 2026, and what quietly died

Older guides — including the first edition of this book — leaned on a handful of moves that no longer behave the way they used to. Dating an investigator faster than a bad haircut is confidently running a move the platforms closed years ago. So learn the idea underneath each dead trick, and go find the current equivalent.

- **Google cache:** is gone. For twenty years you could type `cache:example.com/page` and read a Google-hosted copy of the page. The link vanished from search results in early 2024 and the operator itself no longer works. The principle holds: search engines still ingest pages and still hold internal copies for ranking. What you can still see is the *snippet* — the two lines under the blue link — which is often a preview of text the site has since removed. Read the snippets carefully; they are a cache in miniature.
- **Named front-end mirrors** are mostly dead. The public Nitter instances that were passed around in Slack channels

through 2022 and 2023 are largely gone; the last of the Bibliogram instances rotted out around the same time. The principle holds: whenever a platform locks down logged-out reading, someone builds a lighter frontend that scrapes the public parts. Do not memorise the name of a service, memorise the *shape* — "read-only mirror of platform X" — and go looking for the one that survives this year.

- **Wayback is not complete, and it is not permanent.** Two assumptions to unlearn. Wayback does not have every page — many sites are excluded, and its coverage of social platforms is thin and thinning. Coverage also decays for reasons that have nothing to do with robots.txt — the Internet Archive stopped honouring that retroactively back in 2017. Snapshots disappear because site owners fill in the exclusion form, because DMCA and similar takedowns pull specific captures, and because the big platforms fight crawlers hard enough that even the snapshots that do exist are often a JavaScript shell, a login wall, or a rendering error dressed up as a page. The principle holds: archives remember what the target forgot. But not everything, and not forever — check sibling archives, and record what you found while you can still see it.

Before you look

Two stop signs, both worth respecting.

Archives are not passive. Wayback logs requests. archive.today logs requests. Rate-limits exist and will bite if you hammer them, and a burnt IP can cost you a case. Move at human speed. If a page has three snapshots, load the three snapshots — do not scrape the whole calendar. And never request a fresh capture of a live target's page unless you have thought hard about what you are doing. Three reasons. First, it is a *tell* — some archive services fetch the URL from an IP the target's site logs, some trigger link-preview crawlers that show up in the target's analytics, and a careful subject watching their referrer logs sees a stranger asking about them by name. Second, you are permanently preserving content the target may have been counting on to decay; in a protective-order, source-protection, or anti-stalking context that is not neutral, it is an act you took on the record. Third, in some jurisdictions deliberately preserving a live individual's material in a harassment or restraining-order matter can be argued as interference with their rights over their own data. Not legal advice; the working rule is the same either way. Read what already exists; do not conjure new evidence into being without a real reason to.

The second stop sign is OPSEC on your own side. Use a clean research browser with no logged-in Google — Google's cache-adjacent products log every URL you fetch against your identity, and research history mixed into personal search history is a leak waiting to happen. Turn off sync. Use a profile that does not carry your real cookies. Boring hygiene, and it matters

most on the day you forget it.

The method, step by step

Step 1 — Fix the URL forms. A page has more addresses than the target ever thinks about. The canonical form (example.com/post/1234), the vanity form (example.com/@dale/purnell-for-district-4), the share form (example.com/s/aB3xQ), the embed form (example.com/embed/1234), the mobile subdomain (m.example.com/...), the AMP or reader-mode variant, the deep link into a specific comment. Different caches indexed different forms. A Wayback query on the canonical URL may return nothing; the same query on the share URL may return five snapshots. Write out all the plausible forms before you query anything.

Step 2 — Walk the Wayback timeline. Open web.archive.org/web/*/<url> and read the whole calendar, not just the latest capture. The most recent snapshot is often the least useful — it may be captured *after* the deletion, showing the 404. Older snapshots are usually richer, taken when the target was still publishing openly. Note the *capture time* on every snapshot you save; that is the timestamp that matters, not today's date.

Try it (2026). Pick any public account you follow — your own is fine. Open web.archive.org/web/*/<their profile URL> and

read the timeline top to bottom. Notice how coverage clusters around events (a viral post, a news mention) and thins out in the quiet months. Note the last capture before the account changed in any meaningful way — a rename, a bio rewrite, a going-private. That last-before-change snapshot is the one you would want in a real case, and finding it should feel routine, not clever.

Step 3 — Check the sibling archives. [archive.today](#) first — it captures things Wayback refuses or misses, particularly single social posts and paywalled news. Then, depending on the target's country and topic, check the specialist archives: Perma.cc for anything a lawyer or academic might have preserved, the UK Web Archive for .uk domains and British public bodies, Arquivo.pt for Portuguese and lusophone content, national library web archives elsewhere. Mentioning them here as *shape*, not endorsement — the point is that "web archive" is not one service, it is a family, and different members caught different things.

Step 4 — Read what the search engines left behind. The cache: link is dead. The snippet is not. Run `site:example.com "<rare phrase from the deleted page>"` and read the two-line preview under each result. Preview text is generated from an internal snapshot the engine has not necessarily refreshed. If the target had a distinctive turn of phrase — a slogan, an unusual name, an oddly specific number — quote-search it across the open web

and see where else it landed. A rare phrase is a fingerprint; a common one is noise.

Step 5 — Follow the third-party quoters. If a public figure said something quotable, someone quoted them. Reverse-image the missing avatar to find blogs and forum posts that embedded it. Quote-search a distinctive sentence from what you remember of the page — from a screenshot in a news story, from your own notes, from a critic's rebuttal. Aggregator sites, opinion blogs, and single-topic forums are unglamorous and rich.

Step 6 — Sweep the ecosystem copies. The delete did not touch comments the target left on other people's posts, tagged photos, quote-replies, or share URLs pasted into public group chats. On social platforms, search the target's *handle* rather than their profile — you get every mention, and every mention is a fragment of the page they took down.

Step 7 — Freshness discipline. For every claim in your notes, record the URL of the *specific snapshot* you got it from and its *capture time*. Not "found on Wayback." A snapshot URL with a timestamp is a citation; "Wayback" is a rumour. When you rank a claim on the ladder — weak / moderate / strong / confirmed — make sure each supporting signal points to a snapshot, not a memory.

A worked case, dead ends and all

Dale Purnell — fictional, midwestern, city council district four — ran an unsuccessful re-election campaign in 2019 on a Facebook page called "Purnell for District 4." After a scandal in early 2026 involving a contracting decision from his old term, the page was quietly deleted along with his personal account. A reporter at the local weekly, chasing what Purnell had actually promised voters in 2019 about that contract, asks whether any of the campaign page is recoverable. The URL she has is facebook.com/purnellfordistrict4.

The Google cache dead end. First instinct — old muscle memory — was `cache:facebook.com/purnellfordistrict4`. Google returned a "no results" page. The operator is dead; the muscle memory is a bad habit. Move on.

The Wayback dead end. Wayback had three snapshots. Two were the JavaScript shell — Facebook rendered client-side, and the snapshot captured the loading spinner and nothing else. The third was more recent than the others and hit a different wall: by the time Wayback's crawler reached the page, Facebook's own anti-crawler defences had returned a login prompt and a rendering shell, and that is what got preserved. Earlier captures on this domain have thinned out for the ordinary reasons too — site-owner exclusions and takedowns work retroactively even when `robots.txt` does not.


```
wayback listing for purnellfordistrict4
```

```
-----
```

```
* facebook.com/purnellfordistrict4
```

```
snapshots found: 3
```

```
2019-10-14 captured (JS shell)
```

```
2019-11-02 captured (JS shell)
```

```
2023-04-18 unusable (auth wall)
```

```
-----
```

```
0/3 usable
```

Three snapshots, three walls.

The archive.today dead end. archive.today had one snapshot, from October 2019. This one actually rendered — a real page, real posts, real pinned content at the top. Except the snapshot was of the profile view, not the individual post view, and the pinned comments under each post — where in 2019 Purnell had answered voter questions about the contract, sometimes at length — were collapsed and not captured. The snapshot preserved the wall and lost the conversations.

The turn. The reporter had one thing to work with: she remembered a phrase from Purnell's speeches — "prudent stewardship of the district's dollars" — that had shown up on the page verbatim more than once. A quote-search for that phrase, restricted to blogs and news, was the crack.

dork: "prudent stewardship of the district's dollars"

-
1. citizenswatch-oakhaven.blog 2019-09
"quoting Purnell's page in full below..."
 2. district4-neighbors forum 2019-11
thread: "what did he actually promise"
 3. oakhaven-weekly.com/rss.xml 2019-10
preview: "...prudent stewardship..."

3 hits, all 2019, all outside FB

The phrase carried where the URL did not.

The first hit was a citizens' watchdog blog that in September 2019 had pasted three paragraphs of Purnell's contract-policy post in full — the blogger wanted to argue with them line by line, and preserved them line by line in the process. The second was a neighbourhood forum thread titled "what did he actually promise," in which a member had screenshotted a post and someone else had transcribed the text under it. The third was the RSS feed of the local weekly's own aggregator, which had pulled Purnell's Facebook posts into a sidebar widget in 2019 and had never cleaned the database — preview text of six posts still sat there, addressable by URL if you knew the pattern.

Ranking. The blog quote is strong on its own for the paragraphs it quotes — it is verbatim, it is timestamped September 2019 on the blog's own archive, and the blogger had no motive to fabricate a target he was already criticising in his own words. The forum screenshot plus transcription is **moderate** — a screenshot can be edited, and the transcription

introduces a second point of failure — but where it overlaps the blog quote it is confirmed, and where it extends into pinned comments the blog did not touch, it stays moderate. The aggregator RSS is moderate for preview text and no more — a preview is a fragment, and fragments can be misleading without the surrounding paragraph.

The reporter got a partial reconstruction of the deleted page, cited to three independent sources with capture times, and a clear line about which paragraphs she could quote as *what Purnell wrote* (strong) versus *what he appears to have written* (moderate). Purnell's delete had reached the trunk. It had not reached the blog, the forum, or the aggregator, and it never would.

A second case: the one that doesn't close

A different request the same month. A private Instagram account, deleted in mid-2022, allegedly belonging to a witness in a civil matter. No archive coverage — Wayback had zero snapshots on the vanity URL and none on the numeric ID either; archive.today had none; the specialist rings that mirror Instagram content had never seen the handle. The avatar was cached in one third-party analytics tool's public preview, but the image was a generic sunset and reverse-search returned only stock. Nobody had quoted the account. No screenshots surfaced on the open web. The neighbourhood the account had posted from was small enough that a forum search should have turned up *something*, and it did not.

Report to the client: could not attribute. No layers paid out. I gave them the list of layers I had checked, the queries I had run, and the capture times of the negative results, so that if the case reopened later they would know what had already been tried. The mess is the job, not a mistake.

A safe exercise

Pick a public news article from three or more years ago on a story you remember — a national event, a local election, anything with real reporting. Open Wayback and walk the article's capture timeline. Read the earliest capture, a middle one, and the most recent. Notice the corrections, the changed headlines, the edited paragraphs, the photos that were swapped or removed. Note the capture time on each. You are not investigating anyone; you are training your eye to see how a single URL is really a family of versions, and how the version you find today may not be the version somebody read on the day it mattered. That reflex — *which version am I looking at, and when was it captured* — is the whole discipline in miniature.

Recap

- Deletion under the user's control is not deletion in the ecosystem — the copies you never owned keep being copies.
- Every copy has a decay curve; some layers refresh in days, others hold for decades.
- Check all seven layers before calling a page gone; do not skip a layer because the last case did not need it.
- Google cache: is dead; front-end mirrors mostly are; Wayback is neither complete nor permanent. Learn the principle, not the tool.

- Move at human speed on archives; never request a fresh snapshot of a live target's page unless you know what you are trading.
- Fix all the URL forms before you query; different caches indexed different forms.
- Record the *capture time* of every snapshot in your notes, and cite the specific snapshot URL, not the archive's homepage.
- Some cases do not close, and reporting "could not attribute" with the layers you checked is a real result.

What comes next

The next chapter turns from deleted *pages* to deleted *profiles* — accounts that were closed, renamed, or scrubbed by the person behind them, and how much of a person still stands after the account they used is gone.



CHAPTER TWO

Reconstructing a Deleted Profile

An account leaves the platform. The person does not leave the ecosystem.

You type the handle into the address bar and the page loads for half a second before the platform decides. *Sorry, this page isn't available.* No avatar. No banner. No 404 with a friendly cartoon. Just the generic wall the site shows when an account has been deleted, deactivated, or scrubbed by the trust team.

Most people close the tab. Some try a different capitalisation of the handle. A few think to check a cache, get one hit, and stop

there.

You are going to do more than that, because the wall you just met is a wall on *one* storefront. Behind it, the person who ran that storefront was tagged in wedding photos, quoted in a local newspaper, added to a group by an old colleague, and named in a chamber of commerce dinner post. None of that lived on their profile. It lived on other people's profiles, on other people's servers, in caches that answer to nobody they know. The account is gone. The person is not.

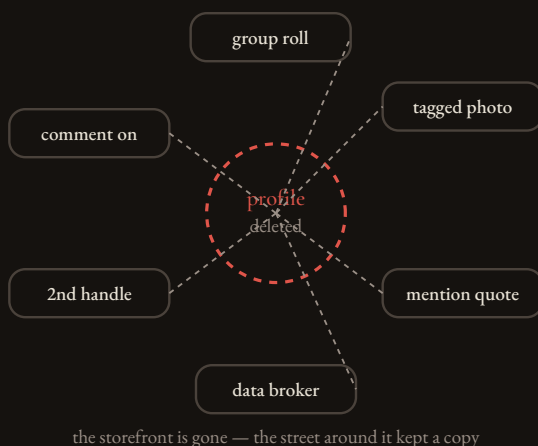
This chapter is about rebuilding the person from what the ecosystem kept. It is a different job from Chapter One. There you were rescuing a specific *post* someone took down. Here you are rescuing an *identity* someone tried to close. The techniques rhyme. The friction is heavier, and so is the ethics.

The idea: the person is not the profile

Think of the deleted account as a rented storefront on a busy street. While the tenant was there, they leaned out the window, waved at neighbours, joined the street's mailing list, appeared in the background of other shopkeepers' photos, argued with a customer who wrote about it in the local paper. When they hand the keys back, the landlord paints over the sign. That is what "account deleted" means: the storefront is empty and the sign is gone.

Everything else stays. The neighbours still have their photos. The mailing list still has the old address. The paper still has the article. The customers still remember. The landlord — the platform — can only decide what to do with the shop itself. They cannot walk down the street erasing everyone else's memory.

THE DELETED RING



The centre is a shuttered storefront: the profile that no longer resolves. Around it, still lit, are the neighbours' walls that tagged the shop, the group rolls that listed it, the archives that snapshotted it, the data brokers that sold a copy of its sign years ago, and the second account down the block that nobody thought to close.

Deleting the storefront does not delete the street. Your job is to walk it.

What survives when a profile dies

The exact catalogue shifts by platform and by year, but the shape holds. When the profile itself is gone, you are usually left with some subset of the following.

- **Tagged photos on other people's walls.** A photo the target was tagged in lives on the *poster's* page. The tag becomes a "user unavailable" label after deletion, but the poster's caption often still names the person in plain text, and the face is still in the frame.
- **Comments the target left on public content.** Threads on public news pages, brand posts, community pages, and creator posts frequently keep old comments intact after the commenter's account dies. The username collapses to a placeholder; the text, and often a cached avatar thumbnail, remains.
- **Group and page membership rolls.** Membership lists, event RSVPs, page moderator lists, and admin logs live on the group's side. Cleanup after a user deletion is uneven — you will still see the name in "past attendees," in old admin credits, and in event photos.
- **Cross-post quotes and embeds.** A tweet embedded in a 2019 blog post is a copy of that tweet, served by the blog, not the platform. A YouTube community-tab share that a channel reposted to their newsletter is a copy in the newsletter. Deleting the original does not reach into the

embed.

- **Platform-data breaches that predate the deletion.** The 2019 Facebook scrape, the 2021 LinkedIn scrape, the various forum-database dumps that circulate in leak markets: if the person was on the platform on that date, their record is in that dump and will stay there. Handle with care and only for lawful purposes.
- **Data-broker snapshots.** People-search sites, business-registration mirrors, and directory scrapers routinely captured public profiles years ago and never re-checked. A profile deleted in 2024 can still be reproduced from a broker's 2019 snapshot, often with the old bio, old employer, and old photo attached.
- **Friend-graph residues on other people's accounts.** The target's own friends list is gone with them, but on platforms where friend lists can be left public, they still appear inside *other people's* lists as an unnamed placeholder or as their old display name preserved in cached copies. A cluster of open profiles that all silently list the same missing friend is a fingerprint.
- **Second and legacy accounts on cross-platforms they forgot about.** People delete the loud account and leave the quiet ones running: an old DeviantArt from art school, a Goodreads under a pseudonym, a Strava, a Letterboxd, an old blog on a defunct host. These rarely get swept up in a "delete my presence" pass.

What quietly stopped working

Three moves that older guides taught confidently no longer do the work they used to. Learn what died and what did not.

- **Facebook Graph Search.** For years you could enumerate friends of a named person, photos taken in a place, posts liked by a user, all through structured search queries. Facebook removed Graph Search in 2019. **The principle holds:** the friend graph is still the shortest map from a deleted account to the person's real life. You just cannot query it centrally anymore. You reach it by walking the open friends by hand.
- **LinkedIn's public "people you may know" pages.** These used to be crawlable without login and gave you the 2nd-degree map of a target directly. They sit behind an account wall now, and probing them from a logged-in account leaks *you* into the target's suggestion network (the bidirectional PYMK trap covered in Volume 2). **The principle holds:** the 2nd-degree ring still betrays a hidden 1st degree. You just have to build it from other public signals rather than a scraped page.
- **Bulk friend-list scrapers.** Technically feasible on some platforms, categorically a breach of their Terms of Service, and unhelpful in court. We do not do it. **The principle holds:** the target reappears across the open lists of people who knew them. You reach it by manual reading of what those people chose to leave public, not by scripting a harvest. Any tool or

service that promises otherwise is selling you a lawsuit.

If a technique in this chapter names a specific site or endpoint, assume it may be dead by the time you read this. The shape of the move outlasts the door.

Before you look

Rebuilding a deleted identity leaves more traces on more people than almost any other work in this book. The friend whose wedding album you load ten times is going to see a spike in their post's view counter. The old blog you fetch three times from three different snapshots is going to show a small pattern in its logs. If your work is sensitive, treat every fetch as a visible event on someone else's property, and act like a guest who is aware they are being noticed.

Use a proper research browser — a separate profile, no personal logins, no contact syncing, no address book access, no telemetry to your real identity. The bidirectional "people you may know" problem from Volume 2 applies with full force here: probing the target's old ring from a logged-in research account can surface *your* research account to the very relatives and colleagues you are quietly reading. That is how sock accounts die and cases burn.

And the ethics. A person who deleted their account chose, on the record with that platform, to be forgotten by it. Undoing that choice is not neutral. Have a real reason you can say out

loud — a lawful investigation, a public-interest story, a fraud you are documenting, a person you are trying to reunite with a family member — and take only what answers that question. A deleted profile is a stronger privacy signal than a locked one, and it deserves a stronger justification before you walk around it.

The method, step by step

Step 1 — Anchor what you already know. Write it down: the target's real name in the forms you have seen it spelled, the last handle you knew them by, a face if you have one, a city, an employer, a known associate, an approximate date the account went dark. This is your search kit. Everything below pivots off it, and every match you find gets tested against it. Anchor first, always.

Step 2 — Cache pass. Before anything hard, run the deleted profile URL through the archives — the Wayback Machine, the search-engine caches, the smaller mirrors covered in Chapter One. This is the cheap check that occasionally answers the whole question in ten seconds. Do it and move on; do not spend the day here.

Step 3 — Reverse-image the last known avatar. If you can lay hands on *any* copy of the old avatar — a friend's tag, a group's cached member card, a data-broker snapshot from 2020, the last archived version of the profile — reverse-image search it across two or three engines. A fresh account under a different name

using the same face is one of the most common "I deleted my presence" patterns in real life: people rarely take a new photo just because they took a new handle.

Step 4 — Walk the mention ring. Search the target's real name across the open web with the platform stripped out: "Ilse Vroeg" -site:facebook.com -site:instagram.com. Read what other people wrote about them — press quotes, blog posts, forum threads, local news, community minutes. If you know a rare phrase they used in their old bio, quote-search it. If they had a rare middle name or a rare workplace, include it.

Try it (2026). Pick a person you know personally who deleted a social account years ago — and who is comfortable with you doing this as an exercise. Do only Step 4 on them. In a clean browser, run their real name plus their city, in quotes, with -site: filters for the platforms they left. Read the first two pages of results and list every mention of them on someone *else's* property: a friend's blog post, a school newsletter, a race result, a wedding announcement, a chamber directory. Stop there. Show them the list. Ask what they thought was gone, and what surprised them. That reaction — usually a quiet "oh" — is the whole lesson.

Step 5 — Group-side rolls. The target may have left the group; the *photo they posted* to the group's shared album and the *comment they left* on someone else's post inside the group very often stay. Check every group and page you can associate them

with: past-attendee lists on event pages, "founding members" credits on community pages, comment threads on group announcements from the year they were active. Membership metadata is maintained on the group's side and cleaned up unevenly.

Step 6 — 2nd-degree map. Pick two or three known associates whose own connections are still public. Read *their* connection lists by hand and note who recurs. A cluster of five people who all seem to know a person who no longer has an account — and whose comments on old posts refer to that person by first name — is a strong signature of a real cluster. Manual review only. No scripts, no intersections, no scrapers. What one careful reader can hold in their head is exactly the amount of work the rules permit.

Step 7 — Old-data-broker reads. People-search sites and directory mirrors often preserve a profile snapshot from long before the deletion. Read them for what the target used to say publicly about themselves: bio, employer, city, associated addresses, associated phone numbers. Treat every field as a lead, not a fact. Brokers are notoriously wrong — the wrong middle initial, the wrong city, a "known associate" who is actually a stranger with the same surname — and the whole file is often stitched together from three people with similar names. A broker match earns you a hypothesis to test elsewhere. It does not earn you a conclusion.

Step 8 — Rank every fragment on the ladder. Line up what you found, weight it, and be honest. A tag in a wedding photo is moderate. A tag plus a broker record that names the same employer plus a quoted bio on a chamber site is strong. Three brokers echoing each other is one broker: they all bought from the same wholesaler. Weight before you write.

A worked case, dead ends and all

A regional newspaper in a small European town asked for help on a story about a charity board that had voted out its treasurer in a public row in mid-2022. Fictional names, real method. The treasurer, Ilse Vroeg, deleted her Facebook and Instagram the same week the vote made the local paper. The reporter's question was narrow and answerable: before the row, what had Vroeg publicly claimed about her professional credentials? Specifically, had her old bio described her as a chartered accountant or as a financial consultant? The distinction mattered because the first is a regulated title in her country and the second is not.

Anchor. Full name, town, the charity, the month of deletion, and one known associate — a fellow board member named in the paper. No known photo yet. Write it down.

Cache pass. The Wayback Machine had one snapshot of her Facebook profile, taken in early 2022. It opened. It was the platform's private-view HTML shell — the "you must log in to

see this content" wall, captured at snapshot time. No bio. No text. Nothing.

```
Wayback lookup: facebook.com/ilse.vroeg
```

```
-----  
snapshots 1
```

```
2022-03-14 captured
```

```
opening 2022-03-14 ...
```

```
HTML = platform login shell
```

```
bio = not present in capture
```

```
posts = not present in capture
```

```
-----  
snapshot exists, content does not
```

A snapshot of a wall is still a wall.

Dead end two. She had been listed as a member of a public-facing local business group. The group's member roll still existed on its own page. Her row was intact — but the name field read *user deleted* and the profile link was dead.

Membership evidence, no bio.

Dead end three. A LinkedIn URL under her name, cached in an old search result, 302 redirected to "profile not found." Not a snapshot, just a stub of a page that used to exist.

What actually held. Walking the mention ring on her real name plus the town, with the two platforms stripped out, turned up three useful things.

First, a regional business-directory scraper — a broker that grazes chamber sites and republishes bios — still hosted a page titled with her name and the charity, last updated in 2021. The page

reproduced what appeared to be a copy of her old Facebook "About" text word for word, including a two-sentence self-description. Broker data, so I treated it as a lead. But I searched the exact phrase in quotes across the open web to see whether the wording appeared anywhere she had written herself.

```
Mention ring, quoted phrase
-----
QUERY  "financial consultant to
        several non-profits in"
        "Vroeg"
results 3
1) business-directory scraper
   (broker copy, 2021)
2) charity's own 2020 newsletter
   (PDF on their website)
3) local blog "chamber notes"
   (2020 post, quoted her bio)
-----
same wording in three places, two
of them non-broker sources
```

Two independent non-broker copies of the exact same wording.

The charity's own 2020 newsletter — a PDF still hosted on the charity's website — quoted her bio verbatim in a "meet the board" section. A local blog covering chamber-of-commerce events had quoted the same paragraph in a 2020 event write-up, attributing it to her introduction at the podium. Both used the phrase "financial consultant". Neither used the phrase "chartered accountant." The broker's copy matched.

Second, a photo album on the chamber's own site from a 2019 dinner, posted by another member, tagged her by full name in

the caption. She was wearing a printed name badge in the front row. Face captured, name captured, event captured — all on someone else's property, all still online.

Third, an old blog run by a fellow board member had a "farewell" post from 2018 congratulating her on stepping up as treasurer, describing her prior role in a single line: *"a consultant who has helped several of our sister charities."* Consistent.

Where that leaves the ladder. The reporter's question was narrow: chartered accountant, or financial consultant? Two independent non-broker sources — the charity's own newsletter and a local blog — quoted the same wording, and a broker copy matched. The chamber photo confirmed she was the same Ilse Vroeg the charity had, in case anyone tried the same-name-different-human defence. **Strong on the specific claim.** I did not go looking for what her deletion was *for*, because the reporter had not asked, and the wedding photos of her cousin's cousin were not going to teach me anything the job needed. Stop when the answer lands.

A second case: the one that doesn't close

A different request, a week later. A face pulled from a screenshot, a nickname, a claim that the person had run a small vendor account on a marketplace that shut in 2023. The account had been deleted before the shutdown announcement. No archives. No tagged photos on any third-party feed I could reach. Reverse-image on the face returned nothing above thin noise. Two plausible name candidates surfaced from broker mirrors — both real people, both roughly the right age, both in roughly the right region — and nothing separated them.

I wrote it up honestly. *Could not attribute; probable name candidates: two, cannot distinguish; no independent thread strong enough to elevate either.* The client wanted a name. The evidence did not carry one. Not closing a case is the job, not a failure — and a wrong name given confidently is a harm you cannot take back.

A safe exercise

Do this on yourself. Pick an account of your own that you deleted years ago — an old MySpace, a dead ICQ handle, a defunct forum profile, an early Twitter you nuked in your twenties, a blog on a host that went out of business. In a clean browser, run the method on you: cache pass, mention ring on your name plus the era, broker lookup, 2nd-degree crumbs from friends whose lists are still open. See what survives. Write down what surprised you.

You will find things. Most people do. Then decide, one item at a time, whether you want to try to take any of it down — and appreciate that the person on the other end of your investigations gets the same feeling in reverse.

Recap

- The account is gone. The person is not. What survives lives on other people's property.
- Anchor what you know before you look — name, face, city, associate, deletion date.
- Cache pass first, cheap and occasional wins.
- Reverse-image any surviving avatar; a fresh account under a new name is the most common self-migration.
- Walk the mention ring on the open web with the target's own platforms stripped out.

- Group rolls, event pages, and comment threads are maintained on the group's side and cleaned up unevenly.
- Broker snapshots are leads, not facts, and three brokers echoing each other is one broker.
- 2nd-degree by hand, never by script; scraped friend lists violate ToS and taint the case.
- Weight everything on the ladder; two moderate threads are not a strong one.
- Have a nameable reason before you rebuild a deleted identity, and take only what the reason needs.

Chapter Three walks from a single re-surfaced fragment back to a whole life — turning one lucky hit into a documented, weighted profile without over-reading what you found.



CHAPTER THREE

Reading a Photograph for a Place

Every picture that includes the world outside is a coordinate waiting to be read.

A photograph lands in your inbox at 11:14 on a Tuesday. No caption. No file metadata — the sender screenshotted it, twice, from an app that strips everything on the way through. What you have is a corner of a balcony, a plastic chair, a striped cup, a strip of sky the colour of weak tea, and the diagonal edge of another building leaning into the frame. A woman is being held somewhere against her will. Her sister sent you the picture from

a burner. Somebody on the other end of a phone is asking where she is.

You are not going to guess. Guessing is what people do when the picture feels too small to speak. This picture is not small. This picture is loud. It is loud in a language most people do not read, and the whole discipline of this chapter is learning to read it — to walk the frame element by element and let each element throw away places it cannot be. The right mood for the work is *narrowing*. Not searching. Narrowing. You start with the whole earth and you subtract.

The idea: narrow, don't guess

Geolocation from a photograph is not a lucky guess and it is not a party trick. It is a controlled narrowing. Every element you can read in the frame either *excludes* many places on earth or *concentrates* attention onto a few. The road paint excludes half the continents. The plate on the parked scooter excludes half of what is left. The tree in bloom excludes half of that. Do this ten times, patiently, and the earth collapses to a district. Do it two more times and it collapses to a block.

The one framing you must be able to say back in one sentence: every visible element either excludes places, or concentrates on them; the job is to keep peeling exclusions until only a small set remains. Everything below is technique in service of that sentence.

What a photograph actually tells you

Before you touch any tool, you need to know what you are looking at. A trained eye reads a photo the way a mechanic listens to an engine — as a stack of independent signals, each one narrowing the field.

- **Shadows and sun.** A shadow points away from the sun. Its length, relative to the object casting it, bounds the sun's altitude, which bounds the time of day for a given date and latitude. In the Northern Hemisphere above the Tropic of Cancer the sun sits to the south at noon; in the Southern Hemisphere below the Tropic of Capricorn it sits to the north; between the tropics the noon sun can sit either way depending on the season, and that ambiguity is itself a signal. A single crisp shadow gives you a hemisphere and, with the right astronomy, a latitude band.
- **Terrain profile.** Distant peaks, a flat horizon, the line of a coast, the way a hill rises behind roofs — terrain is stubbornly local. A jagged snowline visible over apartment blocks eliminates most of the planet. A dead-flat horizon eliminates the rest.
- **Road markings and lane widths.** Paint colour, dash pattern, the ratio of stripe to gap, the width of a lane, the colour of the kerb — all vary by country and often by decade of resurfacing. Yellow centre lines mean one set of places; white ones mean another. Bright white dashed shoulder lines

with a chevron shoulder tell you a specific set of highway codes.

- **Signage.** A stop sign is an octagon almost everywhere, but the word on it, the border weight, the reflective backing, and the mounting hardware differ. Regulatory signs in a particular font, using a particular restricted alphabet, will narrow to a region within a country in seconds.
- **License plates.** Background colour, aspect ratio, character set, whether there is an EU-style blue side-stripe, whether the plate is one line or two, whether it has a state or oblast prefix. Plates are the single densest per-pixel geolocation signal in most street photos.
- **Vehicles and orientation.** A car model tells you which markets it was sold into. But the loudest single bit is which side the steering wheel sits on. Right-hand drive collapses the world to a specific set of countries. A left-hand-drive car in a right-hand-drive country almost always means it was imported for a very particular reason — that itself is a signal.
- **Power lines, utility poles, streetlights.** Surprisingly regional. Wooden versus concrete poles. Insulator colour and shape. Whether the pole carries three phases or one. Streetlight fixtures come from a small number of manufacturers whose regional contracts are stable for decades.

- **Flora and season.** A jacaranda in violet bloom brackets a climate zone and a month. An olive grove eliminates most of the Northern Hemisphere. Maples in autumn colour narrow the calendar to weeks. Even lawn grass species read regionally to a trained eye.
- **Architecture.** Roof pitch tracks snow load (steep for heavy snow, shallow for none). Balconies read local — Iberian tilework, French wrought iron, Soviet-era concrete cantilevers, Gulf-region shading screens. Watch specifically where air-conditioner units are mounted — a grille on the wall under the window, a compressor on the roof, a split-unit box on a purpose-built shelf: each is a regional habit.
- **Script and language fragments.** A two-letter abbreviation on a mailbox, a chalkboard glimpse, a shop sign at the frame's edge, a sticker on a lamppost. Even one word in a distinctive alphabet narrows harder than a hundred general clues.
- **EXIF residues.** If the file survived with metadata, read it *last*. Camera make, timestamp, and GPS may be present, missing, stripped, or forged. Treat any number as a claim to verify against everything else in the frame — never as a fact that overrides what your eyes are telling you.

Read the picture in three passes

The discipline is three passes, and if you skip the first you will bias yourself so hard that no amount of tooling recovers the case.

First pass — naked eye, no tools. Open the picture full-screen. Do not open a browser tab. Do not upload it anywhere. Write down, in a plain list, every element you can see. "Blue tiles, six-sided pattern. Plastic chair, white. Chalkboard fragment, left edge. Awning across street, red-and-white vertical stripes. Overhead cable, single wire, wooden pole with insulator on top." The list is boring. Boring is the point. You are building an inventory before you form a theory.

Second pass — sort by exclusion. For each element, ask what it *rules out*. The overhead single-wire cable on wooden poles rules out most of Western Europe's newer urban districts. The chalkboard fragment in what looks like a Latin script rules out most of Asia. The plastic chair rules out nothing at all — write "no signal" next to it and stop looking at it. You are not trying to identify anything yet. You are shrinking the map.

Third pass — pick your levers. Now pull out the two or three highest-signal elements — the ones that would collapse a huge area if you searched them. Search *those*, not the whole photograph. The whole photograph is a bad query. A tight crop of the specific chalkboard hand, the specific awning stripe, the specific plate on the scooter — each of those is a good query. This is where you finally reach for a browser.

The method, step by step

Step 1 — Log the picture. Note the source, the person who sent it, and any capture-time claim they made ("she sent this Tuesday morning"). The claim is a claim; the shadow in the frame is a witness. You may need to reconcile the two later.

Step 2 — Read the frame naked-eye. Write the inventory. Resist reverse-image search — running it now will replace your fresh reading with whatever the engine's index happens to hold, and you will spend the next hour chasing hits you did not need.

Step 3 — Continent and hemisphere pass. From the shadow, the vegetation, and the script fragments alone, place the picture on half a planet. Sun on the north wall at midday puts you south of the equator. A crop of the chalkboard glimpse gives you a script family. Deciduous foliage in leaf in December puts you south of the equator or in a small tropical band. Do this before you touch a country.

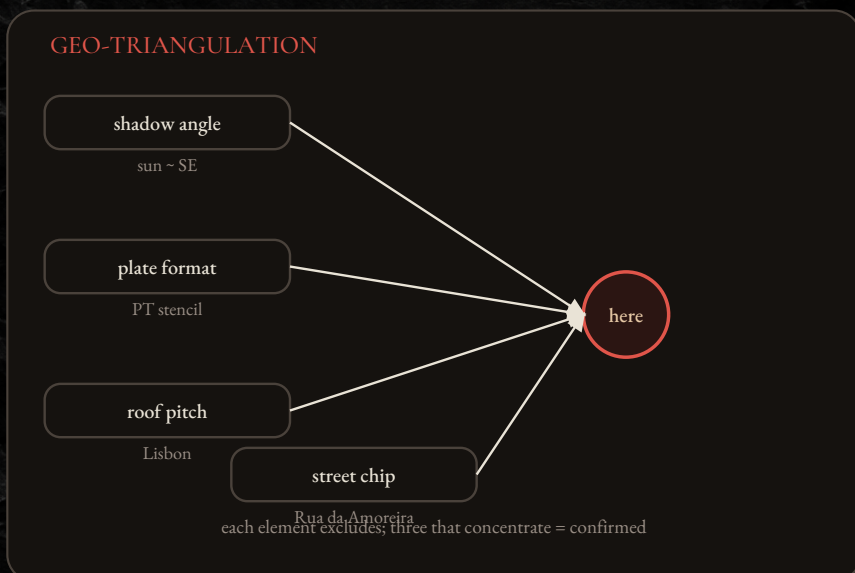
Step 4 — Country pass. Now the plate format, the road markings, the driver's side of the traffic in the frame, the signage font, and the language fragments. Cross-check at least three independent axes before you name a country. If the plate says one country and the road paint says another, you have a border town, an imported vehicle, or a mistake — none of which you resolve by picking your favourite answer.

Try it (2026). Pick a public news photo from a country you do not know well — a wire-service street shot with the caption hidden. Work country-only from the frame: plates, paint, signage, one script fragment. Write your answer down, then reveal the caption. Do this ten times and you will feel the exclusion muscle grow. Do it once a week and it will not shrink.

Step 5 — Region pass. With a country in hand, narrow to a region. Terrain (mountains, sea, plain, delta). Architecture (northern steep roof vs southern flat). Foliage (Mediterranean vs Alpine vs steppe). Secondary language on shop signs if the country has regional languages. Regional chains — a supermarket logo you can read, a pharmacy cross of a specific colour, a fuel brand that only trades in one part of the country.

Step 6 — Reverse-image now, with a targeted crop. The whole-frame reverse-image is a beginner's move and it almost always returns nothing useful. Crop to the single most distinctive element — a specific sign, a unique piece of architectural trim, a mural, a numbered manhole cover — and reverse-search *that*. Try more than one engine; their indexes diverge sharply. Reverse-image is a lookup, not a discovery — you are asking "has this exact object been photographed and indexed elsewhere," not "what is this."

Step 7 — Street-view walk. Once you have a candidate street or district, walk it in satellite and street imagery. You are matching corners, not vibes. The specific angle at which the neighbour's building cuts into your frame. The paint on the balcony railing. The pattern of the tile floor. The specific tree — species, height, position relative to the corner.



Three independent axes intersect on one balcony: the exact angle of the neighbour's roof-line, the position of a single flowering jacaranda in the courtyard, and the distinctive box of an air-conditioner mounted above a specific window. Any one axis matches many places. All three matching, at once, matches one.

Step 8 — Confirm on independent axes. A candidate location is *confirmed* only when at least two — better, three — independent axes align. Building silhouette plus the courtyard

tree plus the neighbour's AC unit. Not "the balcony looks right and I have a feeling." Rank the outcome on the ladder honestly: a plausible district is weak, a matching street is moderate, three independent axes on a single building is **strong**, and *confirmed* is the rung above that — reserved for the case where an independent source (a caption on another photo of the same balcony, a real-estate listing, the subject themselves) closes the loop.

A worked case, dead ends and all

A journalist forwards you a picture. A small café balcony. Blue-and-white six-pointed tiles on the floor. One white plastic chair. A striped cup — red-and-white. Half a chalkboard visible at the left edge, the top word cut off, then in a fragment of chalk cursive: *...e uma cerveja*. Across the street, at the right edge, the corner of an awning — red and white vertical stripes. A single scooter parked below, its rear plate half-visible. Question: where.

Naked-eye pass. You write it down. Blue tiles, geometric. White plastic chair, generic. Chalkboard, Latin script, cursive hand. Fragment reads "and a beer" in Portuguese or something close. Awning, red-and-white stripes, glossy. Scooter, plate stencil-cut white on light background. Overhead: no wires visible, urban infill. Sky: hazy, warm, late morning based on shadow angle. Shadow on the balcony floor: short, cast toward the south wall, so the sun is somewhere to the north — you check yourself twice, because a south-facing sun would put you in the

Southern Hemisphere, and you do not want to lean on a wrong hemisphere for the next hour.

First dead end. Out of habit, you drop the whole frame into a reverse-image engine.

```
Query: cafe_balcony_full.jpg
-----
Top matches (3 engines pooled):
- "Blue tile terrace, Lisbon"          weak
- "Riad rooftop, Marrakesh"          weak
- "Salvador da Bahia colonial house"  weak
- "Andalusia, azulejo balcony"       weak
- 40+ stock images, blue tile floors  noise
-----
No exact match. All near-matches
scatter across four countries.
```

reverse-image search — full frame

Useless. Blue tile is blue tile — the engine has thousands of them, indexed from realtors, travel blogs, and Airbnb listings. You close the tab.

Second dead end. You crop the red-and-white awning and search that.

```
Query: awning_stripe_crop.jpg
-----
Chain-café brand matched:
- franchise operates in PT, BR,
  ES, MZ, AO — five countries.
- Roughly 1,400 outlets total.
-----
Signal: cafe brand family.
Location: any of five countries.
```

reverse-image search — awning crop

Better — a signal — but still five countries. The awning narrows to a *chain*, and the chain trades in every Portuguese-speaking market plus Spain. You save the chain name and move on.

Reading the language. The chalkboard fragment, *e uma cerveja*, is Portuguese, not Spanish. Spain drops out. You are down to Portugal, Brazil, Mozambique, Angola — four countries.

Third dead end. You commit, prematurely, to Portugal, because you know Lisbon best. You search "azulejo balcony blue geometric" and start walking Lisbon streets in satellite. Half an hour in, you notice something wrong: the specific hexagonal tile pattern in the frame is unusual in Lisbon — most Lisbon balconies use square azulejo. Your candidate rooftops in Alfama do not match. You back out and stop leaning on your favourite city.

Fourth dead end. You pivot to Brazil, because the tile pattern is closer to what you have seen in Salvador. But the power-line profile in the frame — or rather the *absence* of the specific Brazilian overhead cable clutter — does not match Brazilian urban streetscapes, which almost always have a mess of overhead cables cutting the sky. Your frame has clean sky. Brazil's out, or at least demoted.

The pivot. You go back to the scooter's plate. The stencil is unmistakable once you zoom: white characters on a light background, a specific Portuguese national format, with a small

side-strip in a colour that Portugal uses and the African Lusophone countries do not. Angola and Mozambique demote. You are back to Portugal, but not to Lisbon.

You look again at the architecture: the roof pitch visible on the neighbouring building is shallow but not flat. The AC unit above the neighbour's window is mounted on a small wall bracket in a style that reads more Porto than Lisbon to your eye, but you distrust that hunch and go find three reference photos of both cities to test it. The bracket style is common in the Porto metropolitan area, less so in Lisbon. Weak signal, but it aligns with the tile pattern.

Targeted reverse-image, second attempt. You crop the chain awning *plus* a sliver of the numbered building above it — the building has a small, faintly visible azulejo house-number tile, the number partially legible. That crop is now a specific query.

Candidate: Porto district, café
chain outlet register lists
four Porto-area outlets on
small residential streets.

Walking each in street imagery:

- Rua A: awning wrong colour
- Rua B: awning right, but
balcony opposite is glass
- Rua C: no awning visible
- Rua D (Rua da Amoreira, 21-23):
awning stripes match,
balcony opposite matches
tile pattern, house-number
tile matches, jacaranda in
courtyard matches the tree
visible at the frame edge.

Four independent axes align on
Rua da Amoreira, 21-23. Confirm.

street-view scan for Rua da Amoreira

Four axes: awning stripe, tile pattern, house-number tile, courtyard tree. That is **strong** on the ladder. It stays at **strong**, not confirmed — every axis is inferential, and the café chain's public outlet register naming that same street corroborates the finding without providing a hard, unique self-link (nobody in the picture claims the address; no first-party document names it). Confirmed is a rung you reach through a unique identifier or a first-party statement, not by stacking independent inferences however cleanly they agree. You write it up with the chain of reasoning, including every dead end above, so the person reading your report can see not just where you ended up but why the earlier candidates were rejected — and what a *confirmed* signal

would have to look like.

A second case: the one that doesn't close

Another day, another picture: a close-up of a snow-covered pine trunk, a strip of grey sky, a single boot print. No structures. No signage. No script. Every axis you have narrows the same way — Northern Hemisphere, coniferous forest, winter, likely boreal. You examine bark grain, needle-fall pattern, the exact width of the boot print, the specific granularity of the snow. Nothing separates two candidate regions from each other: Karelia in northwest Russia and eastern Quebec both fit every axis you can read. You cannot rank one over the other on the evidence. So you report that, cleanly: two candidate regions, comparable likelihood, here are the axes I read and here are the axes I do not have. The mess is the job. A geolocator who never returns a "cannot separate" verdict is a geolocator who is guessing.

On the ethics of putting people on maps

A geolocation is coordinates, and coordinates are power. In a domestic-violence context, in a stalking case, in a war, the sentence "I know where you took this" is a weapon — sometimes for good, sometimes not. Have a legitimate question before you start; publish only what your question actually needs; and never publish the exact coordinates of a private home unless the public interest is unarguable and the target is a public actor in their public capacity. "The photo is from Porto" is often the whole answer the outside world needs. The block, the street, the number are for you and the client, not for a broadcast.

A safe exercise

Pick a public news photo of a protest or rally you did not attend and could not have been at. Cover the caption. Work the geolocation from the frame alone — country, region, street if you can get there. Compare your answer to the caption once you are done. It is legal, it hurts nobody, it exercises every muscle in this chapter, and it teaches you what your honest error bars actually look like.

Recap

- Geolocation is **narrowing**, not **guessing**. Every visible element either excludes places or concentrates on them; keep peeling exclusions.

- Read the frame in three passes: naked eye inventory, exclusion sort, then targeted search of the two or three highest-signal elements.
- Learn the frame's grammar: shadows, terrain, road paint, signage, plates, driver's side, poles, flora, architecture, script, EXIF-last.
- Do the continent-country-region narrowing before you reverse-image anything.
- Reverse-image with a crop, never the full frame; use multiple engines because their indexes diverge.
- Walk street imagery for corners, not vibes — match angles, trims, specific trees, specific AC units.
- Confirm on independent axes. One axis is a hunch, two axes is a candidate, three axes is strong. Anything called "confirmed" needs a source outside the picture itself.
- Report the dead ends, not just the answer. A geolocation without its rejections is not a geolocation, it is a claim.
- Publish only what the question needs. A district is usually enough for the world; a door number is for the file.

Chapter 4 turns to the same photograph and asks a different question: *when*.



CHAPTER FOUR

Reading a Photograph for a Time

The sun and the seasons are the two most honest clocks in any picture.

The client came back the next morning with a shorter question. We had placed the café in Chapter Three — the balcony on Rua da Amoreira in Porto, the tiled façade across the street, the tram cables settling the frame. That answered *where*. But *where* was never the real question. The real question was *when*. The photograph had been sent to them without a caption, and the person in the frame had denied being on that balcony in the month the client believed it was taken. So — same picture, same

reasoning, different column of information. We had already read the picture for a place. Now we would read it for a time.

Geolocation and chronolocation are sister disciplines. They ride the same loop — observe, hypothesise, verify, hold — and they climb the same ladder from *unverified* through *plausible*, *supported*, *strong*, to *confirmed*. But they read different anchors. Where geolocation lives in tile patterns, kerbstones and skyline geometry, chronolocation lives in sunlight, leaves, weather, and the small dated debris that human life leaves in a frame. A wall calendar. A poster edge. A phone in a hand.

The idea: the picture is a clock

Every outdoor photograph carries two clocks, and both of them tick honestly if you know how to read them.

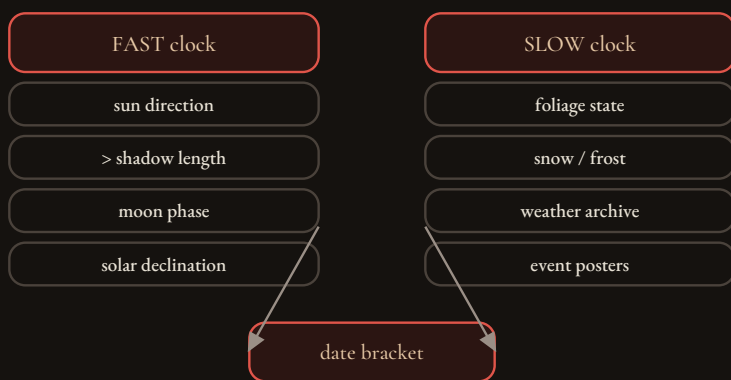
The fast clock is astronomical. The sun's position depends on latitude, longitude, and moment. On any given day at any given place, the sun rises at a specific azimuth, climbs to a specific maximum altitude, and casts shadows at specific lengths and directions. The moon runs its own cycle — phase, rise, set — and when it appears in a frame it hands you a bracket of a few days for free. A visible partial eclipse, if you are ever that lucky, is a single date.

The slow clock is phenological and seasonal. Trees are bare, in bud, in full leaf, in senescence, or dropped. Fields are ploughed, sown, growing, harvested, or stubble. Snow is present, absent,

fresh, or old. These signals move slowly, but they move in one direction over the year, and combined with a latitude they narrow the picture to a month or two of the calendar.

A photograph that shows both clocks — a shadow *and* a leaf, an angle *and* a snowline — collapses the answer very fast.

THE TWO CLOCKS



fast collapses hours; slow collapses months — cross them

The fast clock at the top of the frame, the slow clock at the bottom, and the dated human debris — posters, phones, cars — running down the middle.

What a picture actually tells you about time

- Sun direction and shadow length. A shadow's compass direction tells you the sun's azimuth. The ratio of shadow length to object height tells you the sun's altitude. Together, at a known latitude and longitude, they collapse to a small set of date-time candidates.

- **Foliage state.** Bare branches, bud burst, full canopy, autumn colour, dropped leaves. In a temperate zone this alone brackets you to a few weeks. Note whether it is a deciduous tree — evergreens tell you nothing seasonal.
- **Snow cover.** Presence, absence, freshness (crisp edges versus melted crust), depth against a known reference like a kerb. Fresh snow after a warm spell means a snowfall in the last day or two.
- **Agricultural stage.** Ploughed brown, green shoots, tall standing wheat, cut and windrowed, stubble with straw bales, ploughed brown again. Each stage is a specific window in the local farming calendar.
- **Weather in the frame.** Cloud type — high cirrus versus low stratus versus building cumulus. Ground still wet from rain. Frost on rooftops in morning light. Condensation on windows that only forms when the air outside is very cold.
- **Manmade calendar signals.** A wall calendar on the office wall behind someone. A storefront's "6 days to Christmas" chalkboard. A football match on a bar TV where the score and teams are legible. A brewery's seasonal promotion.
- **Fashion and technology dating.** An iPhone shape in a hand — every generation has a distinctive camera bump. A car model or facelift year visible on the street. A specific streetwear collaboration with a documented drop date.

- **Platform-side signals.** The app's "posted 2h ago" wording — a string that drifts to *yesterday* and then to *3 Jun* within days. Screenshot it before it changes. The Wayback Machine's capture time of the page hosting the image is a hard upper bound: the image existed *by* that date.
- **Astronomical extras.** A moon phase visible in the frame gives a bracket of a few days across a lunar cycle. Rare events — a solar eclipse, a bright comet at a known date, a meteor shower peak — anchor to a single night.
- **Contrails and aircraft.** A visible aircraft, or a fresh contrail with a distinctive shape, can be cross-referenced against flight-tracking history for that region on candidate days. The specific tracker sites available today may not exist in five years; treat the technique as durable and the tool as disposable.

Two clocks working together

The two clocks matter because each of them, alone, is ambiguous. The sun's position on 15 June at 2 PM at 41°N is almost identical to its position on 27 June at 1:50 PM. Shadow-only chronolocation nearly always leaves you with two candidate windows, sometimes one in spring and one in autumn, symmetrical around a solstice.

The slow clock collapses that ambiguity. Full canopy on the tree in the frame excludes the March mirror of a September date. Snow on the ground excludes the June mirror of a December one. Wet pavement on a day the weather archive shows was dry

excludes the day itself.

You almost never get chronolocation from one clock. You get it by making the two clocks agree.

The method, step by step

Step 1 — Freeze provenance. Save the image with its filename and URL. Screenshot the *page* that hosts it, including the "posted N hours ago" label. That label is a text string, not a data point, and it will change on you overnight. Note the Wayback Machine capture history of the hosting page — the earliest capture that contains the image is a hard upper bound on the date.

Step 2 — Read the frame naked-eye for slow-clock cues. Before you open any tool, look. Leaves? Snow? Standing crops? Puddles? Shadows sharp or diffuse? Write down what you see, in words, in your notes. This forces you to notice things you would otherwise skim past when you're already halfway to a hypothesis.

Step 3 — Read the frame for fast-clock cues. Where is the shadow pointing? How long is it relative to the thing casting it? Is the moon visible? Is there any visible celestial reference at all? Note directions in cardinal terms first — "shadow points roughly north-north-east" — because that is what a solar calculator will speak back to you.

Step 4 — Cross-reference weather. If Chapter Three gave you the location, pull the local weather archive for the candidate dates. Rain in the frame plus a dry candidate day excludes that day. Clear sky in the frame plus a candidate day recorded as full overcast excludes it too.

Try it (2026). For any city, a public weather archive will give you hourly cloud cover, temperature and precipitation going back years. Pick a photo of your own from a known date, look up the archive for that hour, and see whether the record matches what your photo shows. If it doesn't, you've learned something about your local station's placement — a useful calibration.

Step 5 — Foliage and phenology. Bud-burst dates for a species at a given latitude vary year to year, but publicly available phenology records give a bracket. A temperate-zone deciduous tree in fresh light-green leaf, sharp edges, no dust — a two-week window in spring. Full canopy dulled by summer dust — a much broader window, June to August.

Step 6 — Calendar and content signals. A football match on the pub TV with a legible score is a hard anchor to a single evening. A "Merry Christmas" window sticker that is not yet weathered brackets December. A concert poster whose date is legible is gold. Read the frame for the dated debris of ordinary life.

Step 7 — Technology dating. A phone or car model released in May 2023 excludes anything before May 2023. This is a floor, not a ceiling — the same phone still exists today. Combine with everything else.

Step 8 — Rank on the ladder. A single-day chronolocation is rare, and almost always requires an astronomical anchor or a dated event in the frame. A bracket of a few days is a good outcome. A bracket of a month is often the honest one. Say the honest answer. The client would rather have *last week of May 2025, strong* than *17 May 2025, we hope*.

A worked case, dead ends and all

Back to the balcony on Rua da Amoreira, in Porto. The client believed the photo was from August. The photographer said no. The picture had been stripped of EXIF by whichever messaging app it had come through — reverse-image sites returned no earlier copies, and the file was clean.

Dead end one. No metadata. No prior upload. Nothing to lift from the file itself.

Dead end two. Sun direction, read naked-eye off a lamppost shadow visible on the pavement below the balcony, pointed roughly north-north-west. Cross-checked against a solar-position shape — for Porto's latitude that shadow direction corresponds to a morning window in either spring or summer. The same shadow direction shows up for a stretch of

April–May and a mirror stretch in July–August. So the client's August hypothesis was consistent with the sun. So was the photographer's implied "not August." The fast clock was doing nothing.

Dead end three. The foliage in the frame was a potted lemon tree on the balcony. Evergreen. Silent. The one street tree visible at the frame edge was a plane tree, but it was cropped so tightly that you could only see a strip of leaf, not enough to say whether it was full canopy or early leaf-out.

Dead end four. Weather cross-check for the client's proposed date — a specific afternoon in August — showed a Porto archive record of heavy building cumulus and a thunderstorm at 4 PM. The photograph showed a flat, high, hazy sky and dry pavement. That afternoon was excluded.

day/hour	cover	temp	precip
14 / 15	OVC	28C	0.0
14 / 16	OVC	27C	4.2 mm
14 / 17	BKN	24C	1.1 mm

weather archive readout — Porto

Then the picture started to give things up.

Along the wall of the café across the street, at the frame's left edge, a small paper poster had been pasted on top of an older one. The top layer was mostly illegible, but two lines were readable when the image was enlarged: a festival name, and the words 5–7 *Junho*. The festival's own website confirmed it had

run on that weekend in one specific year. That year, plus a morning shadow window compatible with early June at that latitude, was a two-day bracket.

Full canopy typically established by late May. On 5-7 June: full leaf, dust not yet accumulated. Frame edge shows this exact state — consistent, not distinctive.

phenology bracket — Porto plane tree

The phenology on that plane tree edge was consistent with the June bracket — full leaf, still fresh — but on its own it would not have separated June from late May or early July. It was corroboration, not the anchor.

The photographer's own hand was visible at the bottom of the frame, holding a phone. The camera bump was the distinctive layout of a model released in May of the same year. That excluded the same dates in any earlier year — so the festival poster, which recurred annually, now pointed at one specific weekend.

The final answer: *morning of either 5, 6 or 7 June, that year, strong*. Not *confirmed* — we did not photograph a clock in the frame — but strong: two independent anchors (the poster, the phone model) plus two consistent readings (the sun, the foliage), and no contradictions. The client had asked for a month and was expecting to be told which. They got a weekend.

Chronolocating video

Video is a photograph that runs. It adds continuity, and continuity gives you more anchors.

The sun's *hour angle* changes by roughly fifteen degrees per hour — that is a clean astronomical constant. The apparent *azimuth* (compass direction) moves at a very different rate: near local noon in summer at high latitudes it can traverse tens of degrees in minutes, while at low sun angles in winter the azimuth barely moves. Which one your video actually shows depends on the room's geometry — a shadow on a wall usually reads azimuth; a light patch tracking across a floor closer to noon reads a mix. Do the arithmetic for the specific date, latitude and hour before you commit to "one degree in five minutes." A patch of sunlight measurably moves in a clip whose duration is a small fraction of an hour; the exact rate is what you have to check, not assume. Cloud shadows move in the same direction the wind is moving at cloud-base altitude, at speeds that a local weather archive records. A clip in which a shadow crosses a garden in eight seconds at a known distance can be cross-checked against wind speed at that altitude for candidate hours.

Contrails are the video chronocator's friend. A crossing pair of contrails, freshly laid, at a known compass direction, at a known moment in a clip, can be cross-referenced against flight-tracking history for that region on candidate days. The specific service

you use for the archive may not exist in five years, so treat the technique as durable and the tool as disposable. The reasoning is what carries forward.

A second case: the one that doesn't close

A studio portrait, indoors. No window. No wall clock. No phone in the frame. The subject wears a plain black t-shirt, generic dark trousers, plain silver ring — nothing that carries a season or a year. The wall behind is painted mid-grey and has no posters, no calendar, no light-switch model detail visible. Lighting is a softbox — could have been set up on any day in any year.

The Wayback Machine's earliest capture of the page hosting the image is January 2023, so the photograph existed by then. Nothing in the frame narrows it further. The best honest answer is *between the date of the earliest online copy and today, unverified beyond that*. Reported: *some time in the last three years, unverified*. Not every question closes, and pretending otherwise is worse than saying so.

On the ethics of putting people on a timeline

Chronolocation of a person's photograph carries a specific meaning: *you were in this place on this day*. Combined with geolocation, it maps a slice of a life. That is a legitimate tool for public accountability of public actors, for reporters checking a claim, for insurance investigators looking at fraud. It is not a tool to build the daily rhythm of a private person's movements.

The same discipline that applies to geolocation applies here — legitimate question, minimal disclosure, no publishing of anything that would compromise the safety of someone who is not a public actor. A weekend bracket for a public figure's denied appearance is journalism. A weekend bracket for an ex-partner's Sunday walks is stalking. Read your own frame before you read theirs.

A safe exercise

Take three of your own old photographs where you know the exact date. Strip the metadata. Hide the date from yourself — write it on the back of a card and put the card in a drawer. Then work them chronologically from scratch: read the slow clock, read the fast clock, pull weather archives, and write down your bracket. Open the drawer only when you've written it. This is the fastest way to calibrate how tight a bracket you can honestly claim, and where your reading fools you.

Recap

- Every outdoor picture carries two clocks: the fast astronomical one and the slow phenological one.
- Neither clock alone is usually enough. Make them agree.
- Freeze provenance first — timestamps, page URL, "N hours ago" strings — before they drift.
- Weather archives are the cheapest exclusion tool you have.
- Dated debris in the frame — posters, phones, wall calendars, match scores — is what pushes a bracket from a month to a week to a day.
- A single-day answer is rare and needs an event or astronomical anchor. A bracket is the usual honest outcome.
- Video adds continuity: sunlight motion, cloud drift, contrails against flight histories.
- Chronolocating a private person's life is a boundary. Ask why before you ask when.

Next chapter, we leave the world of frames and step into the world of racks. From reading a photograph for a time — to reading a network for an owner.



CHAPTER FIVE

Domains and Registration History

Every domain leaves a paper trail. Redaction is a curtain, not a wall.

A year ago you were reading a blog. Not a big one — a personal site with maybe thirty posts, one long series you kept an eye on, a comment section that had settled into three regulars. Today you paste the address into a fresh tab and the browser bounces you somewhere else — a landing page selling a course you have never heard of, in a font the old site would have gagged on. The domain is alive. The site behind it is a stranger's.

Someone owns that name now. Someone owned it then. Between those two owners is a paper trail: a registration, a transfer, a change of nameserver, a new mail provider, maybe a lapse and a recatch by an opportunist. And underneath it all, buried in a snapshot from 2011 that a redaction sweep in 2018 never went back to erase, there may still be an email address — a real one, in the clear, belonging to the person who first registered the domain when nobody was paying attention.

You have just moved out of people-work and into infrastructure-work. The tools change. The instinct does not. You are still walking a ring around a dot in the middle, and the ring is still lit.

The idea: a domain is a paper trail

A domain is not a website. That is the first thing to fix in your head, because until you have fixed it, half of what follows will feel like magic.

A website is what shows up in the browser: pixels, text, images, a shopping cart. A domain is a *record* about that website — a record kept by a registry, echoed by a registrar, and re-echoed by half a dozen public lookup services and passive-collection services around the world. Every time someone changes that record — new registrant, new nameserver, new mail provider, new expiry date — the change is written down, and often the *old* value is written down too, somewhere, by someone.

You do not need to break into anything. You read the trail.

The trail is thinner in 2026 than it was in 2015, because a redaction era arrived in 2018 and hid a lot of what used to be in the clear. But *thinner* is not *gone*. Two things about a domain are almost impossible to hide completely: the past, because snapshots outlive the redaction sweep that came for them; and the neighbours, because a domain lives on infrastructure it shares with other domains, and the neighbours often tell you more than the target ever meant to.

Terms you will hear a lot

Fix these first. Every technique below uses at least two of them.

- WHOIS — the public record about who registered a domain and when. Pronounced "who is." You query it, you get back a text block.
- Registry vs registrar. The registry is the authority that runs a top-level domain (.com, .io, .de). The registrar is the vendor you actually buy the domain from — GoDaddy, Namecheap, a small boutique. Same domain, two different companies behind it.
- DNS — the Domain Name System, the phone book that turns a name (example.com) into an address a computer can reach (an IP).
- Record types. A — the IPv4 address the name points to. AAAA — the IPv6 address. MX — where mail for this

domain is delivered. NS — which nameservers hold this domain's DNS records. TXT — freeform text records used for policy and verification, and the loudest leak channel of the bunch.

- **Nameserver.** The server that actually holds the DNS records for a domain. Every domain points at one or more nameservers, and those nameservers cluster in useful ways.
- **Historical WHOIS.** Snapshots of the WHOIS record taken *before* the 2018 redaction era, when registrant names and emails were usually in the clear. Kept by commercial archive services.
- **Reverse IP.** Every other domain served from the same IP address as your target. Sometimes noise, sometimes gold.

What WHOIS still shows in 2026, and what it stopped showing

Before roughly 2018, a live WHOIS query on most top-level domains gave you the registrant's name, email, postal address, and phone number in plain text. That era is over. GDPR arrived in Europe, ICANN's policy shifted in response, and most registrars now return a redacted record for the personal fields: "REDACTED FOR PRIVACY," "Data Protected," "See registrar's privacy policy."

What *does* still come back, in 2026, for almost every domain you query:

- The registrar — the vendor.
- The creation date — the age of the operation. A "consulting firm" whose domain was registered eleven weeks ago is telling you something about itself.
- The updated date — the most recent change to the record.
- The expiry date — when it lapses if unrenewed.
- Status flags — `clientTransferProhibited` (and cousins like `clientDeleteProhibited`, `clientUpdateProhibited`) are anti-hijack locks set at the registrar level — they stop unauthorised transfers, they are not privacy flags, and a redacted registrant with a "prohibited" status is normal not suspicious. `serverHold`, by contrast, is a different animal: it means the registry has pulled the domain out of the zone — the domain will not resolve — usually because of a legal dispute, non-payment, or an abuse enforcement action. A `serverHold` mid-project is a story. Boring most of the time, useful when they suddenly change.
- The nameservers. This is the single most useful field left in live WHOIS.

The nameservers matter because they cluster. Two unrelated hobby blogs on the same big DNS provider (`ns1.cloudflare.com` and its neighbours) tell you nothing. Two obscure domains sharing a private nameserver name that appears on only nine other domains in the world tell you a great deal.

The principle holds. What looks redacted is not fully hidden. The personal fields on the front of the record went behind a curtain, but the side channels — registrar preference, creation-date age, nameserver clustering, and above all the historical archive — did not.

Historical WHOIS: the 2011 email that never got redacted

Before the redaction era, registrant email addresses and names were public by default for most TLDs. A commercial industry grew up around that — services that quietly recorded a WHOIS snapshot of the world every day, and kept the archive. When 2018 came and current WHOIS went dark, the archive did not retroactively erase itself. What was in the clear before is still in the clear, sitting in those historical databases.

That is the biggest single lever this chapter teaches. If a domain was first registered in 2011 with `operator@personalemail.com` and only redacted after 2018, the 2011 record is still there — and so is the email. A registrant who has never once transferred that domain will not have thought to scrub what they wrote down fifteen years ago, because in 2011 nobody scrubbed it. That is the "one email that got missed by the sweep," and it is often the whole case.

Honest hedge: most historical-WHOIS services are commercial and paid, and the specific vendor named in any guide you read

— including this one, if I named one — may not exist by the time you actually try to use it. Treat the *shape* as durable (pre-2018 snapshots exist and are queryable) and the *specific door* as replaceable.

DNS reading as reconnaissance

The DNS record set for a domain is a short list, publicly queryable, and — unlike WHOIS — it was never redacted. Read every record type. Each one betrays something specific.

- **A / AAAA.** The IP address serving the site. This is the pivot into reverse-IP: what other domains sit on the same host? On a giant shared host or a CDN, thousands of unrelated tenants — near-worthless. On a small VPS with three tenants, all three tenants may belong to the same operator.
- **MX.** Where the domain's mail is delivered. MX records pointing at [google.com](https://www.google.com) or [outlook.com](https://www.outlook.com) mean the operator uses Google Workspace or Microsoft 365 — common, weak on its own. MX pointing at a self-hosted mail server on the operator's own IP tells you the operator likes to run their own mail, and gives you an IP to reverse.
- **NS.** The nameservers. On a generic provider block, worthless for clustering. On an unusual private nameserver name — ns1.mycoolprojects.net — you can reverse-NS: every other domain in the world that points at the same nameserver, which is often the same operator's other properties.

- **TXT.** The loudest leak channel. SPF records list every mail sender the domain authorises. DKIM and DMARC records reveal the mail stack in detail. Verification tokens — a google-site-verification token, a facebook-domain-verification token, a Zoom or Atlassian or Stripe verification token — tell you which platforms the operator has attached to this domain, and the token strings themselves sometimes join two domains together at the same organisational tenant.
- **CNAME.** Aliases. A CNAME pointing `blog.target.com` at the operator's personal Netlify subdomain, or their personal GitHub Pages, is a common accident and a clean pivot.

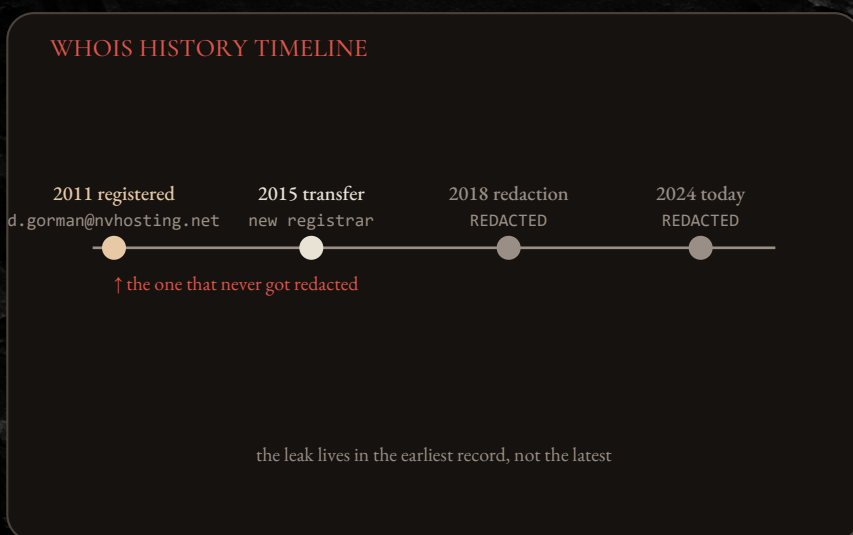
The method, step by step

Step 1 — Read live WHOIS. Query the domain. Write down: registrar, creation date, updated date, expiry, status flags, nameservers. Do not skim any of them.

Step 2 — Pull historical WHOIS. Look at the *earliest* record on file. Then walk forward chronologically and note every transition — a change of registrant email, a change of registrant name, a change of nameserver, a change of registrar. Each transition is a moment where the operator had to touch the record, and each touch is a moment of possible exposure. The 2011 email is usually in the *earliest* record; the transitions tell you the story that followed.

Step 3 — Read DNS. Pull A, AAAA, MX, NS, and every TXT record. Read the TXT records especially — most people leave more of themselves there than they realise.

Step 4 — Reverse-IP the current host. Look up every other domain served from the same IP. Weight the result honestly: on a shared CDN or a big hosting provider the reverse-IP result is thousands of unrelated tenants and tells you nothing; on a small VPS with a handful of tenants, near-neighbours are often the same operator.



A domain's paper trail across time: creation, transitions, redaction sweep, and the pre-2018 snapshots that survive it in commercial archives.

Step 5 — Reverse-NS. Look up every other domain in the world pointing at the same nameservers. Generic provider nameservers (ns1.cloudflare.com and friends) are worthless for

this — the noise floor is too high. Odd, private nameserver names are gold; a nameserver that serves nine other domains, and only those nine, has just handed you an operator's whole cluster.

Try it (2026). Take a domain you operate yourself. Query its live WHOIS and note every field. Query every DNS record type. Read your own TXT records with a stranger's eyes. What do they tell an outsider about which platforms you use, where your mail goes, and which other domains are yours? Most people are startled by the answer. The specific query tool named in any tutorial may have died since — the shape (query WHOIS and every record type from a fresh terminal or a reputable web lookup) does not.

Step 6 — Registrar patterns. Cross-check the registrar against your candidate operator's other known domains. Most people register everything through one vendor out of habit. If seven domains you already tie to the same person are all at the same uncommon registrar and the target domain is at the same one, that is a signature — weak on its own, meaningful in combination.

Step 7 — Look for the missed-the-sweep moment. Read every contact email in every historical WHOIS record for this domain and its neighbours. Read every TXT record for verification

tokens that map back to a specific tenant identifier — a Google Workspace verification token ties every domain in that tenant together. One leaked email or one shared tenant token is often enough to pivot to a person.

Step 8 — Rank. Same ladder as always. A registrar match alone is weak. A registrar match plus a reverse-NS cluster is moderate. A historical-WHOIS record with a full registrant name and email that also matches a real person's contact information elsewhere is strong. Do not stack two moderates and call the result strong; that is not how the ladder works, and it is not how you keep your reputation.

A worked case, dead ends and all

A client asked me to attribute a small operator running a scammy "investment education" site at winspiral-partners.io. The site sold a five-figure "mentorship program" whose testimonials were mostly stock photos with invented names. The client wanted a real person behind the operation. Fictional case, real method.

Live WHOIS. Everything I care about is behind a curtain.


```
WHOIS  winspiral-partners.io
-----
Registrar    BigDiscountRegistrar
Created       2023-04-11
Updated       2024-11-02
Expires       2027-04-11
Status        clientTransferProhibited
Registrant    REDACTED FOR PRIVACY
Email         REDACTED FOR PRIVACY
NS1           ns1.cloudflare.com
NS2           ns2.cloudflare.com
-----
personal fields = redacted
```

Everything you can read in 2026 for a fresh domain.

Nothing to grab from the personal fields. A large discount registrar, a creation date in April 2023, generic Cloudflare nameservers. I write it all down anyway.

Reverse-IP dead end. The site sits behind Cloudflare, so the A record points at a Cloudflare edge address. Reverse-IP on a Cloudflare address returns thousands of unrelated tenants — a Ukrainian food blog, a Portuguese law firm, a broken WordPress from 2019. Weak, rejected. A shared CDN's tenant list is not a cluster; it is a phone book of the internet.

Historical WHOIS dead end. I query the historical archive for winspiral-partners.io. The earliest record is 11 April 2023. Nothing before that. The .io TLD has never been friendly to the archive services in the way .com was, and this domain simply did not exist before its creation date. There is no pre-2018 snapshot because there is no pre-2018 domain. Nothing to work with.

TXT records dead end. I pull every TXT record. Generic SPF (v=spf1 include:_spf.google.com ~all), generic DMARC set to quarantine, no distinctive verification tokens. Google Workspace, standard configuration, no tenant identifier that ties to anything else. Nothing distinctive.

Four dead ends in a row. At this point a lot of people would report *insufficient signal* and stop. I do not, because the ring is bigger than one domain — a domain is one dot, and dots have neighbours.

The pivot. A passive-DNS pass (the technique gets its own chapter in Chapter 8; take it here on faith) surfaced a sibling domain — `winspiral-support.com` — that had appeared in the same DNS neighbourhood a year earlier. The site at that domain was a nearly identical clone of the `.io` site, apparently retired and left as a redirect. The `.com` had been *registered in 2016*, well before the redaction era became universal for `.com`.

I ran historical WHOIS on the `.com`.


```
Historical WHOIS winspiral-support.com
```

```
-----
```

```
2016-08-19 first record
```

```
Registrant Daniel Gorman
```

```
Email d.gorman@nvhosting.net
```

```
Registrar NicheRegistrarLLC
```

```
2018-05-24 redacted (GDPR sweep)
```

```
Registrant REDACTED FOR PRIVACY
```

```
2021-03-02 registrar transfer
```

```
Registrar BigDiscountRegistrar
```

```
2024-11-02 updated
```

```
-----
```

```
2016 record = missed the sweep
```

The pre-2018 snapshot the operator never went back and scrubbed.

There it is. The 2016 registrant record has a name and a personal email. The email domain, [nvhosting.net](mailto:d.gorman@nvhosting.net), is a small hosting brand that resolves to a one-man shop's landing page. Searching the exact address surfaces a public LinkedIn profile for a "Daniel Gorman," a small-time crypto affiliate marketer whose banner image happens to be one of the same "mentor" stock photos on the winspiral-partners.io site.

Two corroborators land quickly. First, the .com's 2021 registrar transfer moved it to the same discount registrar the .io uses today — a registrar pattern across the operator's cluster. Second, both domains' Google Workspace verification tokens (visible in TXT) reference the same tenant identifier — the two domains sit inside the same mail tenant, and that tenant is [nvhosting.net](mailto:d.gorman@nvhosting.net).

Rank it honestly. The 2016 historical WHOIS record is the one clean, self-linking signal — a first-party assertion of ownership at

a point in time. The registrar-cluster pattern and the shared mail tenant *feel* like two more votes, but they are both really the same underlying fact viewed twice: the operator's admin habits. Someone who buys through one discount registrar and runs both domains inside one Google Workspace tenant is expressing a single procurement decision, not two independent decisions. Counted correctly, that is one self-link plus one supporting-habit signal — moderate-to-strong, not clean strong.

The client got a person, an email, and a corroborating LinkedIn — and an honest note in two parts. First, the person named in the 2016 record is the *registrant*, which is not necessarily the same as the *operator today*. A domain can change hands. Second, the two "independent" corroborators are not fully independent of each other; the report labels them operator-habit signals rather than three separate votes. To climb cleanly to strong we would need a signal from a genuinely different collection method — matching content on Daniel Gorman's own writing, a photograph he posted that reappears on the site, a public admission on his LinkedIn. The report names what would take us there.

A second case: the one that doesn't close

A different domain, same week. A one-page site accusing a named individual of fraud, no author, no contact. The client wanted to know who put it up.

The domain was registered fresh in June 2024, private registration from the first day. Live WHOIS: fully redacted. Historical WHOIS: nothing older than June 2024 exists, because the domain did not exist. Nameservers: generic Cloudflare, so reverse-NS is worthless. Reverse-IP: single Cloudflare tenant, same worthlessness. TXT records: generic SPF, no verification tokens, no mail configured (there is no MX).

There is nothing at the domain layer to pivot from. Every side channel this chapter teaches returns noise or nothing. Honest report to the client: *insufficient signal at the domain layer alone*. The trail continues in other layers — the certificate transparency logs of Chapter 6, the passive-DNS neighbours of Chapter 8, the site's HTML fingerprints and analytics IDs which are also Chapter 8's territory. The domain by itself has told me nothing. That is a real answer, and I write it as one.

A safe exercise

Do this on yourself, on a domain you actually own. Read the live WHOIS. Read every DNS record type. Read your TXT records with a stranger's eyes, looking for verification tokens and mail-stack details. Then write a short paragraph: from those records alone, what could an outsider learn about which platforms you use, whose mail you route, and which of your other domains sit in the same neighbourhood? Most people are startled by how much of themselves is publishable-by-default in the DNS.

Recap

- A domain is not a website; it is a *record* about a website, and every change to that record is written down.
- Live WHOIS after 2018 usually redacts the registrant, but still returns registrar, dates, status flags, and — most useful — nameservers.
- Historical WHOIS keeps the pre-2018 world in the clear. The 2011 email that never got scrubbed is often the whole case.
- Read every DNS record type — A, AAAA, MX, NS, TXT. TXT leaks the most.
- Reverse-IP and reverse-NS are pivots, not answers; weight them by how noisy the neighbourhood is.

- Registrar preference and verification-token tenants are cluster signals — weak alone, meaningful in combination.
- Rank on the ladder as always: two moderates are not a strong; one clean self-linking signal is.
- When the domain layer returns nothing, say so. There are other layers.

Chapter 6 walks into the next one — the certificate transparency logs, where every TLS certificate ever issued for a name is written down in public, forever.



CHAPTER SIX

Certificate Transparency

Every certificate that browsers trust is now published, forever, in the open.

A payroll team at a mid-sized company spends a Friday afternoon stitching together a new staging environment. It is meant to be internal-only. Their DNS never mentions it in public zones; their firewall blocks casual scans from outside; nobody at the company has told a soul. Then their operator, ten minutes before leaving for the weekend, runs `certbot` — the standard tool for provisioning a TLS certificate from Let's Encrypt — against a hostname called `staging-payroll.corp-example.com`, because HTTPS is easier

than a browser warning on Monday morning.

Within seconds, that hostname is public. Not by a leak, not by a misconfiguration, not by anybody scanning them. By the design of the modern web. The certificate the operator just requested was posted to an append-only log the moment it was issued, and it will sit there, readable by anyone, effectively forever. The wall stayed up. The firewall stayed up. The internal DNS stayed private. But the *name* — staging-payroll.corp-example.com — is out.

You did not scan them. You did not probe them. You read a compliance record that the browser industry, deliberately, made mandatory. This chapter is about that record.

The idea: browsers made privacy of TLS-named hosts impossible

In the mid-2010s the browser industry ran out of patience with mis-issued certificates. A handful of high-profile incidents — a certificate authority quietly issuing a cert for a domain it had no business signing, a compromised issuer minting Google-branded certs, governments leaning on national CAs — showed that the private-trust model was not auditable. The response was Certificate Transparency, formalised in RFC 6962: a network of append-only public logs. Every publicly-trusted certificate must be published to at least one log at the moment of issuance. If a cert is not in the logs, modern browsers refuse to trust it. In practice that "modern browsers" is Chrome's policy since 2018 and Apple's since the same year; Firefox has never enforced CT. Between Chrome and Safari's combined market share, CT is effectively mandatory for any cert meant to face the public web.

That policy has a second-order consequence that the drafters knew about and accepted. Any hostname protected by a publicly-trusted TLS certificate is a hostname listed in a public log. Not "discoverable in theory" — listed, in a searchable database, with a timestamp, forever, indexed by dozens of aggregators.

You are not exploiting a bug. You are reading a compliance record. That framing matters, because it also draws the line: everything in this chapter is public-record work; nothing in this

chapter authorises you to *touch* any host you discover. We will come back to that.

Terms in one line each

- CA (Certificate Authority) — the organisation that signs a certificate; Let's Encrypt, DigiCert, Sectigo, GoDaddy, and roughly a dozen others handle almost everything.
- CT log — an append-only public record of every issued certificate; you can search it, subscribe to it, and never delete from it.
- SAN (Subject Alternative Name) — a list of hostnames a single certificate covers; often dozens, sometimes hundreds.
- Wildcard — a SAN of the form *.example.com that covers every subdomain without naming any of them.
- CN (Common Name) — the legacy field that used to hold the primary hostname; still often populated, but SAN is what matters now.
- Precertificate — a copy of the certificate the CA is about to issue, marked with a "poison" extension so browsers will not accept it, submitted to the logs first so the log can return the Signed Certificate Timestamps that get embedded in the final cert. The precertificate is the entry you see when you search a CT log; there is no separate later entry for the real cert.
- Issuer — the CA that signed a particular cert; the same operator often uses several across a year.

What CT actually gives you

- A near-complete enumeration of a target organisation's *externally-visible* hostnames — including staging, admin, api, dev, payroll, jenkins, git subdomains that the operator did not mean to advertise but did want HTTPS on.
- A historical record — hostnames from expired, revoked, or deleted certs still appear in the log; a subdomain that stopped resolving in 2022 is still visible today as evidence that it once existed.
- A cross-issuer view — one company routinely uses three or four CAs across staging, production, and various SaaS wrappers; searching by domain unions all of them into a single list.
- SaaS tenant leaks — a SaaS provider issues per-customer certs (acme-corp.zendesk.com, acme-corp.atlassian.net), and searching the SaaS's own domain by SAN reveals every customer whose tenant name was ever certified.
- Timing — the precertificate for a new environment is often visible hours before the environment is announced publicly, because operators provision TLS before they cut the ribbon.
- A cross-organisation view of shared infrastructure — two brands sharing a cert on the same SAN list are, unambiguously, sharing hosting.

What CT is not

CT only sees *publicly-trusted* certificates — the ones the browser will trust. An organisation that runs an entirely internal Public Key Infrastructure (self-signed, or signed by a private CA they control and distribute to their own machines) does not appear in CT at all. CT also does not reveal what runs *on* any host — only the name. A hostname alone is exposure, not attribution: it tells you a thing exists at an address, not who is behind it or what it does. Everything in this chapter turns hostnames into leads that later chapters convert into people and into confirmed use.

The method, step by step

Step 1 — Anchor the domain. From Chapter Five's WHOIS and DNS work you already have the base domain the target actually controls, plus any aliases they own (acquisitions, marketing sites, country-code variants). Write them down. CT queries return exactly what you ask for; a missed alias is a missed limb of the tree.

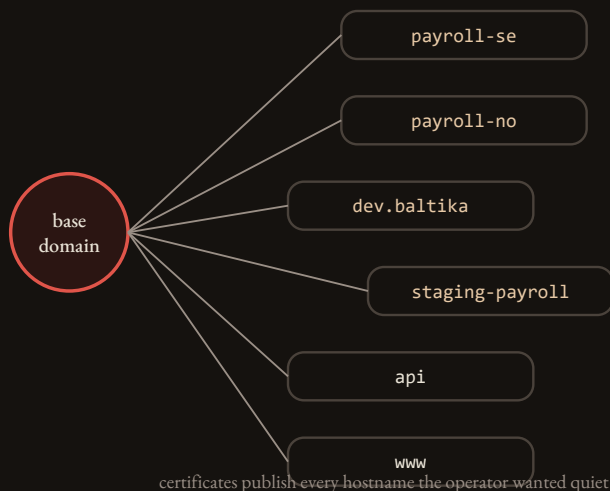
Step 2 — Query CT for the base domain and every known alias. The shape of the query is the same across every CT search interface — `crt.sh` and its peers all take a domain expression and return the certificates whose SAN list matches it. Pull every hostname from every SAN across every returned cert, deduplicate, and sort by first-seen timestamp. What you have now is the target's externally-certified hostname history.

(Interface names date fast; the shape — "query by domain, read the SANs" — does not.)

Step 3 — Read the naming pattern. Operators are not creative. admin, api, staging, stage, stg, dev, qa, uat, test, preprod, payroll, internal, intranet, vpn, git, gitlab, jenkins, jira, wiki, mail, smtp, sso, auth, id, admin-<region>, <product>-<env> — this is the classic leak-pattern vocabulary. A cluster of them tells you the *shape* of the internal environment before you look at any content. Six subdomains named jenkins, sonarqube, nexus, git, jira, wiki sketch an entire DevOps stack without a single probe.

Step 4 — Cross-reference to DNS. A hostname visible in CT that also resolves — has a live A or AAAA record — is reachable right now. A hostname visible in CT that no longer resolves is retired but was real at some point. Both matter. The live ones are current attack surface for a defender or current investigative surface for a researcher; the dead ones sketch what the environment used to look like, which is often more revealing than what it looks like today, because operators clean up production and leave staging in the log forever.

CT: ONE DOMAIN, MANY SANs



The base domain at the root, a fan of SANs off each certificate, and the split between live-in-DNS branches and historical-only ones. The historical branches are the tell.

Step 5 — Look for tenant leaks. Every SaaS product that gives customers their own hostname puts those hostnames into CT. Take a SaaS the target is likely to use — a helpdesk, a wiki, an issue tracker, a payroll processor — and search CT for the SaaS's domain. Grep the returned SAN lists for the target's name. A certificate covering `acme-corp.zendesk.com` proves Acme Corp has a paid Zendesk tenant. Nobody at Acme published that. Nobody at Zendesk published that. The CT log did, because Zendesk had to certify it.

Try it (2026). Search a CT interface for your own domain. Read the timeline of certificates you have issued

to yourself over the years. Note the staging hostname you forgot about, the acquired brand you still own, the wildcard from the CA you thought you had stopped using. This is the exercise that teaches the leak-pattern eye; it also fixes a few of your own leaks along the way.

Step 6 — Watch the feed. Some services (mention as *shape*, not as endorsement — the specific names change) let you subscribe to a domain: every new certificate issuance triggers an alert to you within minutes. This is how researchers spot product launches, acquisitions, and new regional expansions before the press release. A new SAN appearing under a competitor's domain named payroll-de at 03:17 on a Tuesday is a data point you would never otherwise have.

Step 7 — Rank. A hostname is *exposure*, not *attribution*. A live admin subdomain says the organisation exists and runs an admin interface; it does not name a person, prove use, or authorise any further step. Only combined with content (Chapter Seven) or infrastructure fingerprints (Chapter Eight) does exposure become a lead that points at a human. Weight the fragment on the ladder honestly, then decide.

A worked case, dead ends and all

A financial-press client asked me to check a single competitive claim before they published. A rival startup was telling investors that a small SaaS company called Halkstone Labs — publicly described only as "a fintech tooling shop" on a one-page marketing site — was quietly building a payroll product for the Nordic market. Halkstone's site said nothing about payroll and nothing about the Nordics. The client did not want to contact the target and tip them off. They wanted to know, from open sources only, whether the rival's claim held up. Fictional company, real method.

Anchor. From the marketing site and the Chapter Five WHOIS pass, the base domain was `halkstonelabs.com`. No obvious aliases, no acquired brands surfaced. So a single CT query.

The first pass was mostly boring.

```
crt.sh style listing, halkstonelabs.com
-----
47 certificates, 3 issuers
oldest 2021-04, newest 2026-06

SAN samples (dedup):
  www.halkstonelabs.com
  mail.halkstonelabs.com
  api.halkstonelabs.com
  stage.halkstonelabs.com
  status.halkstonelabs.com
  *.halkstonelabs.com   (wildcard, 2024-11)
-----
nothing said "payroll" yet
```


Forty-seven certs, and the SAN list looked like every other small SaaS on the internet.

Dead end one — the boring cluster. `www`, `mail`, `api`, `stage`, `status`. Nothing there hinted at a new product line, let alone a Nordic-market payroll product. This is a common pattern: a shop with a small footprint keeps a small SAN list on its main cert and rotates the same handful.

Dead end two — the wildcard. In November 2024 they had issued a wildcard, `*.halkstonelabs.com`. A wildcard covers every subdomain but *names* none of them. So the wildcard certificate itself proves the operator anticipated many subdomains — a small but real signal — while telling you nothing about *which* ones. That is the trade-off wildcards are built on, and it is what makes them attractive to organisations who understand CT.

Dead end three — the ghost subdomain. One older cert, from 2022, had a SAN for `dev.halkstonelabs.com`. Promising. I noted it, then cross-checked in DNS: `dev.halkstonelabs.com` no longer resolves. The cert expired and was never renewed. Historically real, currently dead — evidence that the company had a dev environment at some point, but no live door to walk toward.

Dead end four — the false-positive tenant search. On a hunch that they used common SaaS tools, I ran a tenant search across `atlassian.net` and `zendesk.com`. Nothing under `halkstonelabs` on either. Either they use these products under a different tenant

name, or they don't use them at all. Absence in CT is not proof of absence in reality; the tenant might exist under a name I have not guessed — a common trick is to register the tenant under a project codename that shares nothing with the parent brand. Note the null, move on.

Dead end five — a same-name collision. Sorting the returned certs by issuer, one entry stood out: a cert issued by an obscure regional CA to `halkstone.labs` — no `s`, different TLD, and with SANs pointing at a boutique wine shop in the south of France. Same-name collisions are the rule, not the exception, in CT results, because the log indexes every string every CA has ever signed. I dropped the wine shop and moved on. The lesson: verify that the base domain you queried and the base domain you received match, exactly, before you count anything you find.

Then the log paid out. Sorting the SAN list by first-seen timestamp, three entries from the last month clustered together.

```
Sorted-by-first-seen SAN readout
-----
2026-06-24  payroll-no.halkstonelabs.com
           precertificate, Let's Encrypt
2026-07-02  payroll-se.halkstonelabs.com
           precertificate, Let's Encrypt
2026-07-11  payroll-uat.halkstonelabs.com
           full cert, Let's Encrypt
-----
"no" = Norway, "se" = Sweden, uat = user
acceptance test — a real staging track
```


Three certificates, three weeks, one story.

Read the pattern honestly. payroll-no and payroll-se are strong hints — no and se are the ISO country codes for Norway and Sweden, and no other product-line naming in the Halkstone SAN history uses that suffix pattern. payroll-uat is the giveaway: user acceptance testing is a stage that only exists when a real product is close enough to real customers to warrant a customer-shaped test environment. You do not stand up a UAT for vapour.

Cross-check to DNS: all three resolve, to IPs in the same hosting block as the rest of `halkstonelabs.com`'s infrastructure. So the environments are not just certified — they are live. I did not open them. I did not send a single request to any of the three hosts beyond the name resolution. That was the line.

What I reported. The rival's claim is confirmed by public CT exposure: Halkstone Labs is running a payroll product line with Norwegian and Swedish variants, plus a working UAT environment, first certified in the last four weeks. The evidence is public-record and reproducible; the client can verify it themselves without contacting the target. I flagged the wildcard and the ghost dev cert honestly as background signals rather than the payoff, and I noted that CT tells us the shape and timing but not the *content* — whether the product is one week or six months from customer launch is a question CT cannot answer.

The client ran the piece. The story held.

A second case: the one that doesn't close

A different micro-case, a defence contractor. The client asked for the same treatment: enumerate the externally-visible environment via CT and sketch the shape. The CT search returned exactly one certificate: a wildcard on the marketing domain, issued by a mainstream CA, covering the marketing site and a www alias. That was it. No staging, no admin, no jenkins, no tenant leaks under any SaaS I tried.

The reason is easy to guess and impossible to work around: this contractor runs a fully internal PKI. Every internal host is signed by a private CA distributed to their own machines. None of those certs ever go to a public log, because they are not publicly-trusted and were never meant to be. CT was designed to make *public* trust auditable; it says nothing about private trust by design.

Honest report to the client: *CT gives you nothing here. This target does not leak via CT. Move to Chapter Seven for content-based angles, or accept the limit.* Not every method pays out on every target, and the maturity of the target's PKI is a signal in its own right — one worth writing down.

A note on ethics

Discovering an internal-looking hostname does not authorise you to probe it. Reading a certificate log is public-record work; opening a TCP connection to `admin.corp-example.com`, sending an HTTP request, running a port scan, or trying credentials is a different category of act and, in most jurisdictions, is governed by unauthorised-access laws that do not care that the hostname was easy to find. Stop at the name. The hostname enters your notes as a fact about the target's exposure; it does not become a door you walk through. The exceptions are narrow and formal: a published bug-bounty scope that lists the host, an authorised penetration-test engagement, or an in-scope research agreement — always in writing, always specific. Absent that, the ethical line is drawn right at the boundary between reading the log and touching the host, and the legal line is often drawn in the same place.

A safe exercise

Pick a well-known SaaS product — a helpdesk, an issue tracker, a payment processor — whose customers get their own hostname. Search CT for the product's domain and read the SAN patterns. Note the naming convention (`<customer>.product.com`, or `<region>-<customer>.product.com`, or the reverse), the wildcards the vendor issues to themselves, and the tell-tale internal-sounding hostnames the vendor forgot were exposed. You will come away with a working eye for the leak pattern, and the strong suspicion that most of the SaaS market has quietly published its customer list without ever meaning to.

Recap

- Certificate Transparency turns every publicly-trusted TLS certificate into a permanent, searchable public record — by browser policy, not by leak.
- Any hostname protected by a publicly-trusted cert is a hostname in a public log; you read the compliance record, you do not attack the target.
- The SAN list on a single certificate often names dozens of hostnames; the classic leaks are `admin`, `staging`, `dev`, `payroll`, `jenkins`, `git`, and the rest of the DevOps vocabulary.
- Wildcards hide subdomains behind `*`; they signal breadth without naming names.

- Precertificates and first-seen timestamps let you spot new environments hours before public announcement.
- SaaS tenant leaks — searching the SaaS's own domain in CT reveals per-customer tenant hostnames the customer never announced.
- CT is silent on internal PKI; a target that signs everything with a private CA gives CT nothing.
- Hostnames are *exposure*, not *attribution*. Stop at the name; do not touch the host without written authorisation.

Chapter Seven turns from infrastructure back to people: how the *content* served at those hostnames — a favicon, a copyright footer, an analytics ID, an author name in an HTML comment — carries the human fingerprint that the hostname alone withholds.



CHAPTER SEVEN

From Infrastructure Back to a Person

Servers do not have names. Their operators cannot resist leaving one.

There is a site that sells fake press releases. It looks like a news wire — a masthead, a dateline, a byline that reads only "staff." There is no About page. There is no imprint. The domain is a fresh registration, WHOIS fully privacy-redacted, hosted behind a large CDN. The frontal search dies at the front door.

But the tab in the browser has a favicon — a small orange square with a stylised letter M. And that same favicon, byte-for-byte identical, sits in the tab when you open three other sites that

look nothing like this one: a supplements shop, a "cryptocurrency review" blog, and a fake local newspaper for a town in another country. And two of those four carry the same Google AdSense publisher ID in their page source — a string beginning pub- that ties an ad account to a single Google identity. And that AdSense ID appears again on a fifth site — a small food blog with an author byline and a real name.

This chapter is a hinge. Everything before it has been about infrastructure — domains, DNS, certificates, hosts. Everything after it, once you cross the hinge, becomes about a person again, and that is where Volume 2 takes over. The pivot itself is what this chapter teaches.

The idea: operators leave fingerprints on their tools

A server is anonymous. The person configuring it is not.

Every operator has habits. There is an analytics account they always paste in because it is the one they know the login to. There is a favicon file they carry from project to project because they made it once and never bothered making another. There is a WordPress theme they always reach for because they bought a licence in 2021 and now every site they build starts from the same folder. There is a commit-time pattern that betrays the hours they actually sit at a keyboard, and a language their comments are written in that no configuration file can hide.

Any one of these habits, on its own, is a weak signal. A favicon might be reused because both operators downloaded the same free icon. An AdSense ID might match because someone pasted an old snippet by mistake. A WordPress theme is bought by thousands of people. Each habit alone proves nothing.

Two or three habits that align across independent sources — a favicon match, an analytics-ID match, a commit-timezone match, none of them explainable by the same accident — is a different kind of finding. That is what this chapter builds.

The classes of operator-fingerprint

- **Analytics IDs.** GA4 measurement IDs (G-XXXXXXXXXX), legacy Universal Analytics tags (UA-XXXXXX-Y), AdSense publisher IDs (pub-XXXXXXXXXXXXXXXXXX), Facebook Pixel IDs, Yandex.Metrica counters. The same ID on two sites means the same *account* on the vendor's side, and an account belongs to a single login. This is one of the strongest operator-link signals you will ever find at the infrastructure layer. It is also one of the most over-read: a shared GA snippet can name the *builder* rather than the owner — a junior contractor who reuses the same tag across unrelated clients, or an affiliate ring where a dozen marketers paste in the same AdSense ID. Rank the finding as "same account" first, and only promote it to "same operator" once a second, independent signal agrees.

- **Favicon and image fingerprints.** The favicon is a file. Files hash. If two supposedly unrelated sites serve the *same bytes* at /favicon.ico, that is almost never coincidence — it is the same person deploying the same folder. The same reasoning extends to any small asset that ships with a site's template: a header logo, a placeholder image, a JavaScript file with a distinctive comment.
- **Tech-stack combinations.** WordPress alone is common. WordPress plus a specific paid theme plus a specific plugin loadout — with the plugins loading in the same order — is often per-operator. A tool that classifies tech stacks is useful for the individual features, but the combination is what identifies.
- **Historical WHOIS emails.** Chapter Five already made this point at length. Modern WHOIS is redacted, but a domain that existed before mid-2018 usually has a public registrant email captured in one of the historical databases, and that email is often the operator's real address.
- **Public git and commit metadata.** If any part of the site's code, or a related project the operator was proud of, lives in a public repository, the git history leaks the committer's email, the commit-message language, and the timezone of every commit. A repo whose commits cluster between 21:00 and 02:00 in UTC+3 tells you both what part of the world the operator lives in and what part of their day they treat as work.

- **Error-page leaks.** A misconfigured nginx returns a default page that names the internal hostname. A stray `mod_status` endpoint on an Apache server lists every virtual host it serves — sometimes including sites the operator did not want on the same box. A `.env` file left in the webroot is not rare enough.
- **DMARC report addresses.** The mailbox in a domain's DMARC TXT record — the `rua=mailto:...` — is where email authentication reports are collected, and the person who set it usually pointed it at their own admin address.
- **Contact-form and mail headers.** A contact form's raw output, or the Reply-To header on the mail it generates, sometimes carries the operator's real mailbox rather than the site's public alias.

Independence, again

Volume 2 gave you the ladder rule — weak, moderate, strong, confirmed — and warned that stacking non-independent signals is the fastest way to feel confident about the wrong thing. That rule applies here without modification.

A favicon match and a tech-stack match on the *same site* are two views of one setup. One operator's one folder produced both. That is one signal wearing two coats, not two independent votes.

A favicon match on site A and an analytics-ID match on site B are two independent votes. Different files, different vendors,

different failure modes. When they agree they agree honestly.

The same trap you learned to spot in the identity-graph chapter is here. Ask, of every pair, whether they could have come apart and didn't. If yes, count them. If no, count them once.

There is a subtler version of the trap. A shared analytics ID between a company's marketing site and its own support site is one operator — boring and correct. A shared analytics ID across sites in wholly unrelated industries — a supplements shop, a small-town news blog, a review site for kitchen appliances — is more likely a shared *developer* than a shared *owner*. Before you write the finding down, ask the disproof question: *what alternative story would produce this same fingerprint?* If "one freelancer who built all three" fits as well as "one operator running all three," you do not yet have an operator.

The method, step by step

Step 1 — Anchor. State the pivot point out loud. You have a domain, a hostname, a URL. Write it down. Everything that follows is a spoke from that hub, and if you lose track of the hub you will start mixing findings from different targets.

Step 2 — Pull the page source. Open the site in a browser and view the raw source of the homepage and two or three subpages. Extract every identifier that looks like a tag: analytics IDs, tag-manager container IDs, pixel IDs, site-verification tokens, script src hostnames, stylesheet filenames, distinctive class

names. Do not read the page for what it says. Read it for what its code accidentally admits.

Step 3 — Hash the favicon. Download `/favicon.ico` and compute its hash. The convention that infrastructure-search engines use is MurmurHash3 of the base64-encoded bytes; MD5 or SHA-256 of the raw file works too for cross-checking. Then search for other hosts that serve a file with the same hash. Shodan and Censys, at time of writing in 2026, both index favicon hashes and let you query for them — verify the current query syntax before you rely on it.

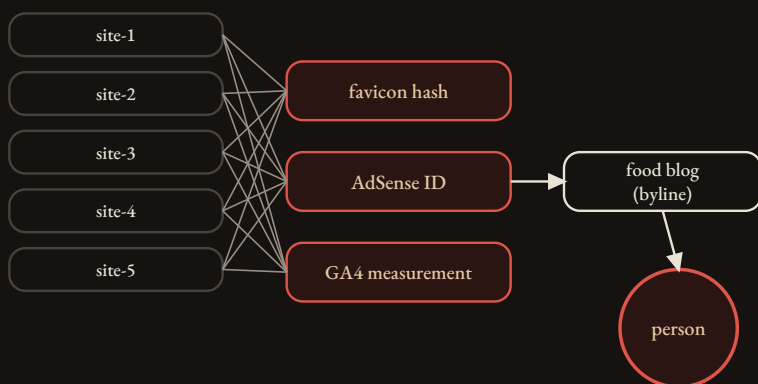
Step 4 — Search for the analytics IDs elsewhere. Take each ID you extracted in Step 2 and search for the exact string across the wider web. Google will find some. Specialised services that index page source for tracking IDs will find more. A single AdSense or GA ID appearing on a second, apparently unrelated domain is one of the strongest operator-link signals available at the infrastructure layer, because the ID by definition belongs to one vendor account.

Step 5 — Read the WHOIS trail again with new context. You did this in Chapter Five for its own sake. Do it again now with a purpose: the email you find in the pre-2018 record is a candidate to test against every other finding.

Step 6 — Look for the operator's own scraps. Fetch `humans.txt`, `security.txt`, `robots.txt`, `sitemap.xml`. Try

contact.html, about.html, team.html, imprint.html. Anything with a name. Then check the archive for versions of those pages from before the operator scrubbed them — the material from Chapter One applies directly.

INFRA → PERSON



operator fingerprints link the anonymous → the named

A single anchor domain in the centre, with spokes running to each fingerprint class: analytics IDs, favicon hash, tech stack, historical WHOIS, error-page leaks, DMARC address, contact-form headers, public git. Each spoke ends in a small cluster of other hosts or identifiers where the same fingerprint recurs. The clusters overlap where two independent fingerprints land on the same second host — those overlaps are the strongest attribution points, and the person is reached by following the overlaps outward.

Step 7 — Pivot to the cross-platform work. The moment any of these steps yields a name, an email address, or a personal

social-media handle, you have crossed the hinge. Everything you learned in Volume 2 — the identity-graph work, the making-the-signals-agree work — now applies to the human on the other end. Hand the pivot off to that toolkit; do not try to re-teach it here.

Try it (2026). Pick a personal site or blog you run yourself. Open its source and list every analytics ID, pixel, verification token, and third-party script it loads. Download its favicon and compute the MurmurHash3 of the base64-encoded file. Now search a favicon-hash index for other hosts with the same hash, and search the web for each analytics ID. See what an outsider could learn about you from the pivot alone. Then decide, coldly, which of those fingerprints you want to leave in place — and which to change tomorrow.

Step 8 — Cross-reference public code. Take a distinctive line from the site's source — an unusual CSS class name, a hand-written JavaScript comment, a chunk of PHP that does not look copied from a tutorial — and search for the exact string on the large code-hosting sites. A hit on a personal repository is the whole game: git history exposes the committer's email, the timezone of every commit, and the language of the commit messages. Even a hit inside a fork can lead you to the person who cared enough to fork.

Step 9 — Rank on the ladder. Write down, in one paragraph, what is *exposed* and what is *attributed*. Exposed: the infrastructure you can see. Attributed: the human you now claim runs it, and at what rung of confidence — weak, moderate, strong, confirmed. If your evidence stops at "one favicon match and one shared vendor tenant," you have a lead, not a finding. Say so in exactly those words. A reader who cannot tell the two apart will treat your lead as your conclusion, and it is your prose, not their carelessness, that let them.

A worked case, dead ends and all

The brief was small and unglamorous. A city council in the run-up to a local election had noticed five news-shaped websites publishing coordinated stories about one candidate. All five sites had appeared inside a six-week window. All five wrote in the same voice. The council wanted to know who ran them.

I started where any pivot starts: WHOIS on all five. Every single record was privacy-redacted by the registrar. Registered through the same commodity registrar, yes, but that registrar has millions of customers — a weak signal at best. Dead end.

DNS next. Every one of the five domains pointed its nameservers at Cloudflare. Reverse-IP was worthless — the addresses I saw belonged to a shared edge that fronts millions of unrelated sites. Dead end.

Certificate Transparency, following the pattern of Chapter Six. Each domain had a Let's Encrypt certificate. Each certificate carried only the site's own hostname and `www.` — no bleed of admin subdomains, no staging environments, no operator-name leaks in a stray SAN. Dead end.

The contact form on each site went, at the HTML level, to a `contact@` address on the same generic Google Workspace tenant. From the outside I could see the tenant existed; I could not see who owned it. Dead end.

Four independent avenues, four blanks. This is what a competently anonymised operation feels like from the outside — it is not that you are missing something clever, it is that the frontal path is genuinely closed.

The break came from the browser tab.

I noticed, while flicking between the five sites in the same window, that the little icons in the tabs looked identical. Not similar — identical. I saved each site's `/favicon.ico`, computed the MurmurHash3 of the base64-encoded bytes, and got the same number five times in a row.


```
favicon-hash.txt
site1.example.tld -1893478234
site2.example.tld -1893478234
site3.example.tld -1893478234
site4.example.tld -1893478234
site5.example.tld -1893478234
default apache icon 1094239847 (not our hash)
default nginx icon -0038284712 (not our hash)
```

Five sites, same favicon file, byte-for-byte. That is not a template default and it is not two people downloading the same free icon. That is one folder being deployed five times.

I moved to page source. Three of the five sites embedded the same AdSense publisher ID.

```
adsense-matches.txt
site1 pub-6602934871052210
site2 pub-6602934871052210
site3 pub-6602934871052210
site4 (no ads)
site5 (no ads)
```

An AdSense publisher ID is not a shared library. It is a payment identifier tied to one Google Account. Three sites carrying it means one operator pointing three properties at their own ad revenue.

Pause here, because this is where the field has burned itself before. A shared analytics or AdSense ID is a well-known signal and a well-known false positive. In 2020, Bellingcat and The Grayzone traded published disagreements over exactly this kind of finding — whether a shared Google account across several outlets proved coordinated ownership or only shared plumbing.

Neither side fully won that argument; the lesson survived it. Treat a matching ID as a strong lead, not a verdict. Before you write "one operator," look for a second independent signal — a favicon match, content overlap, an unusual shared hosting fingerprint. Two votes that could each have disagreed and didn't is a finding. One vote is a lead.

I ran that same publisher ID as a text search across the wider web. It hit on a sixth site — a small, sleepy food blog registered in 2019, unrelated in topic to any of the five, that carried the ID in the footer of every recipe page. The food blog had an author byline: "Iryna Petrenko." A common name in the region — a lead, not a finding, yet.

The blog's current About page was two sentences and gave nothing away. I pulled the same page from the archive at three-month intervals going back to first capture. The 2019 version was chattier — a paragraph about the author, a linked Twitter handle, a city.

Now I had crossed the hinge. From the Twitter handle onwards, everything in Volume 2 applied: the identity-graph work, the cross-platform confirmation, the family-side pivots when the direct account went quiet. That work is not the subject of this chapter, and I will not re-run it here. The pivot itself is the point.

Rank honestly. At the *operator* level — "one person, or one very small team, runs all five of these disinformation sites" — the

finding is strong: three independent votes (favicon-hash match across five, AdSense-ID match across three, AdSense-ID recurrence on a sixth attributed site) that could each have gone the other way and didn't. At the *individual* level — "and that person is Iryna Petrenko of city X" — the finding is moderate on the infrastructure evidence alone. The Volume 2 handoff carried it further. The council got a report that named the rung, not just the name.

A second case: the one that doesn't close

The next target looked like the same kind of job and was not.

One site, one domain, coordinated behaviour on social media around it. WHOIS: fully redacted, fresh 2024 registration. DNS: Cloudflare, generic. CT: one Let's Encrypt cert, one SAN. Page source: no analytics IDs of any kind, no pixel, no tag manager, no verification tokens. Favicon: the default browser globe — no file served at all, so no hash to match. No humans.txt. No security.txt. No public git repo I could tie to it. Every admin surface reachable only through an internal certificate authority the public CT logs never saw. DMARC record present, rua=mailto: pointed at a dmarc@ on a privacy-forwarding service.

Every fingerprint class in this chapter was either absent or generic. This target had been built by someone who had read a book like this one. At the infrastructure layer alone, I had

nothing to pivot from — and the honest report says so, in exactly those words. There is a temptation, when a target closes cleanly like this, to invent one more small signal to justify the hours already spent. Resist it. A clean nothing is a real finding; a fabricated something is a career-ending one. The counter-work for this kind of well-hardened target is where Volume 4 begins.

When it is not a person — when it is a team

Not every infrastructure operation belongs to one individual. A shared favicon and a shared tech stack across ten sites can mean one operator running ten projects — or it can mean one small web agency building ten sites for ten different clients from the same in-house template.

The tell is usually in the analytics, but only if you read it carefully. An agency will often paste each client's own GA or AdSense ID into that client's site, so those IDs will not recur across the ten. But some agencies use a single Google account of their own to serve *some* clients — the newer ones, or those on a maintenance retainer — and hand-roll the tags for others. The pattern you see with an agency is uneven: GA4 IDs that match on some pairs and not others, in a way that does not line up with topic or geography. A single operator running ten fronts produces flat, boring agreement — the same identifiers everywhere.

Other signals separate the two. Different design systems across the "same" cluster — one site on a modern component library, another still on a 2018 theme — is far more typical of an agency serving different clients over years than of one operator building ten fronts in a sprint. Commit-time metadata that shifts timezone across supposedly linked repos points the same way; single operators do not spontaneously relocate between projects. Footer credits linking back to a "site by *studio-name*" line, even in small grey text, are the giveaway agencies leave on the sites of clients who did not pay for credit removal. WHOIS registrant emails at the agency's own domain, rather than at each client's, are another. Any two of these together and the correct attribution is *agency*, not individual.

When the infrastructure fingerprints agree but the account-level identifiers diverge, your attribution is to the *agency*, not to any one client. Say so on the ladder in exactly those words.

Attributing an agency's shared setup to a specific client is a false resolution, and a bad report a lawyer will pick apart in ten minutes. Worth stating plainly: an agency-level attribution is *still* real attribution. It names an actor — a real business with real staff, a bank account, and a reason to have built the site. It just names a different actor than the reader was hoping to name. Deliver the finding as the finding; do not stretch it to fit the reader's preferred answer.

A safe exercise

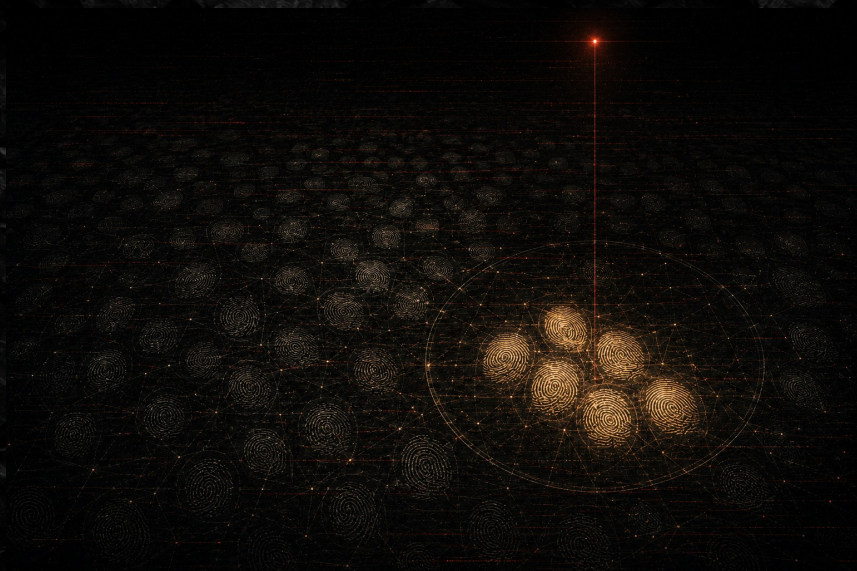
Pick three of your own sites or blogs — anything you personally run. For each one, inventory every operator-fingerprint on this chapter's list: analytics IDs, favicon hash, WordPress or CMS theme and plugin combination, historical WHOIS email if the domain is old enough, any public git repository that carries the site's code, error pages that leak internal hostnames, DMARC report address, contact-form headers. Now decide, one fingerprint at a time, which you are willing to leave in place and which you want to change before tomorrow. You will learn more from that hour than from any amount of reading.

Recap

- A server does not identify itself. The person who configured it usually does, through the tools they reuse.
- Fingerprint classes to check: analytics IDs, favicon hashes, tech-stack combinations, historical WHOIS emails, public git metadata, error-page leaks, DMARC report addresses, contact-form headers.
- Independence still rules. A favicon match and a stack match on one site is one signal. A favicon match on one site and an analytics-ID match on a second is two.
- The strongest operator-link signal available at this layer is the same account-scoped identifier — AdSense, GA, Pixel — appearing across supposedly unrelated sites.

- Wayback is your rescue when the current page has been scrubbed. Old About pages are where operators used to be careless.
- This chapter is a hinge. The moment you have a name, an email address, or a personal handle, the work becomes Volume 2 work.
- When the fingerprints belong to an agency, attribute to the agency. Do not launder a shared template into a personal identity.
- Some targets are built to defeat every technique here. Say so plainly; do not fabricate confidence you have not earned.

Chapter Eight moves from these active fingerprints to the passive traces a domain leaves in DNS over time — the resolution history that outlives the operator's careful clean-up.



CHAPTER EIGHT

Passive DNS and Fingerprints

The internet has a memory. It just does not label it.

A hosting provider advertises itself to a particular kind of customer. No logs. No identity checks. No history. Pay in something untraceable, spin a machine up, tear it down, walk away — nobody, the pitch goes, will know you were there. From inside the provider's control panel the pitch is true.

It is true only from inside the provider. On the day the customer first pointed a hostname at the machine, a public resolver somewhere in the world logged the DNS lookup and later

published it to an aggregator that keeps the record indefinitely. Three months later, when the customer quietly moved to a second provider, a different resolver saw the transition. And six months after that, when they stood up a mirror on a third provider and issued a fresh TLS certificate, a scan index published the *fingerprint* of that certificate's TLS stack — a 62-character shape that happened to match, exactly, the fingerprint of the first server on the first provider.

The customer's provider kept its promise. The internet kept its own memory.

The idea: the internet has a memory

Two kinds of memory, not the same thing.

The first is *passive DNS*, shortened to *pDNS*: the *history* of DNS resolutions across the world. Large public-resolver operators and research collectors log the answers their resolvers hand back, aggregate the logs across sensors, and publish — free with rate limits, or paid for volume. What you get is a time-series: this hostname resolved to this IP between date X and date Y, to a different IP before that, and to nothing at all before it was first observed. You did not query the target. Somebody else's users did, and the answer was written down.

The second is a *scan index*. Every day a handful of projects scan the visible internet — every IPv4 address, every common port — and record what each host looks like at the network layer: open

ports, TLS certificates, HTTP headers, landing pages, favicons. Published as a queryable index keyed by every fingerprint they capture. Some keep historical scans, giving each host a rough time-series over months and years.

Together, pDNS and scan indexes give you a time-history and a fingerprint corpus of the entire visible internet, without you having sent a packet to your target.

What passive DNS gives you

- **Historical resolution.** This hostname pointed at this IP on these dates. A change of hosting provider is a timestamped transition, visible months or years later.
- **Reverse-history.** This IP was the target of *these* hostnames over time. On a small, quiet IP block the reverse-history is a cluster of an operator's properties; on a huge shared host it is thousands of unrelated tenants.
- **Migration tracing.** You can watch an operator move — old IP fading out as caches expire, new IP fading in — and pin the migration to a calendar window. That window becomes a fingerprint of its own.
- **Cluster discovery.** Hostnames that all pointed to the same unusual IP within the same week, especially on a small VPS with three tenants on the box, are almost always the same operator. The rarer the IP, the tighter the cluster.

- **Ghost hostnames.** Names that resolved once, never again, but sit in the pDNS record forever. Ideal for enumerating a target's forgotten test environments — the hostname that was live for a weekend in 2019, resolved thirty times, and has been dead since.

What pDNS is not

Passive DNS only sees what public resolvers logged. A target running its own recursive resolver, or routing lookups through DNS-over-HTTPS or DNS-over-TLS to a stack that does not participate in a pDNS project, is largely invisible — the queries never touched a logging resolver. Coverage is uneven by region and year; older data is patchier, some ccTLDs barely represented.

Commercial pDNS carries the deepest archives and is paid. Free tiers exist but rate-limit hard, with short retention. Both come and go. Treat the *shape* as durable (someone, somewhere, is logging resolver answers) and the *specific door* as replaceable.

What scan indexes give you

- **Open-port surveys.** Every host on the visible internet, catalogued by open port and the *banner* it returned — the short string a service emits when you first connect, like an SSH version line or a mail-server greeting.
- **TLS-cert fingerprints.** The JARM fingerprint of a server's TLS stack clusters identical setups. Two servers with the same

JARM run the same TLS software, configured the same way, on the same port.

- **HTTP header fingerprints.** The Server string, the set of custom X- headers, framework signatures baked into error pages. A rare combination is a cluster signal.
- **Favicon hashes across the entire scan corpus.** Chapter Seven used the favicon at the scale of a single site; here you ask the index for every host on the internet that serves the exact favicon and get back an operator's whole property list in seconds.
- **Historical scan captures.** Some indexes preserve past scans, so a query for an IP returns a rough time-series — what banners it served six months ago, what certificate it used a year ago, when its Server header changed.

Fingerprint types you will actually use

Names change; the shapes below outlast their tools. Learn what each fingerprint *is* before you learn where to look it up.

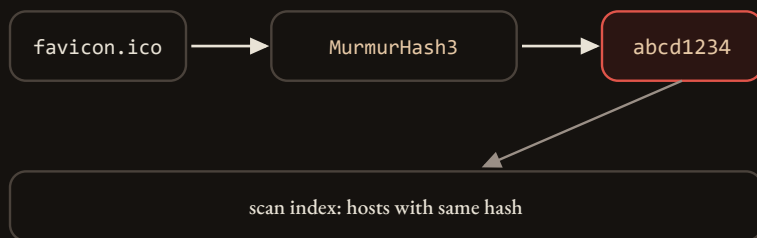
- **Favicon hash.** The site's icon file run through MurmurHash3 and encoded as a short signed integer. Every server serving the byte-identical favicon computes to the same hash — a near-perfect operator-link signal.
- **JARM.** A 62-character fingerprint of a server's TLS handshake behaviour — how the server responds when a client offers various handshake options. Identical JARMs

mean identical TLS-stack configuration. Common JARMs (default nginx, default Apache) mean nothing; a rare JARM is close to a signature.

- **JA3 / JA4 / JA4S.** JA3 is the older client-side TLS fingerprint (the handshake as sent by a client). JA4 is its 2023-era successor with cleaner properties and better resistance to trivial randomisation. JA4S is the server-side variant — and unlike JA3, JA4S is directly useful for OSINT because your target is usually a server. Where a scan index supports it, prefer JA4S over the older JARM for freshly-configured stacks; JARM is still the workhorse for the enormous back-catalogue of hosts already fingerprinted with it. Names change; the *shape* — "this server's TLS behaviour, compressed to a short string" — is what carries.
- **HTTP Server header.** The short string a web server sends identifying itself. Often lies or says nothing (nginx alone is worthless), but a rare custom value — `werkzeug/2.0.1 Python/3.9.6` on a hand-rolled admin panel — clusters hard across an operator's servers.
- **HTML title / meta hash.** The `<title>` of the landing page, or the hash of the `<meta>` block. Where **misconfigured status pages leak** — a scanner sees `Welcome to nginx!` alongside a forgotten `[Redacted] Panel v1.03` on a status endpoint the operator meant to firewall, and the title becomes a signature.

- **X- header sets.** Custom HTTP headers, always prefixed **X-**, that non-standard software adds to its responses. A specific combination in a specific order is often unique to one build of one operator's stack.

FAVICON HASH FINGERPRINT



one file, one hash, many hosts → one operator

A single favicon file, hashed with MurmurHash3, becomes one short string; a scan index searches every host on the visible internet for that exact string and returns the operator's entire property list — the same trick as Chapter Seven, run at internet scale.

The method, step by step

Step 1 — Anchor the domain or IP. From Chapters Five and Six you have the base domain, its externally-certified hostnames, and historical hostnames that no longer resolve. Every anchor multiplies the pDNS surface.

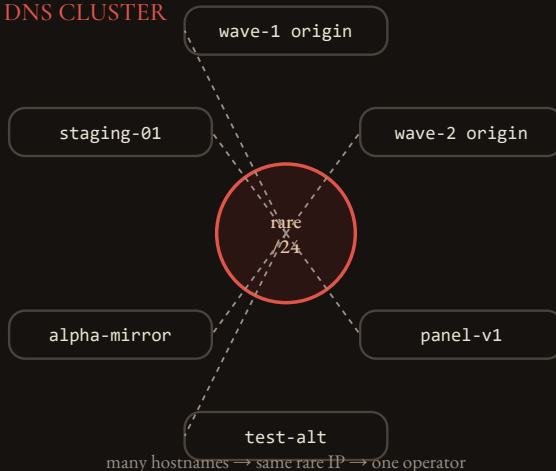
Step 2 — Pull the pDNS timeline. For each hostname, ask a pDNS service for every historical A, AAAA, CNAME, and NS

record it saw. Note every transition — the day an A record changed, the day nameservers moved, the day a CNAME appeared. Each transition is a datable moment.

Step 3 — Reverse the pDNS on each historical IP. Take the IPs from Step 2 one at a time and ask pDNS the reverse question: which *other* hostnames pointed at this IP during the same window? On a giant shared host you get thousands of unrelated tenants — noise. On a small, rare IP where a handful of hostnames pointed at it during the operator's tenancy, the co-tenants are almost always the same operator's other properties.

Step 4 — Query scan indexes for the current and historical IPs. Pull the scan-index record for each: open ports, banners, TLS certificates, JARM, HTTP headers, HTML titles, favicons. Read cold. Note anything unusual — an odd Server string, a non-default port serving HTTP, a status page that shouldn't be public, a title that reads like a private admin panel.

PASSIVE DNS CLUSTER



A single target hostname sits at the top; pDNS unfolds its historical IPs down one axis, and reverse-pDNS on each rare IP fans a cluster of co-tenanted hostnames out sideways. The scan-index record for each IP hangs off it as a small stack of fingerprints. The cluster is not one signal; it is a lattice.

Step 5 — Fingerprint the operator's TLS stack. Take the JARM of the operator's server and ask the scan index for every other host with the same JARM. A JARM shared by twelve million hosts means "default nginx" — worthless. A JARM shared by 47 hosts is a candidate cluster, each host a lead to be corroborated with pDNS neighbourhood and other fingerprints.

Try it (2026). Take a well-known public service — a big news site — and read its JARM in a scan index. Count how many other hosts share it. It will be six or seven

figures: the noise floor for that JARM. Now do the same for a small self-hosted service. The number will be much smaller — rarity is what makes a JARM diagnostic. The specific scan-index name in any tutorial may have died since; the exercise (compute a JARM, count matches, feel the rarity) does not.

Step 6 — Hash the favicon and search the scan index. Same technique as Chapter Seven, but instead of grepping a few candidate sites you ask the index for every host with this favicon hash. A rare hash returning three IPs on three different providers is a cluster you would never have found any other way. The exact recipe in a Python REPL — expected to rot, hedge accordingly — is `mmh3.hash(base64.encodebytes(open('favicon.ico','rb').read()))`; note that `encodebytes` inserts a newline every 76 characters and that is intentional, most scan indexes expect it. In a scan-index query today this looks like `http.favicon.hash:-1246753918` (Shodan syntax, signed 32-bit int). Different indexes use different sign conventions and different query keys; the *shape* — "give me every host serving this exact favicon" — is what carries.

Step 7 — Cluster the results. A candidate related-host from Step 5 is one signal. If the same host also shows up in Step 3's pDNS reverse-history on one of the operator's rare IPs, that is two signals aligning. If its favicon from Step 6 matches too, that is three. The ladder from Chapter Seven governs: independent

signals that agree earn *strong* when they are hard to explain by any rival hypothesis. Fingerprint agreements are almost always independent of pDNS agreements — a JARM was measured by a scanner, a pDNS entry was logged by a resolver — so they stack cleanly.

Step 8 — Rank honestly. Fingerprint clusters are seductive. A common JARM matches millions of nginx installs and proves nothing; a default WordPress favicon matches millions of blogs and proves nothing. A rare JARM shared by 47 hosts is gold; a rare favicon shared by three is close to a self-link. Before you count any fingerprint agreement as a signal, ask how many hosts on the whole internet share it. Millions means no signal at all. Dozens is worth what a chapter of work is worth.

A worked case, dead ends and all

A threat-intelligence team at a payments firm brought me two phishing waves from the same month. Wave one targeted their retail customers with a fake login page; wave two went after their business banking customers with a cosmetically different variant kit. Different domains, different registrars, different designs. Their question was narrow: same operator behind both waves, or two? Fictional case, real method.

WHOIS dead end. Both waves used domains registered through discount registrars with privacy on from day one, different registrars each. Historical WHOIS returned nothing older than

the current registration — both domains were fresh, created weeks before their wave. Chapter Five had nothing.

Cloudflare dead end. Both waves sat behind Cloudflare. Reverse-IP on a Cloudflare edge returns hundreds of thousands of unrelated tenants. Fully fronted, no pivot.

Certificate transparency dead end. Both domains held clean Let's Encrypt certificates issued the moment the phishing site went live. Neither certificate had a shared SAN with the other; no shared wildcard, no shared tenant.

Mail-record dead end. Both had **TXT** records for generic Google Workspace tenants and **MX** records pointing at Google, with different tenant identifiers on each. No SPF quirk that tied them together.

Four dead ends. This is roughly the point where a beginner reports insufficient signal and stops. It was, in this case, where the actual work started.

The pivot. I pulled the pDNS timeline for both hostnames. Both showed a Cloudflare edge IP from the day the hostname first resolved, and nothing else — except that for a two-hour window on the day the wave-two domain went live, before the operator finished putting Cloudflare in front of it, the hostname had briefly resolved *directly* to an origin IP on a small European VPS provider. And a week earlier, on wave one, an identical misconfiguration had leaked a direct origin resolution — to a

different IP on the same small provider's /24.

```
pDNS timeline, both waves
-----
wave-1 paylink-secure-verify.top
      2026-05-14 09:12 A 203.0.113.44
                          (EU VPS, small)
      2026-05-14 10:47 A 104.21.x.x
                          (Cloudflare edge)
      2026-05-14 10:47 ... stays Cloudflare
wave-2 business-pay-portal.top
      2026-05-30 14:03 A 203.0.113.61
                          (EU VPS, same /24)
      2026-05-30 15:58 A 172.67.x.x
                          (Cloudflare edge)
-----
two brief origin-leaks, same /24,
weeks apart
```

Two Cloudflare misconfigurations, weeks apart, both leaking origin to adjacent addresses on one small provider.

Both origin IPs were adjacent addresses in a /24 belonging to a small European reseller with maybe two thousand customers total. Not a coincidence; the operator had forgotten to front their real host for two hours on two separate days.

Fingerprints on the origin IPs. I queried a scan index for both. Both ran the same nginx version on the same non-standard port, both had the same JARM. That JARM was extremely rare — the index showed it on exactly 47 other hosts across the visible internet. Not millions. Forty-seven.


```
scan-index summary, JARM cluster
-----
JARM 2ad2ad0002ad2ad0002ad2ad2ad...
matches 49 hosts total (both origins
        plus 47 others)
common? no - near-zero baseline
common characteristics:
- nginx 1.21.4 on port 8443
- self-signed cert, CN=proxy
- HTTP header set:
    Server: nginx
    X-Frame-Options: DENY
    X-Panel-Build: 1.03
- favicon hash 1173925469
-----
X-Panel-Build is not a standard header
```

A hand-built reverse-proxy config, on a rare port, with a custom header and a distinctive favicon — a fingerprint of one operator's stack.

I read the scan record for each of the 47 hosts one at a time. Two served the byte-identical favicon the wave-one and wave-two phishing pages served, hash 1173925469. One, on a quiet Sunday, served a default Welcome to nginx! landing page — and, alongside it, a publicly-reachable status endpoint whose HTML title read [Redacted] Panel v1.03. That title matched the X-Panel-Build string from the JARM cluster, and matched a private phishing-kit panel advertised on a well-known criminal forum earlier that year.

Stack it honestly. First, the pDNS origin leak on the same rare /24 for both waves — moderate on its own, since a shared small

VPS is unusual but not conclusive. Second, the JARM shared with only 47 hosts, especially with the matching X-Panel-Build header — moderate-to-strong. Third, the favicon hash shared between the two phishing pages and two JARM-cluster hosts — independent, strong. Fourth, the status page whose title revealed the exact panel version and matched a named private phishing kit — independent, near-diagnostic on its own. Four independent signals, four different collection methods, all agreeing. I reported **strong** at the operator-cluster level: same operator behind both waves, running the same private kit, on infrastructure they briefly failed to hide, twice.

The client fed it into their internal fraud graph. I noted the caveat I always include: fingerprint clusters at the network layer identify the *tools and infrastructure*, not the human. Two colleagues sharing a kit and a hosting habit look the same at the JARM layer as one person alone. Chapter Nine closes that gap.

A second case: the one that doesn't close

A different case, same week. A well-operated adversary, low-volume collection operation. Fronted by a large CDN and staying fronted; no origin-leak windows in pDNS across a year of history. TLS terminated at the CDN, so the JARM I saw was the CDN's, shared by millions of hosts. Favicon the plain default. Headers generic. Landing pages carried no custom titles.

Every fingerprint was absent or too common to prove anything. Reverse-pDNS on the historical IPs returned dense clusters of unrelated tenants on shared hosts. This operator had done everything right at the network layer, and there was nothing to cluster.

Honest report, one paragraph: *no attribution possible from network-layer signals alone. Their infrastructure-layer opsec is competent; nothing in pDNS or the scan indexes rises above the noise floor. A different layer — content behaviour, human tradecraft, timing patterns on posted material — is the next place to look.* Not every method pays out. Saying so honestly is part of the method.

On buying data you did not need to buy

Commercial pDNS and scan-index services are legal to buy, and for serious infrastructure work you will end up buying at least one. Understand where the data came from before you route work through it.

Passive DNS is a live consent problem. The data is aggregated from public-resolver log streams — Google Public DNS, Cloudflare's 1.1.1.1, university and enterprise resolvers, and a long tail of smaller collectors. Some of those resolvers publish a plain, opt-in privacy policy that says "your queries may be shared with security researchers." Many do not. When you query a pDNS index for "every host example.com ever resolved to," you

are reading a database built partly from ordinary users' DNS lookups — lookups they did not know were being kept, sold, and searchable years later. That is uncomfortable ground. It is not made comfortable by the fact that the domain name in the query is public; the *fact of the lookup*, tied to a resolver and a timestamp, is behavioural data about somebody. Buying it does not launder it.

Scan-index data is less fraught. The visible internet is scanned by right of being visible — you are reading what the target's own server chose to respond with — but licence terms still vary and some scan indexes redistribute derivative fingerprints of hosts the operator would not have expected to be catalogued.

If you work in a regulated environment — EU-facing corporate DD, journalism with a lawyer, safeguarding work, any adversarial proceeding — read the provider's data-sourcing statement in full, ask specifically about consent and lawful basis on the pDNS side (not the scan-index side; those are different questions), get the answer in writing, and keep it in the case file next to the queries you ran. Ask your organisation's counsel where the line is for your jurisdiction *before* you route a client's engagement through a paid pDNS API — not after. The service is legal to buy; whether the use is defensible in your jurisdiction, and whether the source is defensible to a court, is on you.

A safe exercise

Take a domain you personally own — a hobby site, a spare project, an old blog. Pull the free pDNS history for it. Read the trail of every IP it ever resolved to, every nameserver it used, every migration between hosts you had forgotten about. If the domain is old enough, the trail is longer than you remember. Most people are startled by how much of their own hosting biography is visible to anyone who thinks to look.

Recap

- The internet keeps two kinds of memory: **passive DNS**, the time-history of DNS resolutions logged by public resolvers, and **scan indexes**, the fingerprint corpus of the visible internet from regular scans.
- pDNS gives historical resolutions, reverse-history on IPs, migration windows, cluster discovery on rare IPs, and ghost hostnames that resolved once and were never renewed.
- pDNS is silent on private-recursive setups and DoH/DoT stacks that never touched a logging resolver; commercial coverage is deeper than free.
- The fingerprints you will actually use are the **favicon hash**, **JARM** (and **JA4S** where the index supports it), the **HTTP Server header**, the **HTML title or meta hash**, and rare **X-header sets**. **JA3** is the older client-side cousin, mostly for defenders.

- Fingerprint clusters are seductive; a common JARM proves nothing and a rare JARM is a signature. The rarity is the signal, not the match.
- Read the licensing and provenance of any pDNS or scan data you buy — the source's lawful basis is on you, not on the vendor.
- Network-layer attribution identifies operator-tooling clusters, not humans; some targets do this layer right and give you nothing.

Chapter Nine is the capstone — one case that pulls every layer in this book together, from the ring around the dot through people-work, domains, certificates, content, signal-stacking, and everything in this chapter, into a single walk from a starting name to a defended finding.



CHAPTER NINE

The Full Hard Target

One case, every technique, honest limits.

Everything in this volume has been one instrument at a time. A cache. A deleted account. A photograph. A DNS record. Techniques in clean light, one after the other, the way a book has to teach them.

Real hard-target work does not arrive in that order. It arrives in a lump: six domains and a pseudonym, and a newsroom asking whether it is safe to name a person. The techniques come out of the bag as the case asks for them, not as a chapter list.

This chapter is one case, run end to end. I am going to use every layer from Chapters One through Eight without stopping to re-teach any of them — if you need to check what CT logs or passive DNS actually are, the earlier chapters are still there. What you are going to see here is the *shape* of a hard-target investigation, dead ends and all, and how it earns its way onto the ladder.

I will show you what did not work as carefully as what did. The clean version of this case would fit on a napkin. The honest version is why we have a chapter.

A note on this case. Everything below is an illustrative composite. Every domain, email address, phone number, coordinate, WHOIS record and personal name is fictional and does not correspond to any real registered domain, mailbox, person or property. Domain strings were selected to fall outside real ownership at the time of writing; any live-domain collision that emerges after publication is unintentional and does not name a real operator. Any resemblance to a real Torrey, a real Mikelis Ozoliņš or a real Baltic operator is coincidental.

The brief

A Baltic investigative newsroom has been circling a story for four months. Ahead of the 2024 and 2025 local election cycles in Latvia and Estonia, a network of small, plausibly named "community news" sites — village registers, neighbourhood diaries, one that publishes only school-board minutes — has been reliably surfacing a specific set of narratives at a specific tempo. Each site presents as independent. Each has a P.O. box in its footer. Each is written in what reads as competent local dialect. The network is publicly ascribed, in a few earlier reporters' notebooks, to a single operator known only by the pseudonym "Torrey."

The newsroom's editor is careful. She will not run a piece that names Torrey unless the attribution reaches at least the strong rung — three independent, non-collapsible signals, no self-link required — and she wants the dead ends listed alongside the findings, in the same font, so her lawyer can read the whole thing.

Our brief, from her, in one email: *is Torrey one person, one team, or a mirage — and if a person, who?* A written attribution report at the end, ranked honestly, with everything we did not do also written down.

That is the whole ask. We are not to interview, contact, or provoke the target. We are not to log into anything as anyone.

We are here to read the open web carefully and to say a hard true thing at the end of it.

The scope, in one line

Open sources only. No contact with the target, no contact with anyone adjacent to the target, no probing of any host beyond reading records the internet already exposes to anyone who asks. Every screen we look at we archive; every claim we make lands on the ladder with the evidence beside it.

Anchor

The newsroom hands us six domains. These are the network's public entry points — sites Torrey has been publicly associated with in earlier notebooks, none of them contested. This is where we start and, as far as scope is concerned, where we are permitted to start.

- [kalnina-vesti.lv](#) — village of Kalniņa, "community news"
- [riga-vidzemes-nedela.lv](#) — Riga inner-suburb weekly
- [daugavgriva-hronika.lv](#) — port district chronicle
- [tallinn-mustamae-teataja.ee](#) — Mustamäe district reporter
- [tartu-annelinn-uudised.ee](#) — Annelinn quarter news
- [päru-raeküla-teataja.ee](#) — Pärnu district weekly. (The display form carries Estonian ä/ü; in every DNS or CT query the domain resolves as `xn--prnu-raekla-teataja-gwb62c.ee` — the ACE/punycod form. This book uses the display form

throughout for readability; a working query uses the ACE form.)

Two categories: four Latvian, two Estonian. All six were registered between 2019 and 2022, according to the surface WHOIS reply that anyone can pull. Each publishes on a weekday cadence. Each has a masthead that names no editor. Each says, in its own footer, that it is *independent, community-supported, and non-partisan*.

We write the six names down in a case note with three columns — *claim, evidence, rung* — and nothing enters the report until it earns one.

Layer 1 — The surface says nothing

The first pass is the pass a competent reader would do in an evening. And the first pass says almost nothing.

Live WHOIS on all six returns the same shape: registrant redacted, admin redacted, tech redacted, privacy-service proxy in front. Two of the .lv records go through one Latvian privacy proxy; the other two through a different one. The two .ee records go through an Estonian service. Different proxies, no shared registrant string, no shared address.


```
whois - kalnina-vesti.lv (live)
-----
registrant: [privacy-proxy]
admin:      [privacy-proxy]
tech:       [privacy-proxy]
created:    2020-04-14
updated:    2023-11-02
name-server: ns1.cloudflare.com
name-server: ns2.cloudflare.com
-----
```

live public WHOIS, redacted at source

The content is careful. It is not obviously partisan and not obviously coordinated. Two months of front pages read like small-town journalism — road works, school fetes, a mild editorial about municipal transparency. The narratives the newsroom is worried about are there, but seeded lightly, in a paragraph three articles down, once every couple of weeks.

Every footer lists the same shape of contact: a P.O. box, a hello@ on the site's own domain, a Google Workspace tenant behind the mail exchanger. The P.O. boxes are different addresses in different cities. Nothing rhymes.

Every site sits behind Cloudflare. Every certificate is Let's Encrypt, ninety-day rotation, wildcard where present. No two share an IP at the front. No two share a mail server. The DNS looks, at a glance, like six unrelated people made six unrelated sites.

That is the mood. The target has read a book like this one. Nothing on the surface admits anything. Which means the surface is not the place to spend the day.

Layer 2 — Infrastructure fingerprints

Chapters Seven and Eight are how you get *behind* a surface that has been polished. We start pulling.

The first thing to check is the *page* itself. A Cloudflare front cannot hide what the origin *renders* — favicons, analytics tags, framework markers, the HTML shape. We fetch the six front pages and hash each favicon.

```
favicon hash — all six network domains
-----
kalnina-vesti.lv           -1246753918
riga-vidzemes-nedela.lv   -1246753918
daugavgriva-hronika.lv    -1246753918
tallinn-mustamae-tea.ee   -1246753918
tartu-annelinn-uud.ee     -1246753918
parnu-raekula-tea.ee      -1246753918
-----
```

mmh3 of favicon.ico, byte-identical

Six independent community sites do not accidentally ship the same favicon byte for byte. That is one signal. It is not enough — a template shop could have handed the same asset to six clients — but it is a hard start.

We view source. All six carry the same GA4 measurement ID: **G-9K2D4X7QH1**. GA4 IDs are tenant-scoped; they belong to

one Google Analytics property, controlled by one Google account. Six "independent" sites reporting to one analytics dashboard is a second signal, and this one is harder to hand-wave. A template shop shares CSS. It does not share the client's analytics tenant.

One of the six — [kalnina-vesti.lv](#) — carries a `humans.txt` file at the root, which appears to have been left in by accident. The file is empty except for a single comment line: `# built by MO, 2020`. Two initials. We note them and move on — initials alone earn nothing yet. They will land later when the same two letters reappear as the initials of the person the other layers are pointing at, at which point they will become one small confirming coincidence, not attribution.

Now we go behind Cloudflare. Passive DNS on the six domains shows the expected Cloudflare A-record history for most weeks. But on a single weekend in November 2023, two of the six — [daugavgriva-hronika.lv](#) and [pärnu-raeküla-teataja.ee](#) — briefly resolved to a non-Cloudflare address, a small European VPS provider, autonomous system AS200000-range, before flipping back on the Monday. Someone rotated a certificate or restarted a proxy and, for forty-eight hours, the origin sat exposed.

We pull scan-index records on that origin IP for the weekend in question. Both domains served from the same host, on the same port, with the same TLS stack. JARM fingerprint matches to the byte.


```
scan-index — weekend of 2023-11-04
-----
host    [origin IP redacted]
443/tcp
  jarm   2ad2ad0002ad2ad00042d42d0000..
  cert   CN=daugavgriva-hronika.lv
  cert   CN=xn--prnu-raekla-teataja-gwb62c.ee
  server nginx/1.24.0
-----
```

third-party internet-scan index, historical

Two of the six network sites, physically colocated, on one small VPS. Combined with the favicon and the GA4 tenant, we have three genuinely independent signals pointing at a single operator behind the network. The *operator-cluster* attribution now sits at strong. Torrey is not a mirage. Torrey is one hand. We do not yet know whose.

Layer 3 — The deleted room

The next lift comes from something Torrey did years before Torrey existed.

The GA4 ID we lifted — G-9K2D4X7QH1 — is a hard identifier. We search for it across public code and page corpora, and it appears on a seventh site: baltika-kaleidoscope.eu. A food-and-travel blog. Nothing overtly political. Different topic, different visual template, different footer language.

The current version of the blog has no author byline. But the Wayback Machine (Chapter One) has captured the About page eleven times between 2019 and 2024. Six of those captures —

everything before June 2022 — list a byline: Mikelis Ozoliņš, resident of Jūrmala, contact mik.ozolins@yandex.ru. The June 2022 capture is the first one where the byline and email have been scrubbed.

```
web.archive.org — /about (2021-08-12)
```

```
-----
```

```
"About the author"
```

```
"Mikelis Ozoliņš — Jūrmala-based writer  
on food, travel and coastal culture.  
Reach me: mik.ozolins@yandex.ru"
```

```
-----
```

archived page, captured 2021-08-12

Someone scrubbed the byline in mid-2022 — which is roughly when the Torrey network started publishing at cadence, and roughly when a competent operator would be pulling old identity artefacts off the surface web.

Cache alone can be argued with — a Wayback capture is one source. So we look for ecosystem quotes (Chapter One again). A small Latvian local-news aggregator, in a 2020 round-up piece about restaurants in the Kurzeme region, quotes a full paragraph from a baltika-kaleidoscope.eu post, with attribution: "*as Mikelis Ozoliņš writes in his diary of the summer, [quoted paragraph].*" Byline and city. Independent site. Not a mirror.

That is Chapter Two territory now — reconstructing a deleted identity from the ring around it. The FOAF pass on the pre-2022 About page turns up two more corroborations: a

friend-of-the-blog post from an Estonian food writer in 2021 addressing "Mikelis" by first name and thanking him for a Jurmala restaurant recommendation, and a comment thread on a competing Latvian blog where "Mikelis O." appears in a signature under a comment posted from the same yandex.ru email visible in the About page.

So a person exists: Mikelis Ozoliņš, Jurmala, food blogger, active 2019–2022, then scrubbed. And that person's GA4 tenant, still live and unrenamed, is now measuring the Torrey network. That is not proof — it is a very strong pull. A shared analytics tenant across a scrubbed personal blog and a network of anonymous political sites is a two-source vote for a single operator behind both.

Layer 4 — The photograph

Now we go where Chapters Three and Four live.

One of Mikelis's 2021 posts on baltika-kaleidoscope.eu — captured cleanly in three Wayback snapshots — carries a photograph with no caption and no visible EXIF. The composition is odd: a strip of pavement, the top of a low white building at the frame's corner, a slice of pale winter sky, and one plastic sunlounger on what is clearly a hotel balcony. The image reads as an "I'm somewhere" post rather than a food photograph, which is why it is worth chronolocating: it *dates* Mikelis's movement pattern.

Geolocation first. Visible in the frame:

- A sign fragment across the street, in Cyrillic, only three letters visible: ...pop...
- Another sign, further back, in Latin-script hotel-brand type, partial: ...ISON
- A specific balcony-rail weld pattern — narrow verticals every ~90mm, an atypical spacing for European hotel builds
- The angle of the sun and the shadow of the balcony above put the building's face at roughly south-south-east

Cyrillic on street signage plus a Radisson-brand fragment plus the balcony architecture is a small enough intersection to be worth checking against a shortlist. Latin-alphabet hotel brands with Cyrillic-country properties: Belarus, Russia, parts of Central Asia. The winter light, the palm-adjacent planting visible along the pavement, and the specific balcony-weld pattern narrow it further. Street view along Kurortny Avenue in Sochi shows a hotel with that exact balcony pattern and a signage sight-line that resolves the ...pop... fragment to a Курортный street sign across the road.

We do not claim more than we can see. The geolocation is: almost certainly a specific hotel on Kurortny Avenue, Sochi. Confidence: strong, with the residual that a second Radisson-brand hotel elsewhere on the Black Sea coast could carry a similar balcony spec. Named residual, not dismissed.

Now the date. Chapter Four discipline. The sun angle on that south-south-east façade, given a Sochi latitude, brackets to a morning in late winter — roughly late January through late February. Visible on a rooftop across the way there is a thin sheet of frost surviving in shade, which in Sochi puts us in the colder half of that window. And on a bus-stop across the street, at the edge of frame, a fragment of a concert poster is legible enough to read "Зимний джаз" and a two-digit day. That festival, per public event archives, ran on specific dates in February 2021.

So: Mikelis, in February 2021, was in Sochi. That is not by itself an attribution finding. It is a *pattern* fact — a Jurmala-based food blogger travelled to a Russian Black Sea resort in an off-season month — and the newsroom will use it later. For our chapter, it is a chronolocated anchor: the deleted identity had a physical, dateable presence.

Layer 5 — WHOIS history

Now Chapter Five, and the vote that binds the two sides of the case together.

Live WHOIS on baltika-kaleidoscope.eu is redacted. Historical WHOIS on the same domain, from a pre-GDPR archive dated 2017, returns a clean registrant email: ozolins.m@inbox.lv. Different email from the mik.ozolins@yandex.ru visible on the About page — but the same person, on a Latvian mailbox

provider, in a plausible surname-first form.

That single record does not do the work alone. What does the work is what turns up when we pull historical WHOIS on the *Torrey network* domains — specifically the two that briefly leaked their origin in Layer 2, *daugavgriva-hronika.lv* and *pärnu-raeküla-teataja.ee*. Both were originally registered in 2017, three years before they carried a single article, under the same *ozolins.m@inbox.lv*. Both records were re-cloaked behind a privacy proxy in 2018, when the .lv and .ee registries began honouring GDPR-shaped redaction requests.

```
historical whois (pre-GDPR archive)
-----
domain    daugavgriva-hronika.lv
created   2017-05-22
registrant-email  ozolins.m@inbox.lv
registrant-name    M. Ozolins
registrant-city    Jurmala
-----
```

archived registry snapshot, 2017

The same record shape appears for *pärnu-raeküla-teataja.ee*, dated three months later. Two of the six *Torrey network* domains, in their pre-redaction form, registered under Mikelis Ozoliņš's personal email, from a *Jurmala* address, three years before the network started publishing.

This is not one vote. This is two independent domain-registration records, on two different national registries, agreeing with each other and with the food blog's own

pre-redaction record. The pull is very hard. It does not, by itself, prove that Mikelis operates the network *now* — a domain can change hands quietly, and a WHOIS from 2017 is not a statement about 2024. But combined with the GA4 tenant match and the operator-cluster fingerprint, it is a substantial third rope.

Layer 6 — CT and forgotten staging

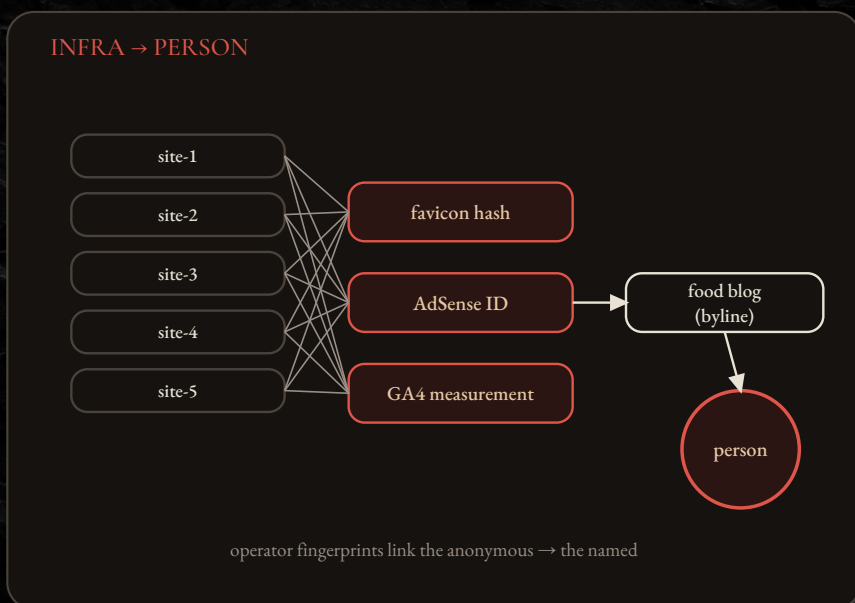
Chapter Six. Certificate Transparency remembers what an operator forgot to hide.

We pull CT for the six live Torrey domains. Nothing exotic — Let's Encrypt certs on the apex and `www`, ninety-day rotation, no wildcards leaking subdomains. Clean.

We pull CT for `baltika-kaleidoscope.eu`. And here is where the operator's *habits* start to speak. In August 2020, CT records a precertificate issuance for `dev.baltika-kaleidoscope.eu` — a development subdomain, presumably long since taken down. The pattern is standard: `dev.` prefix. What is more interesting is the *suffix* pattern on a second precertificate two weeks later: `dev-uat.baltika-kaleidoscope.eu`. A `-uat` suffix is a personal convention. Not wrong, not rare in the abstract, but a *particular* person's habit.

We check the CT history of the network domains for that same fingerprint. And on one of them — `tartu-annelinn-uudised.ee` — a CT record from January 2022 shows a precertificate for

dev-uat.tartu-annelinn-uudised.ee. Same prefix. Same suffix. Same convention, on a site whose surface WHOIS names no one.



Six Torrey network domains at the top, baltika-kaleidoscope.eu off to the right, and four independent bridges crossing between them: shared GA4 tenant across all seven; two Torrey origins colocated on one VPS by JARM; two Torrey domains sharing a pre-2018 registrant email with the food blog; and the dev-uat. naming habit repeated across the food blog and one network site.

An operator-habit fingerprint is soft on its own. It becomes hard when it sits inside a case where three other bridges are already carrying weight. It moves nothing on its own; it moves the ladder-rung by one careful notch when everything else is already in place.

Layer 7 — Cross-platform closure

We do not need Volume 2 to make the attribution. We have enough. But the newsroom wants to know that the man behind the email is the man in the tagged photograph — the *face* on the *name*.

ozolins.m@inbox.lv is a Volume-2 style pivot. Reversing that email across the public web, using the same cross-platform toolkit anyone in this trade uses, returns three hits that survive verification:

- A dormant Twitter/X account, last active 2019, with a bio linking to baltika-kaleidoscope.eu and a display name of "M. Ozoliņš."
- A profile page on a Latvian business-directory site — a self-published listing for a freelance copywriting service, contact ozolins.m@inbox.lv, city Jurmala, first name Mikelis.
- A decade-old personal blog on a Latvian blogging platform, active 2010–2013, with several self-portrait photographs.

The self-portrait blog is what lets the face onto the ladder — carefully. On a public Facebook page belonging to a Jurmala chamber of commerce, a 2018 photograph of a small-business event is captioned with three names, one of them "Mikelis Ozoliņš." The strongest tie between that photograph and the personal email is the Twitter/X bio: a dormant account whose bio explicitly links to baltika-kaleidoscope.eu and whose display

name matches "M. Ozoliņš." That is a first-party assertion by an account under Mikelis's own control that ties his name to the food blog. It is the leg the visual attribution rests on.

A visible physical mark on the left side of the neck — a small pigmented nevus visible in three 2011 self-portraits on the personal blog and in the 2018 chamber photograph — is **auxiliary corroboration**, not an independent leg. A mole is not a diagnostic biometric; under cross-examination a competent opposing expert flattens it in ten minutes. It matters only as a consistency check on top of the Twitter-bio self-link and the shared-name-and-city pattern. We call it what it is in the report.

Individual attribution now sits on three genuinely independent legs, with one carefully-labelled residual:

- **Leg 1 (historical WHOIS).** Pre-2018 records on two Torrey network domains under Mikelis's personal email, matching the food blog's own pre-redaction record. Two domain records share one operator's admin decision, so we count them as one leg — the same "one procurement decision viewed twice" that Chapter Five warned about — but the additional match on the *unrelated* food blog registered separately is a second, genuinely independent occurrence of the same identifier.
- **Leg 2 (shared GA4 tenant).** GA4 measurement ID G-9K2D4X7QH1 across the Torrey network and the food blog. Independent of the WHOIS leg because it is a live

Google account rather than a historical registration.

- **Leg 3 (public self-link).** The Twitter/X account's bio linking "M. Ozoliņš" to baltika-kaleidoscope.eu. First-party assertion, independent of both above.
- **Reinforcing, not counted:** the Jurmala chamber-of-commerce photograph and the mole consistency check are *auxiliary corroboration* on top of Leg 3, not a fourth vote. Naming them as reinforcement (and refusing to count them as an independent leg) keeps the rung math honest.

Rung on individual attribution: strong. Not confirmed — Mikelis has never publicly claimed operation of the *Torrey network* itself, and the newsroom would need either a self-link naming the Torrey properties or a first-party document to move that last rung.

The dead ends we do not usually show

Six threads that went nowhere, listed by name so the report is honest:

- **Reverse image on the hotel-window photograph.** Returned zero visually similar images across every reverse-image tool I tried. The photograph was too idiosyncratic to have been reposted anywhere, and too plain to be a stock crop. Geolocation had to be done by hand from the sign fragments and the balcony architecture, not from a

match.

- Four of the six Torrey origins never leaked. Only two of the six domains briefly rotated off Cloudflare in November 2023. The remaining four stayed opaque throughout. We can point at the cluster, but we cannot see the machine on which four of the six actually live.
- The P.O. box addresses. All six footer P.O. boxes resolve, on closer inspection, to a single shared mail-forwarding service that markets itself to small businesses. Hundreds of clients use it. The addresses are not evidence of anything about the operator; they are evidence that the operator hired a mail-forwarding company.
- DMARC reports on baltika-kaleidoscope.eu. Public DNS shows a DMARC record with an `rua=mailto:` pointer, but the destination is a privacy-forwarded mailbox on a service that specialises in DMARC aggregation. The trail dies at the forwarder. No registrant, no metadata visible from outside.
- JARM cluster width. The JARM fingerprint we lifted on the origin VPS in Layer 2 has, at the time of the scan, roughly 6,400 matching hosts across the internet-wide scan corpus. That is a wide cluster. It only became useful after filtering by autonomous system and country of hosting — the same JARM in the same AS in the same small European jurisdiction narrowed the shortlist to two hosts. Without that filter the fingerprint alone would have said nothing.

- The chamber-of-commerce namesake risk. "Mikelis Ozoliņš" is not the rarest name in Latvia. The 2018 chamber photograph could easily have been a different Mikelis Ozoliņš — same name, same town, unrelated life. What closed the residual on *individual* attribution was the Twitter/X account's own bio linking to baltika-kaleidoscope.eu — a first-party self-link — with the mole as an auxiliary visual consistency check. Absent the Twitter self-link the visual would have stayed *moderate* and the case would have shipped one rung lower; a mole alone is corroboration, not attribution.

Dead ends are inventory. A case with none was decorated, not investigated.

What the report actually says

The written attribution report, in one page, delivered to the newsroom's editor and her lawyer:

"The strongest independent evidence points at Mikelis Ozoliņš, resident of Jurmala, Latvia, as the operator of the Torrey network of six community-news sites in Latvia and Estonia. Attribution rung: strong. Current control by Mikelis is not confirmed. The 2017 WHOIS records establish him as the registrant of two of the network domains as of 2017; live registrations are redacted and a domain can change hands quietly. His authorship of the linked food blog baltika-kaleidoscope.eu is a first-party self-link (Twitter/X bio); his registration of the Torrey domains is a

historical record, not a present-tense statement of control.

Independent signals: (i) pre-2018 historical WHOIS on two of the six Torrey domains under ozolins.m@inbox.lv, which is also the pre-2018 registrant of baltika-kaleidoscope.eu and appears on independent public directories under Mikelis's full name; (ii) shared GA4 measurement ID G-9K2D4X7QH1 across all six Torrey network sites and the food blog — measured as one Google account still live and unrenamed since the 2019 blog launch, not just a reused snippet; (iii) shared favicon hash -1246753918 (Shodan signed 32-bit convention) and matching JARM origin fingerprint across two Torrey origins that briefly leaked off Cloudflare in November 2023, both colocated on a single VPS; (iv) operator-habit CT fingerprint — the dev-uat. subdomain naming convention — shared between the food blog and one network site.

Not confirmed: (a) that Mikelis retains present-tense operational control of the network — no first-party statement, no self-link naming the Torrey properties, and no document to that effect has been observed; (b) that the Torrey network is a solo operation rather than a small team using Mikelis's Google account and hosting habits. To move the finding from strong to confirmed would require a public self-link, a first-party document naming Mikelis as current operator, or a positive on-record confirmation from Mikelis himself — none of which we sought. To weaken the finding to moderate would require plausible evidence of a

domain transfer between 2017 and today for the two registered Torrey domains, or evidence that the shared GA4 tenant is administered by someone other than Mikelis."

The line we did not cross

We did not probe any host. Every fetch was a read of a public page or a public record. We did not attempt to reach an admin panel, a login page, or any non-linked endpoint on any of the seven domains.

We did not log into any platform as anyone other than a reader looking at open content. We did not create accounts to see private profiles, and we did not join any closed group.

We did not contact Mikelis. We did not contact any friend, colleague, or family member of Mikelis. We did not use his 2018 chamber-of-commerce co-attendees as a way to triangulate his current whereabouts.

We did not publish, and did not include in the report, a private home address, even though the geolocation of a separate 2019 blog photograph would have narrowed it to a single Jurmala street. The address was not the ask; the identity was.

The professional is the one who knows where to stop.

Recap

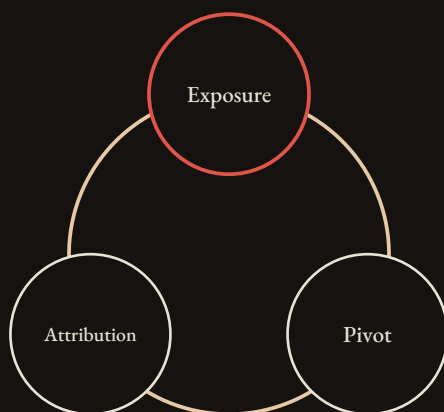
- Use every layer the case needs and no more; techniques are for questions, not for showing off.

- Independent signals must be *actually* independent — three facts arriving through one account are one source, not three votes.
- Historical records (WHOIS pre-2018, Wayback captures, CT logs, pDNS history) are what let you see what an operator scrubbed.
- Cloudflare hides an origin only as long as the origin never once forgets to hide.
- Operator-habit fingerprints — subdomain conventions, analytics IDs, favicon hashes — move nothing alone and matter a lot when three other bridges are already carrying weight.
- Dead ends are inventory. List them by name in the report; do not hide them.
- Chronolocation is often the finding, not the flourish — a *when* pattern can matter more to a story than a *where*.
- End every case with a written report the client's lawyer can read, with claims, evidence, rung, and *not-done* in one document.
- The rung must match the evidence. Strong is not confirmed, and confirmed requires a first-party link no amount of stacking can substitute for.

The loop, one last time

Even at this scale, the shape of the work was still the shape it always is. A network of anonymous fronts exposed a fingerprint (favicon, GA4, JARM). Each fingerprint pivoted to a new surface (a scrubbed blog, a leaked origin, a CT precertificate). Each pivot fed the attribution ladder one rung. Nothing was skipped, nothing was leapt.

THE LOOP



one clue, run in a circle until it points at a person

Exposure to pivot to attribution — the same loop as the first case in Volume 1, at a larger radius and against a target that had actually tried to hide.

Turn the page. Volume 4 is about turning this skill into a craft.

What You Can Do Now

— *and what comes next.*

Stop for a second and look at how far you have come.

In Volume 1 you learned to pull one thread. In Volume 2 you learned to weave threads from many platforms into one confident person. In Volume 3 you learned to read the places nobody meant to leave a thread — the deleted account, the captionless photograph, the certificate log, the misconfigured origin. You learned that "gone" is almost always "displaced," and that the world keeps records the operator was not thinking about when they set the record.

Something else changed for you in this book, too. You stopped needing the target to cooperate. In Volumes 1 and 2, most of the work was reading what a person had put in the open. Here, most of the work was reading what a person had *tried to hide* — and doing it without touching a lock, without a login, without a bypass, using nothing but public records read carefully. That is a much colder feeling. Sit with it. It is the feeling professionals talk about when they say the work is not always fun. It is the feeling that separates the professional from the enthusiast.

Keep the habits, not the tools

Platforms will change. Cloudflare will change. Historical-WHOIS vendors will merge or die. New scan indexes will appear. None of that matters. The *shape* of the work does not change: name the legitimate question, anchor what you have, walk the records the world already made public, count independent signals honestly, weigh every finding on the ladder, and take only what the question needed. Learn that, and every new tool is just another door into the same method.

Where to keep sharpening

- **Work yourself, hard-target style.** Delete an old blog. Then try to reconstruct it from the outside. Then close every hole you find. Best exercise in this volume.
- **CTF and verification challenges.** The good ones now include chronolocation, geolocation and infrastructure-pivot rounds. They are the safest way to make the eye faster.
- **Missing-persons search events** (such as Trace Labs) — meaningful, careful, alongside people who have done this for years, inside a firm no-contact rule.
- **Read defenders.** The other side of what you just learned is asset-management and attack-surface work. Their write-ups on their own leaks read like your case notes in reverse — and they teach you what leaks matter and what does not.

A bridge to Volume Four

This book was about the buried places. Volume 4 — The Professional — is about turning the skill into a craft you can be paid for and trusted with. Decision-grade reports.

Chain-of-custody. Vetting engagements. Counter-OSINT — how a target defends against exactly what you just learned. Automation you can actually rely on. And the honest ethics of a working investigator, not a hobbyist.

For now, you can read a place that does not want to be read, and know honestly what you can and cannot say about it. That is a lot. Keep your notes, keep the discipline. See you in Volume 4.

— Yehor

Appendix A — A Working Toolbox (2026)

The shape of tools that still work, and how to think about them.

This appendix names *shapes* of tools, not endorsements. Every specific service listed here may be gone, paid, or worse by the time you read this. What matters is the shape — knowing *what kind of tool* you are reaching for tells you what to look for when the current one dies.

Archives and caches

- General web archives. The Wayback Machine (web.archive.org) is the workhorse — reads the full capture timeline, supports URL-pattern search (web.archive.org/web/*/example.com/*). Sibling: archive.today / archive.ph (single-shot snapshots, harder to censor, useful when Wayback has been retroactively blocked). National web archives — UK Web Archive, Portugal's Arquivo.pt, Library of Congress — often preserve local content the global archives missed.
- Search-engine cache leftovers. Google's cache: operator is effectively dead as of 2024. Bing's cache limps on. Do not build a method on either.

- **Community snapshot rings.** Small archives run by researchers or activists sometimes preserve entire subcultures (chan-boards, defunct forums). Shape: search for the domain plus "archive" or "mirror" plus a small niche keyword.

Reverse-image and photo geolocation

- **Reverse-image engines.** Google Images, Yandex, Bing Visual Search, TinEye. Yandex is often the strongest for photo-of-a-place. Feed *targeted crops* (a sign, a distinctive corner) — full-frame reverse-image usually returns noise.
- **Face-search engines.** PimEyes-style services exist but are commercial, legally sensitive (GDPR, BIPA), and out of scope for most personal work. Use only inside an authorised engagement with a documented lawful basis.
- **Sun / shadow calculators.** SunCalc-style web apps (compute solar azimuth at a given date and location). Extremely useful for chronolocation.
- **Historical weather archives.** OpenWeather, Meteostat, and government meteorological services publish hourly weather archives you can query for a candidate date and location.
- **Historical satellite.** Sentinel Hub, Google Earth's time-slider, Planet's public briefings. See when a construction site appeared between passes.
- **Street-view walks.** Google Street View, Yandex Panoramas, Baidu Map, Mapillary, KartaView. Use whichever has

coverage of the region — Mapillary is community-uploaded and often the only source in less-covered areas.

Domain, DNS, and WHOIS

- **Live WHOIS.** The registrar's whois output, plus who.is, whoiscologne.io-shape front-ends. Read every field, especially nameservers.
- **Historical WHOIS.** WhoisXMLAPI, DomainTools, SecurityTrails and similar. Commercial and paid; free tiers rate-limit hard. This is the single lever most likely to give you a pre-2018 registrant email — worth the cost if the case is real.
- **DNS.** dig, nslookup, or web front-ends like mxtoolbox.com. Read A, AAAA, MX, NS, TXT, CNAME — every record type says something different.
- **Passive DNS.** SecurityTrails, VirusTotal (free tier surprisingly usable), Farsight DNSDB (enterprise, gold standard). Microsoft retired the RiskIQ-derived Defender Threat Intelligence portal in 2024; if you are reading a guide older than that which still names it, mentally translate to SecurityTrails or DNSDB for the same shape of data.

Certificate transparency

- **CT search interfaces.** crt.sh (Sectigo, free, workhorse), Censys certificate search, Google's CT search. Read every SAN in every cert; pay attention to precertificates.

- **CT monitors.** Subscribe to a domain and get notified on every new cert. `certstream-shape` services do this in real time.

Scan indexes

- **Shodan** — the classic. Every open port on the visible internet, queryable, with fingerprint filters.
- **Censys** — cleaner cert and TLS data, strong JARM support.
- **ZoomEye** — Chinese scan index, often has coverage the Western ones miss.
- **BinaryEdge** and **Onyphe** — additional scan corpora worth knowing.
- **FOFA** — Chinese, favicon-hash search among the strongest.

Infrastructure fingerprints

- **Favicon hashing** — the MurmurHash3 of the raw favicon bytes, base64-encoded (with newlines every 76 characters, which is intentional and load-bearing), is the format most scan indexes accept. In a Python REPL: `import mmh3, base64; mmh3.hash(base64.encodebytes(open('favicon.ico', 'rb').read()))`. In Shodan today: `http.favicon.hash:-1246753918` (signed 32-bit int; other indexes use different sign conventions and different query keys — the shape is what carries).

- JARM — 62-character server-side TLS-stack fingerprint. Open-source tool from Salesforce; scan indexes surface JARM natively. Still the workhorse for the enormous back-catalogue of hosts already fingerprinted with it.
- JA4S — the 2023-era successor to JARM's world, server-side and directly OSINT-relevant. Prefer it over JARM for freshly-configured stacks where the index supports it; use JARM everywhere else. JA4 (client-side) and JA3 (older client-side) exist and are mostly used by defenders identifying malware families.
- HTTP header fingerprints — Wappalyzer, BuiltWith, Whatweb. Detect frameworks, CDNs, CMSES, analytics stacks.

Analytics-ID search

- Google Analytics ID search — SpyOnWeb, DNSlytics, Analyzer-style services. Feed a UA- or G- ID, get back other sites carrying it. Also works for AdSense pub-, Facebook Pixel IDs, Yandex.Metrica counters.

A note on cost

Half of the shapes above have free-tier front doors, half are gated behind paid services. Do not front the personal cost of a commercial vendor for casual curiosity. If the case is real — a paying client, a legitimate journalistic public interest, a documented safeguarding matter — the cost is a line item. Otherwise, stay in the free tier and accept its limits.

Appendix B — Templates

Copy, paste, use. Written for real cases.

These are working shells. Fill in, delete what does not apply, keep what does.

B.1 The one-line scope

Before the first pivot, write this. If you cannot, you have curiosity, not a case.

```
I am looking into [subject] for [client / editor / self] to  
answer  
[specific legitimate question] on the basis of [lawful basis  
: contract,  
public interest, safeguarding, own footprint]. In scope: [wh  
at].  
Out of scope: [what]. Stop conditions: [what would end this]  
.
```

scope: one line

B.2 Geolocation report card

Photo: [URL / hash / where it came from, and when I got it]
Capture-time claim (if any): [what the source said]
Provenance chain: [how I got the file – screenshot? reupload
? original?]

Excludes (naked-eye pass, no tools):

- [element] → excludes [region / hemisphere / country]
- ...

Concentrates (highest-signal elements only):

- [element] → concentrates on [region]
- [element] → concentrates on [region]

Targeted reverse-image / street-view / satellite work:

- [what I looked at, what returned]

Confirmed location: [address or coordinate range]

Independent axes confirming: [1, 2, 3]

Rung: weak / moderate / strong / confirmed

Uncertainty note: [what I could not confirm]

geolocation card

B.3 Chronolocation report card

Photo / video: [reference]

Fast-clock signals (astronomical):

- Sun direction: [reading]
- Shadow-to-object length ratio: [reading]
- Moon phase (if visible): [reading]

Slow-clock signals (seasonal / weather):

- Foliage: [state]
- Snow / frost / rain traces: [state]
- Weather archive for candidate date at candidate location: [match / conflict]
- Manmade calendar signals: [poster, TV screen, wall calendar]
- Fashion / tech dating cues: [phone, car, streetwear]

Date bracket: [narrowest defensible range]

Rung: weak / moderate / strong / confirmed

Uncertainty note: [what I could not narrow]

chronolocation card

B.4 WHOIS and DNS trail table

Every row is a claim someone else will need to verify a year from now, after the live record has changed. Preserve first, cite second.

Domain: [name]
Examiner: [your name]
Examined on: [YYYY-MM-DD, timezone]
Tool versions: [whois client X.Y, dig X.Y, passive-DNS provider + query date]

Live WHOIS

- Registrar: [name]
- Created: [YYYY-MM-DD]
- Updated: [YYYY-MM-DD]
- Expiry: [YYYY-MM-DD]
- Status flags: [list]
- Nameservers: [list]
- Registrant: [redacted / name]
- Source classification: primary (registry) / secondary (aggregator) / broker
- Preservation: [Wayback URL | Perma.cc ID | archive.today ID] captured [YYYY-MM-DD HH:MM]
- Downloaded artefact SHA-256: [hash], stored at [case-file path]

Historical WHOIS

- Earliest record: [date] – [registrant email / name if any]
Source: [provider + product], classification: [primary / secondary / broker]
Preservation: [snapshot ID + timestamp], SHA-256: [hash]
- Transition 1: [date] – [what changed]
Source, classification, preservation, hash: [as above]
- Transition 2: [date] – [what changed]
Source, classification, preservation, hash: [as above]

DNS today

- A / AAAA: [IP list] – resolver [name], captured [YYYY-MM-DD HH:MM]
- MX: [mail hosts]
- NS: [nameservers]
- TXT: [records, especially verification tokens]
- CNAME: [aliases]
- Preservation: [dig +short output stored at path], SHA-256: [hash]

Passive DNS

- IP history for [domain]: [chronology]
- Reverse-history for [current IP]: [related hostnames if any]
- Provider: [name], query date: [YYYY-MM-DD], classification : broker
(pDNS is always a broker view; cite the provider, not "the internet")
- Preservation: exported JSON at [path], SHA-256: [hash]

Notable signals: [what stands out]

Not preserved / could not preserve: [what, and why]

whois trail

B.5 Infrastructure pivot map

Target site(s): [list]

Fingerprints observed:

- Favicon hash: [value] → matched on [N] other hosts (source: [scanner + date])
- GA4 / UA / AdSense ID: [value] → matched on [sites] (source, date)
- JARM: [value] → matched on [N] hosts (source, date; note cluster width)
- HTTP Server / X- headers: [notable strings]
- Historical WHOIS emails: [leaked addresses]
- humans.txt / security.txt: [content]

Independent votes for one operator:

1. [signal] on [sites] – why independent of votes 2 and 3
2. [signal] on [sites] – why independent of votes 1 and 3
3. [signal] on [sites] – why independent of votes 1 and 2

Attribution:

- Operator-cluster rung: [weak / moderate / strong / confirmed]
- Person rung: [weak / moderate / strong / confirmed]
- Residual namesake / agency ambiguity: [what would move this]

infra pivot

B.6 The hard-target attribution report

The written deliverable. Short is better than long — a client should be able to read it in five minutes and act on it. Every claim carries its source, its classification, and its preservation. The report must force you to name what you did *not* confirm; a report that only lists findings is a marketing document, not evidence.

Subject: [name / handle / cluster]
Prepared by: [examiner name and role]
Prepared on: [YYYY-MM-DD]
Prepared for: [client / editor / internal file]
Report version: [v1.0 – increment on any material change; keep a changelog line]
Changelog: [v1.0 initial; v1.1 corrected X per Y on YYYY-MM-DD]

Classification and handling

- Distribution: [Confidential – client only | Restricted – named recipients | Internal]
- Handling: [encrypt at rest, no forwarding without written approval, retention window N months]
- Do not quote in external publication without written approval from [name]

Lawful basis for this work

- [Contract with named client dated YYYY-MM-DD | Legitimate interest of the publisher in matter of public concern | Safeguarding | Regulated DD under [regime]]. All processing of personal data is limited to what is necessary for the stated question.
- Jurisdictions in scope: [list]
- Data-subject rights (where the subject is in the EU/UK): the subject may request access, rectification, or erasure of personal data held by [controller name] at [contact]. Requests to be handled per [controller]'s privacy notice dated [date].

Question answered

- [One plain sentence. The question the client asked, not the question that was easiest to answer.]

Attribution – shippable hedged sentence

- Write the finding as a single sentence a lawyer will pass.
Model:

"The strongest independent evidence points at [X] as the operator of

[Y] as of [date range]. Current control by [X] is not confirmed. No

first-party statement or self-link has been observed."

- Rung: [weak / moderate / strong / confirmed]

- What "confirmed" would require (and why we are not there):
[specific evidence – a first-party document, a self-link, an on-record

statement – that would move the finding one rung]

Independent signals supporting the attribution

For each: what it is, why it is independent of the others, the source and

its classification, and how it is preserved.

1. [signal]

- Source: [name], classification: primary / secondary / broker

- Preservation: [Wayback | Perma.cc | archive.today] ID [X], captured [ts]

- Artifact SHA-256: [hash], stored at [path]

- Independence: [why this is not the same fact as signals 2 and 3]

2. [signal] – as above

3. [signal] – as above

What we did not confirm (mandatory)

- [Specific fact 1 the reader might assume is confirmed and is not.]

- [Specific fact 2. Current control. Present-tense operation . Intent. Funding.

Any adjacent person. Any allegation the evidence does not carry.]

- Namesake / agency residual: [what alternative explanation survives, and

what would rule it out]

Out of scope / not attempted

- [List. This is not weakness, it is discipline. Contacts not made, hosts not probed, private data not sought, adjacent persons not investigated.]

Method summary

- [Three-line paragraph. No tool-name catalogue; the client does not need it. Name the categories of source used (registry records, certificate transparency, archived captures, public directories) so a reviewer can reason about coverage.]

Data provenance (per cited fact)

- Every claim in this report is listed in the case file with:
 - : source URL,
 - source classification (primary / secondary / broker), capture timestamp,
 - preservation snapshot ID, and SHA-256 of any downloaded artefact.
- Case file: [path or secure link], SHA-256 of the case-file manifest: [hash]

Examiner sign-off

- I confirm the findings above reflect the evidence in the case file as of the date of this report, that no material contradictory evidence has been omitted, and that I have named the limits of the work honestly.
- Signed: [examiner name], [date]
- Reviewed by: [second examiner or supervising analyst], [date]

attribution report

B.7 Language a lawyer will accept

Attribution sentences are where competent work most often gets rewritten by an editor or served with a letter. The pattern is nearly always the same: the finding was defensible; the sentence was not. Pair each rejected form with the accepted form and keep the pairing near your keyboard.

Attribution certainty - Rejected: *X operates the network.* -

Accepted: *The strongest independent evidence points at X as the operator of the network. Current control by X is not confirmed.*

Operator vs registrant - Rejected: *X owns the domains.* -

Accepted: *X was the registrant of two of the six domains in pre-2018 WHOIS records. Present-day registration is behind a privacy proxy; current ownership is not confirmed from public records.*

"Confirmed" vs "strong" - Rejected: *We confirmed that X runs the site.* - Accepted: *The finding sits at the "strong" rung on our internal ladder — three independent, non-collapsible public signals. It is not "confirmed"; a first-party self-link or on-record statement would be required for that.*

Individual vs agency - Rejected: *X is behind the campaign.* -

Accepted: *The operator-cluster evidence points at a single hand behind the network. The evidence does not distinguish between X acting alone and X acting on behalf of an organisation; the residual is named and not resolved.*

Present-tense claims from historical records - Rejected: *X controls the infrastructure.* - Accepted: *X was named in registry records dated [YYYY]. A domain can change hands without a public record; we make no claim about control after [YYYY] beyond what the signals in section [N] carry.*

The "we have not confirmed X" carve-out - Rejected (omission): [report lists only findings, no negative statement]. - Accepted: *We have not confirmed, and this report does not assert: (i) that X currently operates the network; (ii) any funding relationship; (iii) any intent behind the network's editorial choices; (iv) the involvement of any person named adjacent to X in the source material.*

Adjacent persons - Rejected: *X's associate Y is also involved.* - Accepted: *Y appears in one 2018 photograph with X. The photograph places them in the same room on one date. It carries no attribution weight for the network and Y is out of scope for this report.*

The rule under all of these: write the sentence you would be willing to read back, aloud, in a deposition. If any word in it goes further than the evidence in the case file, cut that word.

Appendix C — Red Lines

One card. Read every year.

Nothing in this book requires you to cross a red line. If a technique tempts you across one, you have picked the wrong technique. There is almost always a way to answer the legitimate question that does not need the shortcut.

The lines here are not exhaustive. They are the ones a Volume-3 reader is most likely to walk up to.

Access

- **No logging in that you are not entitled to.** Reading a public snapshot is fine. Guessing a password, replaying an old cookie, or using a leaked credential to access a live account is unauthorised access under most jurisdictions (CFAA in the US, Computer Misuse Act in the UK, GDPR Article 32 breach elsewhere).
- **No bypassing paywalls, robots files, or rate-limits by force.** A robots.txt is not a *legal* barrier in most jurisdictions, but abusing rate-limits or evading a service's explicit access control is a fast route to civil liability and, in some cases, criminal exposure. If the operator has explicitly asked automated readers to stay out, stay out.

- No probing hosts you enumerated. Finding an admin subdomain in CT is public-record work. Fetching that subdomain's login page is a request. Trying credentials on it is unauthorised access. Enumerate; do not touch.

Data provenance

- Understand what you buy. Commercial pDNS, historical WHOIS, and scan-index data all come from log aggregation somewhere. Some vendors obtained consent from the resolver operators, some did not. If you work in a regulated environment, ask.
- Do not use breach data to log in. A password from a leak proves the account existed. Using that password to access a live account is a crime almost everywhere. See Volume 2 Chapter 5.
- Cite every source in every report. Every claim gets a URL and a capture time. If a source is paid or under NDA, cite the vendor by name, not the internal identifier.

People

- The person is not the target. Volume 3 makes it very easy to slide from *the operator* into *the operator's family, their neighbours, their landlord, the child in the background of the geolocated photo*. Do not slide. Collect what answers the question and stop.

- Never geolocate a private home to publish it. Not even if it is unarguable. Not even in a leak-driven investigation. The public interest almost never requires the exact coordinates.
- Never chronolocate a person's movements to publish them. The pattern of a private life is not a public record. State the finding to the client if the case requires; do not publish it.
- Domestic-violence / stalking / high-risk carve-out. If you are asked to find someone who left their old life on purpose, decline. Full stop. There are professionals inside safeguarding services who do this properly. You are not one of them from a book.

Ethics of exposure

- Public interest is not the same as public curiosity. A newsroom's public-interest test asks: is the harm to the subject proportionate to the harm to society if this stays hidden? A curiosity test asks: can I find this out? The first justifies publication; the second never does.
- Minimal disclosure. Publish only what answers the question. If you found a wedding photo to confirm a residency, name the residency, not the wedding.
- You are not the last check. If your report goes to a client who might act on it — investigative reporter, corporate DD, safeguarding officer — expect them to add their own tests. That is not distrust; that is process.

The line you set for yourself

Above every rule: the one who knows where to stop, not the one who finds the most. If you cannot say in one plain sentence why you are following this thread, stop following it. Read every card in this appendix at the start of every case. It takes a minute. It saves careers.

Appendix D — Take It Back

Everything you just learned to read — closed against yourself.

Every chapter of this book taught you to read a trail. This appendix is the closing exercise: read your *own* trail, and close it where you can. Half of it is muscle memory once you have done it once; the other half is a maintenance habit.

Do this twice a year. Whenever you get uneasy — a new job, a public appearance, a move to a new city, a personal safety concern — do it again the same week.

The archives (Ch1)

- Search the Wayback Machine for every domain you have ever owned. Read the timeline. If something is embarrassing, request removal from Wayback (they honour some requests). If something is worse, understand: request-then-forget is not enough — the third-party mirrors may still carry it.
- Quote-search rare phrases you used to write. See what third parties preserved. Anything that surprises you is a signal for the next section.

The deleted profiles (Ch2)

- List every account you have ever created. Old MySpace, ICQ, Xanga, LiveJournal, defunct forums, that gaming clan wiki from 2011. Search each by handle and by email.

- **Reverse-image** the avatar you used in each. A reused avatar across two accounts you thought were separated is a shortcut you left for anyone.
- **Read the friend graph residue.** On the platforms that still show mutual friends, see what a stranger could infer about your network.
- **Data-broker opt-outs.** People-search sites keep snapshots for years. Every US-based site has an opt-out page — most are hostile, all work eventually. There are consolidated services that will do this on your behalf; not an endorsement, just a note.

The photographs (Ch3 and Ch4)

- **Strip EXIF** from anything you post. Most social platforms strip it now, but blogs and cloud shares often do not.
- **Read your own posted photos as a geolocator would.** House number in the corner. Reflection in a mirror. Distinctive tree. Take them down or crop them out.
- **Do the same for chronolocation.** A pattern of always-Tuesday posts from the same café places you in a habit. If the habit matters for your safety, break it.
- **Special care for children.** A photo of a school badge, a school-run route, a uniform in a family shot — any of these is a chronolocation of a child's Tuesday morning. Do not post them.

The domains (Ch5)

- **Read your own live WHOIS.** For every domain you own, see what is public. If your registrant email predates 2018, historical WHOIS still has it — a professional adversary will find it there.
- **Read your own DNS.** MX records leak your mail provider; TXT records leak your Google Workspace or Microsoft 365 tenant; SPF leaks every sender you allow. None of these are secret; understanding what they betray is what matters.
- **Move personal domains to privacy-enabled registration** if you have not. The 2011 leak is done — but 2026 forward can be quiet.

The certificates (Ch6)

- **Search CT for your own domains.** Read every SAN in every cert. If a staging environment you forgot about is there, that hostname is public forever. Take the host offline if it is dead; move it to a private CA (out of CT) if it is live and truly internal.
- **Beware wildcard certs.** A wildcard SAN protects everything, but the *fact* that you have wildcards on a domain tells a professional you have subdomains worth naming.

The infrastructure fingerprints (Ch7 and Ch8)

- **Inventory your operator-fingerprints.** Do every personal site you run carry the same favicon? The same GA measurement ID? The same tag manager? If yes, and the sites should not be linked, un-link them.
- **Vary your tech stack across identities that must not connect.** Two sites you meant to keep separate should not share the same paid WordPress theme, the same avatar file, the same CSS class names.
- **Check your JARM.** If you self-host, run the JARM tool against your own edge. If your fingerprint is rare, an adversary can find every other host you run with the same setup.

The habits

- **Password manager, 2FA on everything.** This is a Volume-1 habit; still the single largest reduction in your attack surface. Not optional.
- **Contact-syncing off.** Bidirectional PYMK will link your research identity to your real one the first time both apps read your address book. See Volume 2.
- **Compartmentalise.** A research browser is not your personal browser. A research phone is not your personal phone. A research handle is not any handle you have ever used before.

- When someone is already looking for you. If you know or suspect you are being surveilled or stalked, do not run this checklist on your normal accounts — the pattern of the sudden hardening is itself a signal. Change everything at once, from a clean device, in one session. Or ask a safeguarding professional; there is no shame in it.

The line

You will not close every hole. You do not need to. You need enough friction that a casual adversary quits, and enough discipline that a serious adversary has to work. Read every chapter of this book as an instruction manual for your own defence. Then close what you can, live with what you cannot, and get back to the work.

Glossary

Plain-language definitions of every term this book uses more than once. Flip to it any time.

- **AdSense ID / pub-id** — Google's publisher identifier for ad revenue, formatted pub-1234567890123456. When the same pub-id shows up on two different sites, the same wallet is being paid — which is why it is one of the strongest attribution signals on the ladder.
- **Anchor** — the one fixed known thing you start an investigation from. A domain, a photo, a handle, a phone number. Everything else in the case is reached by hopping away from the anchor.
- **Attribution** — the third move in the loop. Turning "this belongs to some person" into "this belongs to *that* person" — naming the operator behind the trace.
- **Autonomous System (AS)** — a chunk of the internet under one operator's routing control, identified by an AS number like AS13335. Useful for grouping IPs by who actually runs them, rather than by whose name is on the WHOIS.
- **Banner** — the short string a service emits the moment you open a connection to it. Version numbers, software names, sometimes hostnames. Free identification, no login required.

- **Capture time** — the timestamp of when a snapshot was taken, not the time you are looking at it. Every archived page, every scan, every screenshot has one. Cite it, or the evidence rots quietly.
- **Certificate Transparency (CT)** — the public append-only log system where every trusted TLS certificate is recorded. Made subdomain enumeration trivial and killed the idea that a hostname could stay secret.
- **Chronolocation** — dating an image from what is in the frame. Sun angle, foliage, shop signage, snow, roadworks, a poster with a date. Answers "*when*" rather than "*where*".
- **clientTransferProhibited** — an EPP status code on a domain that stops it being transferred to another registrar without unlocking. It is an anti-hijack lock, nothing more. It does not hide the registrant, and its presence tells you nothing about the owner.
- **CNAME** — a DNS record that points one hostname at another hostname instead of at an IP. `www.example.com` CNAME to `example.com` is the classic. Follow the chain — the alias target often reveals a hosting tenant.
- **Confidence ladder** — the four rungs this book grades signals on: weak, moderate, strong, confirmed. A weak signal is a hint; a confirmed one is a self-link or a receipt. Everything in a report should be labelled somewhere on the ladder.

- **DKIM** — a DNS-published cryptographic signature system that proves a piece of mail was really sent by someone authorised for that domain. The `s1._domainkey` TXT records are DKIM.
- **DMARC** — a DNS policy record that tells receiving mail servers what to do when SPF or DKIM checks fail — deliver, quarantine, or reject. Published at `_dmarc.example.com`.
- **DNS** — the phonebook that turns hostnames like `example.com` into IP addresses. Almost every technique in Vol.3 either queries DNS or reads its historical shadow.
- **DoH / DoT** — DNS-over-HTTPS and DNS-over-TLS. Two ways of encrypting DNS lookups so a network watcher cannot see which hostnames you resolved. Matters for research hygiene, not for what you can query.
- **Dork** — a search-engine query built out of operators — `site:`, `inurl:`, `filetype:`, `intitle:`. A well-formed dork turns a search engine into a targeted index.
- **EXIF** — the metadata block inside a photo file. Camera make and model, lens, shutter, sometimes GPS coordinates and the phone's serial. Strip it from anything you publish; read it on anything you receive.
- **Exposure** — the first move in the loop: a trace of the subject that is already sitting in the open. You do not create exposures, you notice them.

- **Favicon** — the small icon a browser shows in the tab for a site. Usually served from `/favicon.ico`. Reused across a subject's properties far more often than they realise.
- **Favicon hash** — a MurmurHash3 of the favicon file's raw bytes, base64-encoded. Scan indexes let you search by this hash — one favicon can identify every host that serves the same file.
- **Fingerprint** — a repeatable identifier derived from a configuration. Same setup, same fingerprint, across different IPs and hostnames. JARM, JA3, favicon hashes are all fingerprints.
- **FOAF (friend-of-a-friend)** — a technique that walks through other people's networks to reach the subject. The subject may lock their profile down; their cousin's public wedding album does not.
- **GA4 (Google Analytics 4)** — Google's current analytics beacon, identified by a measurement ID shaped `G-XXXXXXXX`. Two sites carrying the same measurement ID share an owner.
- **GDPR** — the EU privacy regulation that made registrars redact personal WHOIS data in 2018. It is the reason "why is this WHOIS blank" is almost never a sinister answer.
- **Geolocation** — identifying *where* an image was taken from what is visible in it. Skyline, hills, road furniture, language on signage. Answers "where"; chronolocation answers "when".

- **Historical WHOIS** — snapshots of WHOIS records from before the current one — often from before GDPR redaction. A domain that is anonymous today may have carried a full name and email in 2015.
- **Hop / pivot** — the second move in the loop: turning one trace into the next. From a domain to its cert to another domain. From a handle to an email to a second handle. Every case is a chain of hops.
- **HTTP header** — the metadata sent with an HTTP request or response — Server, Set-Cookie, X-Powered-By, and dozens more. Read them; they leak stack, tenant, and sometimes internal hostnames.
- **ICANN** — the non-profit body that governs domain-name policy at the top of the tree. Sets the rules registrars and registries have to follow.
- **Independence (of signals)** — two signals only count as two votes if they came from separate sources. Three pages of the same scraper repeating one leak is one signal, not three.
- **JARM** — a 62-character fingerprint of how a TLS server responds to a set of probe handshakes. Two servers with the same JARM are almost certainly running the same stack, configured the same way.
- **JA3 / JA4 / JA4S** — fingerprint formats for the *client* side of a TLS handshake (JA3, JA4) and the *server* side (JA4S). Used to group traffic and infrastructure by the software

behind them.

- **Leak / breach** — a dump of user data made public without the platform's consent. Historically the same word is used for both; treat them as evidence, cite the source and date, and never republish raw records.
- **MurmurHash3** — a fast non-cryptographic hash function. Used by scan indexes to fingerprint favicons; not used for anything security-sensitive.
- **MX record** — a DNS record that names the mail server responsible for a domain. Tells you whether a company runs its own mail, uses Google Workspace, Microsoft 365, or something rarer.
- **Nameserver (NS)** — the DNS server that actually holds the records for a domain. Two domains sharing an obscure nameserver often share an operator.
- **Passive DNS (pDNS)** — an archive of historical DNS resolutions, collected by public resolvers over the years. Lets you ask "what did this hostname resolve to in March 2019" long after the record has changed.
- **Phenology** — the seasonal cycle of plants. Bud break, leaf-out, flowering, senescence. In an outdoor photo, the state of the vegetation narrows the capture window to a few weeks.
- **Precertificate** — a version of a TLS certificate submitted to CT logs *before* the final leaf is signed. It is what makes CT-based subdomain discovery possible essentially in real

time.

- **Provenance** — where a piece of evidence came from and how it reached you. Every claim in a report should carry its provenance; a claim without provenance cannot be defended.
- **Punycode / ACE form** — the ASCII encoding DNS uses for internationalised domain names, prefixed xn--. xn--80akhbyknj4f is испытание in DNS wire form. Watch for lookalikes.
- **Ranking (on the ladder)** — the act of assigning weak, moderate, strong, or confirmed to a signal before you write it up. Rank as you collect; do not defer.
- **Redaction (WHOIS)** — the post-GDPR practice of hiding registrant name, email, and address behind placeholder text or a privacy-service alias. Standard, not suspicious.
- **Registrar vs Registry** — the registrar is the vendor you buy the domain from (GoDaddy, Namecheap, Cloudflare). The registry is the authority that runs the TLD itself (Verisign for .com, Nominet for .uk). Different roles, often confused.
- **Research account** — see *sock account*.
- **Research browser** — see *working browser vs research browser*.
- **Reverse-IP / reverse-DNS** — asking "what other hostnames resolved to this IP" (reverse-IP) or "what hostname does this IP claim in a PTR record" (reverse-DNS). Two

different queries, both useful, easy to mix up.

- **RFC 6962** — the specification that defines Certificate Transparency logs. Cite it when someone claims CT is optional; it is not.
- **robots.txt** — a text file at `/robots.txt` where a site politely asks crawlers not to index certain paths. It is advisory. The paths it names are often the most interesting ones on the site.
- **SAN (Subject Alternative Name)** — the list of hostnames a single TLS certificate is valid for. A cert with `example.com`, `dev.example.com`, `staging.internal.example.com` in its SAN just told you three subdomains for free.
- **Scan index** — a searchable catalogue of every host visible on the public internet, kept by services such as Shodan, Censys, or FOFA. You query it; you do not scan yourself.
- **SCT (Signed Certificate Timestamp)** — a receipt from a CT log proving a certificate was submitted to it, embedded in the cert itself. It is what browsers check to enforce transparency.
- **Self-link** — a first-party assertion of ownership. The subject's own website linking to the handle. The strongest signal on the ladder for attribution.
- **Snapshot** — a saved copy of a page at a specific moment. Wayback Machine snapshots, archive.today snapshots, your own screenshots. Always record the capture time alongside.

- **Sock account** — a clean investigative identity separate from the researcher's real one. Also called a research account. Used for anything that involves logging in or interacting.
- **SPF** — a DNS TXT record that lists which mail servers are allowed to send mail claiming to be from a domain. Reveals mail infrastructure and, sometimes, third-party tenants.
- **Subdomain** — a name to the left of the base domain. `dev.example.com`, `mail.example.com`. CT logs, DNS brute-forcing, and passive DNS are how you enumerate them.
- **TLS / SSL** — the cryptographic layer under HTTPS. TLS is the current name; SSL is the older one; people use them interchangeably. Every fingerprintable handshake in this book lives here.
- **TLD** — the top-level domain — the last label of a hostname. `.com`, `.eu`, `.io`, `.pφ`.
- **TXT record** — a free-form DNS record. Holds SPF, DKIM, DMARC, and every SaaS verification token a domain has ever set up. One of the leakiest records in DNS.
- **UA-ID (legacy Google Analytics)** — the older Google Analytics identifier, shaped `UA-XXXXXX-Y`. Deprecated in 2023 but still visible in archived pages; still valid for historical attribution.
- **Verification token** — a random string a third-party service asks you to publish in DNS to prove you own the domain.

TXT records are stuffed with them. Each one names a service the operator uses.

- **Wayback Machine** — the Internet Archive's web-page history service at web.archive.org. The default first stop for "what did this page look like before".
- **WHOIS** — the public record of who registered a domain, when, through which registrar, until when. Post-2018 the personal fields are usually redacted; the operational fields are not.
- **Wildcard cert** — a TLS certificate whose SAN is *.example.com, valid for every immediate subdomain. Convenient for the operator and quietly hostile to enumeration.
- **Working browser vs research browser** — your everyday Chrome, signed into your real accounts, is your working browser. The clean profile with no cookies, no sync, no extensions you did not choose is your research browser. Never confuse them.

If a word this book uses more than twice is not here, that is a bug — flag it. The glossary grows with the reader.